TOWARDS MISSION-DRIVEN SUMMARIZATION FOR TACTICAL NETWORKS

Draft of July 7, 2019 at 22 : 53

BY

JONGDEOG LEE

DISSERTATION

Submitted in partial fulfillment of the requirements
for the degree of Doctor of Philosophy in Computer Science
in the Graduate College of the
University of Illinois at Urbana-Champaign, 2019

Urbana, Illinois

Doctoral Committee:

Professor Tarek Abdelzaher, Chair
Professor Klara Nahrstedt
Professor Robin Kravets
Professor Ramesh Govindan

ABSTRACT

In October 2017, the U.S. Army established six army modernization priorities to build a more lethal force. Considering the increasing importance of information superiority, it is not surprising that the *Army Network* has been selected as one of the six modernization priorities. One of the main reasons behind this decision is that, in the future, battlespace will become more contested by an adversary's sophisticated attacks on network infrastructure. Indeed, modern jamming threats are significantly different than in the 1980s, and the military is looking for a solution to support communication in this contested environment.

There are two directions to address the problem. The first approach would be to make the network systems more robust and reliable, such as by adding in routing and security solutions. Another approach is more efficient network resource management that includes minimizing the transmitted data volume to save network bandwidth consumption. There are two representative techniques in this case: compression and summarization. Compression represents the original data with fewer bytes and is widely used for multi-media data. Summarization, or sampling, selects only a subset of the data from the whole set.

In the current thesis, we propose a new mission-driven summarization layer that understands the application needs and optimizes the network resources using the sampling technique. This work is similar to the existing summarization efforts but is distinguished by three key factors. First, its nature is mission driven. This layer offers declarative application program interfaces to the applications to analyze the application needs so that the layer can flexibly manages network resources on behalf of applications. This is different than the traditional approach, such as time-driven and event-driven approaches where applications need to micromanage network resources. The second difference is generality. Rather than devising the optimized algorithm for a particular application as in state-of-the-art researches, this work is designed to provide summarization as a service. Hence, we focus on an easy-to-port feature across different application contexts. This generality calls for simplicity by observing a fundamental trade-off between generality and complexity. That is, more sophisticated solutions tend to have more assumptions and are more difficult to apply to the wider application domain. The proposed solution is designed to be simple and widely applicable yet effective.

I dedicate this thesis to my father, who is fighting against cancer.

ACKNOWLEDGMENTS

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION . . . 1
1.1 Tactical Networks . . . 1
1.2 Contributions . . . 3
1.3 Impact . . . 4
1.4 Thesis Organization . . . 5

CHAPTER 2 SYSTEM OVERVIEW . . . 8
2.1 Decision-driven Resource Management . . . 8
2.2 System Architecture . . . 10

CHAPTER 3 LOGICAL SHORT-CIRCUITING BASED SUMMARIZATION . . . 20
3.1 Introduction . . . 20
3.2 Design . . . 22
3.3 Implementation . . . 25
3.4 Domain of Applicability . . . 28
3.5 Evaluation . . . 29
3.6 Conclusions . . . 38

CHAPTER 4 REDUNDANCY BASED SUMMARIZATION . . . 39
4.1 Introduction . . . 39
4.2 Design and Implementation . . . 41
4.3 Domain of Applicability . . . 51
4.4 Evaluation . . . 51
4.5 Conclusions . . . 56

CHAPTER 5 AUTOMATED DATA NAMING SERVICE . . . 59
5.1 Introduction . . . 59
5.2 Naming, Retrieval Order, and Summarization . . . 60
5.3 Design . . . 63
5.4 Implementation . . . 69
5.5 Domain of Applicability . . . 70
5.6 Evaluation . . . 70
5.7 Conclusions . . . 74

CHAPTER 6 SUMMARIZATION FOR STREAM CONTENTS . . . 75
6.1 Introduction . . . 75
6.2 Design . . . 77
6.3 Implementation . . . 81
6.4 Domain of Applicability . . . 83
6.5 Evaluation . . . 84
6.6 Conclusions . . . 92

CHAPTER 7 APPLICATION EXAMPLES 94
7.1 Target Tracking 94
7.2 Situation Awareness 106

CHAPTER 8 RELATED WORK 112
8.1 Data Collection Protocols 112
8.2 Decision-centric Management 114
8.3 Redundancy Minimization 116
8.4 Target Tracking 117

CHAPTER 9 CONCLUSIONS 118

REFERENCES 121

CHAPTER 1: INTRODUCTION

Since Admiral William Owens introduced the concept of network-centric warfare in 1996, information superiority through computer networks has been considered one of the most critical factors to win modern battles. This paradigm relies on computer network technologies to enable reliable and robust communication among combat units. The implementation of the model is the DoD Architecture Framework (DoDAF), which was previously called C4ISR. The system promises information superiority to guarantee that friendly forces can observe, orient, decide, and act (OODA) faster than enemies. This OODA loop is an essential concept in decision-making. The effectiveness of the network systems has been shown in many wars, including the Gulf War and Afghanistan War. The importance of computer network systems on the battlefield has only increased over time. In 2017, the U.S. Army selected the top six modernization priorities and decided to invest $1.1 billion in them [1]. It is not surprising that the *Army Network* is one of the six priorities.

1.1 TACTICAL NETWORKS

The overall architecture of the tactical networks is shown in Figure 1.1. Tier-1 is the lowest echelon that supports the ground-level communication from a company commander down to the soldiers. Tier-2 provides connection among companies, battalions, and brigades by aerial relay. Win-T (Warfighter Information Network - Tactical) uses satellite links to share battlefield information with battalions and higher-level units. Every unit from the squad to Pentagon jointly visualizes the battlefield by sharing the information through the tactical networks. In brief, the lower echelons of the tactical networks are small scale and use wireless communication, and the higher echelons are large scale and use wired networks.

As more and more operations begin to rely on the networks systems, however, network resources such as link bandwidth become more limited. More importantly, the network assets become highly prioritized targets of the enemy in an attempt to prevent and deny information sharing among friendly forces. The representative example is a jamming attack that disrupts the existing wireless communications by interfering with wireless signals. The sophistication of network attacks is increasing, making it harder to defend over time; hence, it is reasonable to assume that the network resources on the future battlefields will be highly contested.

Military communication experts look for solutions to support reliable communication among friendly forces through tactical networks. The related work for this problem can

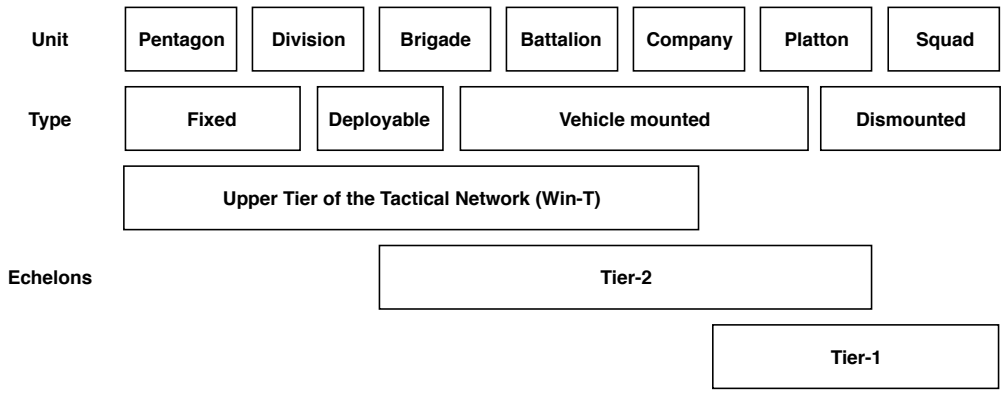


Figure 1.1: Overall architecture of the tactical networks [2]

be broadly categorized into two categories. The first is to make the network system itself more robust and secure. The solutions are mainly platform-specific or threat-specific because it would be difficult to generalize the solution. For example, self-healing multi-path routing can be a solution to make the MANET (mobile ad-hoc network) systems more resilient to link failures [3, 4, 5]. One might suggest making the bottleneck node, such as satellites, more secure from denial-of-service attacks because this can be a single source of failure [6, 7, 8]. Several works proposed anti-jamming solutions, as introduced in [9, 10, 11]. The challenge with this type of solution is efficiency. Upgrading any legacy system is expensive. The solutions that minimize conflicts with legacy systems would be the most productive way to support communication in the contested network when considering the scale of the current tactical networks.

Another type of solution is efficient network resource management. Here, the primary resource that we want to manage is network bandwidth. Therefore, one way would be to reduce the transmitted data volume across the network. Because data would be naturally overloaded in the contested environment, this solution tries to consume the least amount of resources without missing any application requirements. There are two common methods to achieve data volume reduction: compression and summarization. The former technique uses encoded information to represent the original data and hence use fewer bits. The representative example would be video encoding and decoding, such as MP4. However, it requires significant time to compress and decompress content without support from expensive hardware.

Summarization, or sampling, selects a subset of data, called samples, from the whole data set. It is more efficient than compression in terms of time complexity because sample selection does not require expensive operations like compression does. The rationale behind this idea is that applications seldom need all the available data. Therefore, there is always room to optimize the network resources by requesting only necessary information. There

are many research papers on using summarization to minimize network resource usage, but they are all application specific [12, 13, 14, 15].

1.2 CONTRIBUTIONS

In the current work, we propose a new decision-driven summarization layer based on the sampling technique that understands mission needs and reduces the data volume accordingly. This thin layer between applications and network protocols functions as a middleware that minimizes the network resource consumption while meeting the application needs. This work is close to the existing summarization approaches, yet it is different in the following three ways.

The first key difference is the mission-driven nature of this summarization layer. This work implements the command-by-intent paradigm, where a commander specifies only the high-level purpose of the mission, such as the end state after the task, instead of giving individual actions to the subordinate units. Command-by-intent is a more advanced military doctrine than command-by-direct. The benefit of command-by-intent is to maximize autonomy for the subordinate units so that they can flexibly make the best action that suits the current situation. Similarly, the mission-driven summarization layer optimizes the network resources on behalf of the applications by considering the current network conditions, content, or source availability, along with other constraints. Therefore, this new layer lightens the burden of applications by automating the process and providing it as a service.

The second distinguishing property is generality. The current work intends to provide general-purpose summarization services to applications, especially for the military. Therefore, a tailor-made algorithm for a particular application may outperform the algorithm proposed in the present work. However, the novelty here lies in a generalizable summarization service, not in optimality. This work can be ported across various application contexts by simple changes in the metadata. For instance, target tracking and route reconnaissance are completely different context applications, but we can develop them on the new layer by providing different decision rules. Also, we can achieve temporal situational awareness or spatial situational awareness by merely changing the content namespace hierarchy. On top of this, the current work furnishes customizable components to achieve context-dependent optimization without losing generality.

We can usually observe a trade-off between solution complexity and generality. That is, more complex solutions tend to have more assumptions about their use and environment, making them less applicable to a wider range application domain. For instance, PhotoNet provides a sampling algorithm for the image set based on color histogram and KL diver-

gence [12], yet it is not applicable to other types of data sets such as tweets. In this context, the proposed solutions in the current work lean to simplicity to not lose generality. More complicated algorithms would only support a limited number of applications, which does not match with the goal of the current thesis.

Indeed, the properties of simplicity and general applicability have produced many successes in technical transitions to several other mission-centric contexts, as will be described in Section 1.3. These successes prove the advantages of this approach. Again, the current work provides a mission-driven summarization as a general service in a simple yet effective manner.

1.3 IMPACT

The present work combines two separate pieces that arose from different projects. Athena is software that is part of the NS-CTA (Network Science Collaborative Technology Alliance) project that involves the U.S. Army Research Laboratory (ARL), other government researchers, and a consortium of 17 university and industry research organizations. The NS-CTA project has been successful over 10 years, having more than 1,000 publications. Athena has been selected as one of the final six demos in the project. Because our primary interests are to support military applications, the works have been mainly published in military-relevant conferences such as MILCOM and FUSION.

The other main piece of the work is InfoMax. The project was a part of Future Internet Architecture funded by the National Science Foundation. The main focus of the project was to design and implement named-data networks (NDN). InfoMax was designed as a producer/consumer protocol that runs on top of the NDN stacks to provide a summarization service. Espresso was developed as a subsidiary tool for InfoMax to automate data naming. InfoMax and Espresso were published in IC3N and SECON, respectively. The protocols were initially implemented in C++. Later, we extended the Swift version of InfoMax for the iOS application, as shown in Section 7.2. This application was invited to be a demo in SECON. Pub/Sub-Sum, an extension of InfoMax for stream sources, is being submitted to a conference, and we are currently waiting for the result.

We collect all of the proposed software and provide them as a single package in Python. The implementation has been successfully tested with many other tools. The code was run on DAVC [16], the clustering management system developed by ARL, to evaluate the performance of the target tracking application. Also, one of the experiments was conducted on top of EMANE [17], the mobile ad-hoc network emulator tool developed by the Naval Research Laboratory (NRL). MUCAN is the disruption-tolerant network solution de-

veloped by the Raytheon company and provides the pub/sub interface. Athena could be successfully plugged into MUCAN with minor modifications. Recently, Athena was tested on Minimega [18], a tool for launching and managing virtual machines developed by Sandia National Laboratory (SNL). The prior results indicate that Athena is highly compatible with other platforms because it is designed as middleware between applications and communication subsystems. This proven compatibility would be a powerful advantage to make it a candidate solution for the tactical communication on a contested battlefield.

1.4 THESIS ORGANIZATION

In the current thesis, we design, implement, and evaluate the mission-driven summarization layer. Chapter 2 envisions the overall architecture of the proposed layer while giving an explanation of each component. This layer is called Athena. We anticipate there are four types of users in Athena. Commanders issue decision queries to the system. Staff officers define and provide decision rules for the anticipated decisions; these decision rules are called the working set. Human soldiers or autonomous sensors such as drones produce data by observing the physical situations on the battlefield. These sensor data can be fed to human analysts to extract semantics.

Athena supports these users by providing three types of APIs: decision maker, sensor, and annotator. The decision-maker node translates the received decision rule into the decision tree and computes the best data retrieval order to achieve the goal while minimizing the expected cost. This process is described in Chapter 3. The sensor node receives data from human soldiers or sensors and stores these data in the local storage. Then, it offers data on demand over the network. In brief, a decision-maker node is a data consumer, while a sensor node is a data producer. The annotator is a data producer and consumer at the same time because it retrieves data from sensors or from outside of the system to extract semantics and produce them.

Chapter 3 describes the mission-driven nature of Athena and the efficient data retrieval scheduling for optimal network resource usage. Because the command-by-intent paradigm allows the subordinate units to have maximum autonomy that follow the commander's intent, the proposed idea is to have the mission-driven summarization layer find the best data retrieval schedule on behalf of applications to meet the requirements while optimizing bandwidth usage. It utilizes a short-circuiting evaluation, a widely used technique in compilers, to minimize the data retrieval. We implement this layer, called *Athena*, and evaluate its performance in the context of a NATO exercise. In this scenario, the mission is to capture a particular target using city cameras, and the company uses wireless communication for

exchanging data. The evaluation indicates that the data scheduler can significantly reduce bandwidth consumption by requesting the image only from the relevant cameras. We could further optimize the network resource consumption based on the accurate prediction of the target movement.

In Athena, the decision maker needs evidence to support the decision by collecting data from sensors. Sensor data may have a semantic overlap with other sensors, such as images, for the same target but from the different viewpoints. In this case, we might want to minimize redundancy among the transmitted data to maximize information utility while keeping at a minimal bandwidth. Chapter 4 describes redundancy minimization. It assumes that data items are partially redundant with each other; hence, there is a certain hierarchy among the data, and similar content is grouped together. We design and implement this protocol, called *InfoMax*. In this chapter, the underlying network system is assumed to be a named-data network (NDN), the representative instance of information-centric networks. The reason to choose an NDN is that we could exploit hierarchical UNIX-like data names to approximate content similarity by assuming that similar content shares a longer name prefix. Also, an NDN is designed for an Internet scale of networks. We believe that redundancy-based summarization is more effective for big data applications; hence, it would be reasonable to test the protocol on a large-scale network such as the high echelons of the tactical networks. In the evaluation, we use the nation-wide NDN testbed while assuming that combat units in the different theaters retrieve information from the command center.

Chapter 5 introduces *Espresso*, the automated data-naming agent that supports the InfoMax protocol. As mentioned above, InfoMax assumes that data are partially redundant, and similar data are grouped. InfoMax realizes that assumption. For a given data set, it first converts every data item into a vector. Then, it constructs a graph, where the nodes are data, and edges are the computed distance among the nodes based on a given application-specific distance metric. Finally, Espresso applies the hierarchical clustering algorithm to gather similar contents together and assign a hierarchical name to an individual object. Besides the automatic namespace construction, the current work extends the data retrieval scheduling algorithm of InfoMax to meet various sampling needs. For example, the commander may want to sample the reports that are proportional to the urgency of the situation. In this sense, the sampling order can be customizable so that various needs can be fulfilled. In this chapter, we mainly use a data set from Twitter. Twitter data are not representative of a battlefield. However, a battlefield's data is voluminous and noisy, two properties that are shared by Twitter data. We assume that tweets are human soldier reports, and the goal of this application would be to understand the overall situation quickly.

Pub/Sub-Sum is described in Chapter 6 and extends InfoMax in two ways. First, this

work provides the summarization service for streaming contents, whereas InfoMax assumes that the data set is static, where contents are not updated after starting transmission. Second, it supports the many-to-many communication pattern between the producer and consumer, whereas InfoMax is designed mainly for a single producer and many consumers. In addition, this chapter proposes a novel way of congestion control that adaptively adjusts the summarization level depending on the network congestion status. That is, if the network becomes congested, only non-redundant data will be transmitted to reduce the data volume introduced to the network. This work is implemented and tested on Raspberry Pis using Pi cameras.

In Chapter 7, we provide two popular types of military applications. The first application is target tracking and runs in the Athena context. The decision maker requests images from sensors to localize the given target. The app uses a video data set collected by the USC campus camera system. We apply a vision technique for the video set to analyze mobile vehicle patterns. Based on the analysis, the application predicts the next camera position of the vehicle once it disappears from the current camera view. The second application is the automatic headline news maker, which is based on collected tweets under specified topics. The app can be considered the civilian version of situation awareness monitoring urban social events.

Chapter 8 reviews the related work of this research, looking at four categories: data collection protocol, decision-driven resource management, redundancy minimization, and target tracking. Finally, Chapter 9 concludes the thesis.

CHAPTER 2: SYSTEM OVERVIEW

The chapter introduces a new direction in quality-of-service-aware networked sensing that designs communication protocols and scheduling policies for data delivery that are optimized specifically for decision needs. The work complements present decision monitoring and support tools and falls in the larger framework of decision-driven resource management. A hallmark of the new protocols is that they are aware of the inference structure used to arrive at decisions (from logical predicates), as well as the data (and data quality) that needs to be furnished to successfully evaluate the unknowns on which these decisions are based. Such protocols can therefore anticipate and deliver precisely the right data, at the right level of quality, from the right sources, at the right time, to enable valid and timely decisions at minimum cost to the underlying network. This chapter presents the used decision model and protocol design philosophy, and describes a novel system, called Athena, that is the first to embody the aforementioned data delivery paradigm.

2.1 DECISION-DRIVEN RESOURCE MANAGEMENT

The concepts of decision-driven resource management, decision-centric information monitoring, and decision support systems have been studied extensively in prior literature. As discussed later in related work, the system presented in this chapter complements work on existing decision-centric monitoring and support tools, and augments the general category of decision-driven resource management frameworks. It is distinguished in adapting decision-driven resource management to the realm of runtime network data management in the context of distributed sensing and IoT applications. This is as opposed to topics such as managing software architecture, human resources, financial assets, or physical organizations. The application of decision-driven resource management to runtime networking leads to novel challenges and research questions in the design of network communication protocols and communication scheduling policies, presented in this chapter.

This chapter describes a novel model and system for anticipatory information delivery in mission-centric applications, where communication protocols and scheduling policies for data delivery are optimized specifically for meeting decision needs. We envision an environment such as an IoT-enabled smart city [19, 20], where a cyber-physical system [21] is deployed to assist a team in performing a first-response mission in the aftermath of a natural disaster. Mission execution is affected by answers to a set of questions that first-responders might ask. For example, what are the affected areas? What is the extent of damage in each area? Who

are the affected individuals? Where are the survivors? Are there injuries? Who is in most need for help? These and similar questions drive the decisions of the mission commander in allocating human and physical resources to different rescue efforts. A computing system familiar with similar missions can anticipate the types of decisions the commander must make, as well as the nature of mission-relevant information that affects these decisions. The system can then collect such information ahead of time and update it as the situation changes. In recent work, this information delivery paradigm was called *decision-driven execution* [22]. It re-thinks scheduling and communication to consider application models that are more pertinent to mission-driven QoS-aware CPS/IoT systems. We review this recent work, describe its embodiment in a system called Athena, and present significantly expanded evaluation results that describe its advantages over a set of baselines.

Decision-centric anticipatory information delivery is motivated by visions of an instrumentation-rich world where a large number of sensors and other information sources may supply data relevant to an application [23, 24]. There might not exist an *a priori* structure (such as fixed control loops) that statically dictates where each sensor should send its data. Rather, information sources will generally be connected to shared media, allowing flexible reconfiguration to support different (both one-off and repetitive) tasks. A challenge is to configure data collection from appropriate sources in order to meet specific application information needs while minimizing cost. For example, consider the myriad of sensing, storage, and computing devices that comprise a megacity's cyber-infrastructure. Many sensors will be embedded in the physical environment. Different application tasks may need subsets of these devices. A sensor is activated when the user needs to perform a (decision) task that needs data from that sensor. In other words, resource consumption is decision-driven.

At a higher level, one can view decision-centric anticipatory information delivery as a way for data fusion systems to exert *initiative* in anticipating and offering mission-relevant information to mission commanders. For the system to exert initiative, it must know the nature of the mission (i.e., mission-relevant decisions that need to be made) and the unknowns that need to be determined in making mission-relevant decisions. The paradigm offers an exciting foundation for rethinking resource management as a process of arbitrating the acquisition and movement of data among processing components to meet decision needs. Such arbitration is guided by a novel set of questions: What data would be more relevant for making a decision? Which sensors are most appropriate for collecting such data? When should the data be collected to meet freshness needs of the decision? What data should be cached in the distributed system, and which nodes should store it to support aggregate decision needs most efficiently? At least three conditions must be met by the resource management algorithms; (i) the data collected for making a decision must be of sufficient quality to support

the decision, (ii) it should be acquired sufficiently recently such that it is not stale by the time the decision is made, and (iii) the decision made based on collected data must meet the relevant decision deadline. A distributed system that supports the decision-driven paradigm must optimize communication, storage, and scheduling to meet these constraints. In this thesis, 3 describes the first problem and solution while leaving the real-time constraints in future work. Rather, we more focus on the holistic system design and implementation, called Athena.

2.2 SYSTEM ARCHITECTURE

A data fusion system that supports decision-centric anticipatory information delivery may simultaneously retrieve data for multiple missions. An example mission might be to track a specified target (such as a robbery getaway vehicle or an escaped fugitive) in a city using deployed sensors and security cameras, or to rescue a group of individuals trapped in the aftermath of a natural disaster. Execution of the mission starts with collecting information relevant to the mission. We assume that such collection occurs over a tactical network of limited resources, which is often the case in such missions as disaster response (where much infrastructure has been destroyed leading to resource shortages). The primary purpose of the data collection system is to conserve the resources expended on data collection while at the same time collecting enough data to support the mission commander or decision maker. In the context of a mission, the commander or decision-maker needs information to make decisions. In a rescue mission, an example decision might be: which evacuation path to follow in order to rescue the disaster survivors? In a tracking mission a decision might be: which sensors to turn on in order to track the target? An interesting trade-off is involved. Namely, collecting more information may overwhelm the underlying resource-constrained network, leading to delays that may negatively impact the ability of the user to obtain information on time and hence their ability to make timely decisions. In contrast, collecting inadequate information might impede one's ability to make a correct decision. For example, if one does not collect enough information on the health conditions of various roads in the aftermath of a large natural disaster, one might attempt to evacuate the survivors over a road that ends up blocked or otherwise unsuitable. The key is therefore to collect just the right amount of information to make a decision. With that in minds, several levels of indirection allow the system to optimize information delivery for the respective missions, as follows:

- *The decision working set:* For each mission, the system maintains the set of most common decisions to be made, called the *decision working set*. For example, in a

rescue mission, decisions may need to be made on the best evacuation route for each survivor. The set of all such mission-relevant decisions constitute the working set.

- *The decision model:* In deciding on a course of action, the commander or decision-maker must consider several relevant variables that impact the decision outcome. For a trivial every-day example, in deciding what to wear in the morning, one might consider weather conditions (such as temperature and precipitation). These variables constitute the unknowns that need to be determined for a decision to be made. Hence, for each decision in the working set mentioned above, the system must know the relevant unknowns, as well as how these unknowns imact decision outcome. We call it the *decision model*. The choice of decision model itself gives rise to interesting research questions that warrant further attention. In the simplest model, decisions are viewed as choices of a course of action among multiple alternatives [25, 22]. More on the decision model will be mentioned later in the chapter. The viability of each individual alternative depends on the satisfaction of several predicates. Making a choice can therefore be thought of as an evaluation of a logical expression of multiple predicates. For example, "if it is (i) sunny and (ii) warm, I will wear a t-shirt; else, I will wear a sweater".

- *The unknowns:* Consider a predicate such as "if it is (i) sunny and (ii) warm". Evaluating such a predicate requires determination of the value of one or more *unknowns*. In the example quoted above, the unknowns are parameters of local weather that determine whether or not it is sunny and whether or not it is warm. The decision model specifies the unknowns whose value needs to be determined in order to decide on the viability of each course of action.

- *The evidence data objects (or simply, data objects):* Determination of the unknowns entails acquisition of corresponding evidence. Data objects such as images, videos, or sound clips, generated by appropriate sensors, can supply the needed evidence. For example, a picture taken several minutes ago at the location of interest, showing that it is sunny, would constitute evidence that "sunny" can be evaluated to "true".

- *The sources:* Often, a piece of evidence (e.g., a picture that shows whether it is sunny or rainy) can be supplied by any of several alternative sources, such as multiple cameras overlooking the scene. The system must choose a source such that timely and relevant information is provided at low cost. This gives rise to appropriate source selection and data delivery scheduling protocols.

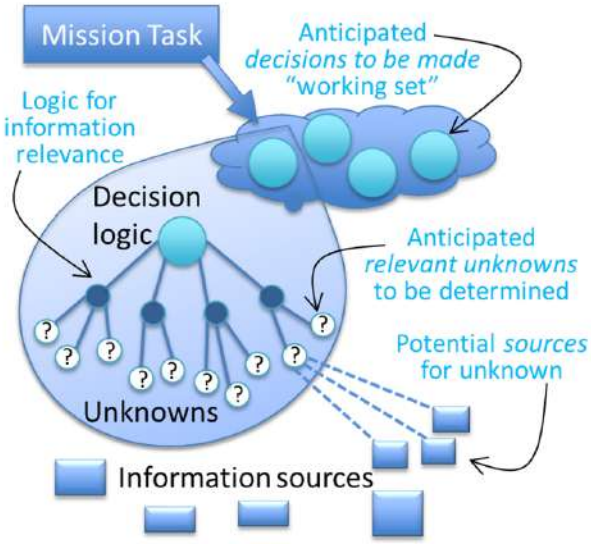


Figure 2.1: A Conceptual Architecture for the Decision-driven Execution

Figure 2.1 is a conceptual view of the architecture, where missions are broken into decision working sets, decisions are associated with logical expressions in appropriate unknowns, and the unknowns are linked to sources that can supply the relevant data objects (or evidence).

A decision-driven resource management system allows applications to make queries we call *decision queries*, or decision tasks, that request information needed for a decision. The system comprises nodes that contribute, request, or help forward data needed for these decisions. It manages the acquisition of evidence needed to evaluate the viability of different courses of action involved in decision-making. By accounting for models of decisions and sources, the system carries out the required information collection and transmission in a more efficient and timely manner to support decision making.

Athena is a recently developed system that implements the above architecture. An illustration of Athena system components is shown in Figure 2.2. For completeness, we list all components of the overall system below. Later in the chapter, we shall focus only on data communication and scheduling policies and the components that impact them. Figure 2.2 presents the overall architecture.

There are four types of users, as follows:

- *Commander:* Athena is a decision-support system. Hence, the primary target consumer is a commander. This user issues a decision query to Athena. Once the system has resolved the decision, the commander receives the solved answer for the given question.

- *Staff officer:* A decision rule should be predefined to resolve the query given by the

commander. We expect this job can be done by a staff officer who specializes in a particular domain such as operations and logistics. This user can also give a probability for the predicates in the decision rule.

- *Solider:* This user mainly produces data by observing the physical state of the battlefield. It can be a human soldier or an autonomous sensor such as a drone. Also, it needs to be able to retrieve data to understand the current situation.

- *Analyst:* An analyst extracts semantics from the raw data generated by sensors. For example, the human operator in the tactical operations center can report that the bridge is disconnected based on the image being sent by the soldiers. Or it can be an AI agent such as a computer vision algorithm that determines the status of, for example, a bridge. The sensors can be the analysts as well if the required information is known ahead of time. In the same example, the solider can say that the bridge is damaged instead of sending images of the bridge.

To support the above users, Athena provides three types of APIs. The details are as follows:

- *Decision maker:* These APIs are for commanders and staff officers who deal with decisions. When a staff officer sends a decision rule, Athena translates it into a decision tree and stores it to the decision tree repository. Then, the query resolution engine retrieves the decision tree from the pool once the matching decision query has been requested. It utilizes the source selector and prediction components to schedule the best data to resolve the given decision. After scheduling, the selected data are requested by the pub/sub protocol. This process is iterated until the decision has been made. Then, the decision is sent back to the application so that the commander can check the answer.

- *Sensor:* Sensors such as human soldiers need APIs to produce the collected information. Sensor APIs provide this interface. It is on-demand, which means the data are first stored in the local storage and sent by the pub/sub protocol when requested.

- *Annotator:* Annotators create labels based on the retrieved data. Therefore, annotator APIs need to understand the definitions of labels and request the relevant data. Once the labels have been resolved, then the values will be stored in the local semantic storage. If other nodes request the resolved labels, the values will be sent by the pub/sub protocol to the network.

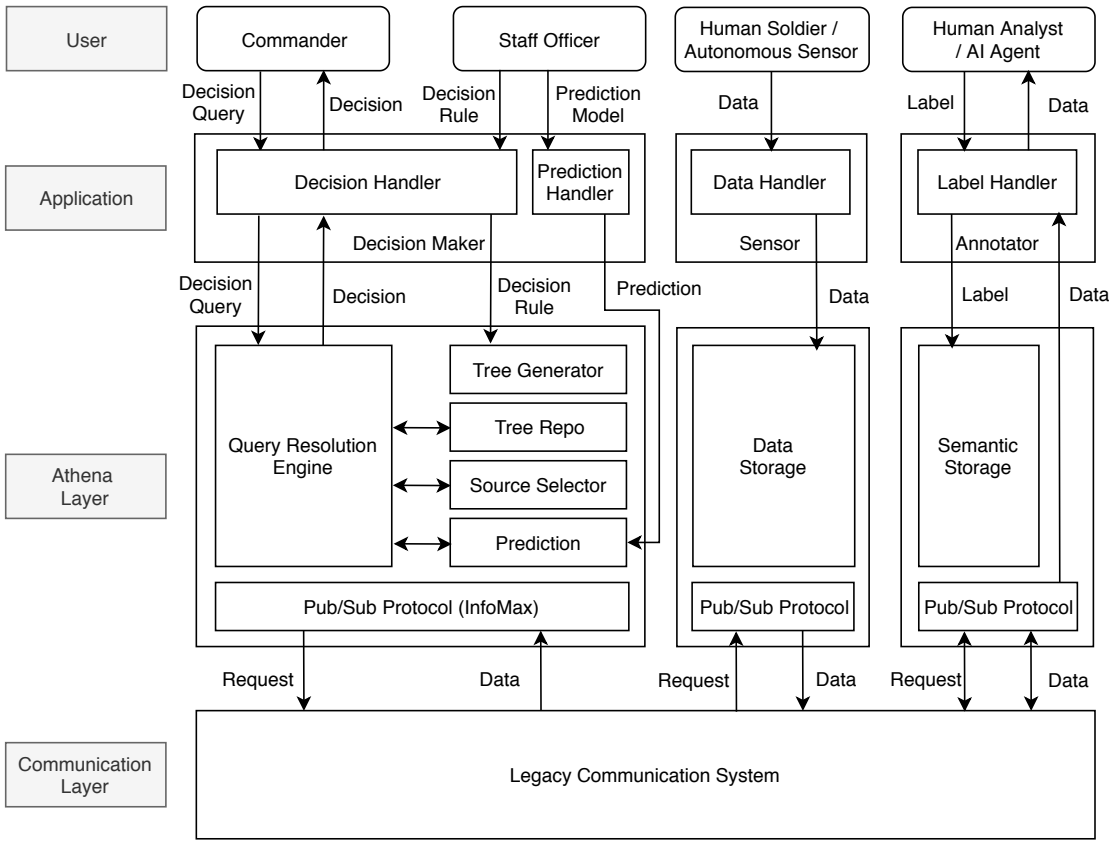


Figure 2.2: Athena's architectural design, as well as how information propagates through different system components.

2.2.1 Exploiting Decision Structure: An Illustrative Example

A key innovation of the decision-driven system lies in a novel query interface that allows applications to express decision needs in a manner that helps the resource management components properly prioritize data acquisition. Specifically, a query may specify a logical expression that describes the decision structure. This expression specifies the predicates that need to be evaluated for the corresponding choice (of a course of action) to be made. In this model, there are no limits on the types of queries that can be expressed as long as they can be represented by Boolean expressions over predicates that the underlying sensors can supply evidence to evaluate.

There are many possible ways that such expressions could be obtained. In many applications, especially those involving liability or those where human teams must operate efficiently under adverse or dangerous conditions, a well-prescribed operation workflow is usually followed. The workflow specifies how individuals should act, under which conditions a given course of action is acceptable, and what checks must be done before emparking on

an action. Training manuals, rules of engagement, doctrine, standard operating procedures, and similar documents describe these workflows, essentially documenting acceptable decision structures. Decision logic could also be learned by mining datasets that describe conditions observed and decisions taken on them by an authority. Such an approach, for example, may be used to reverse-engineer strategy used by an expert or by an adversary. Finally, in some cases, decision logic could be algorithmically derived. For example, in a vehicular navigation application, the driver will generally seek a route that satifies some machine-checkable property, such as a condition on expected commute time, quality of route, or length of commute. Hence, the logic for the decision on route from alternatives on a given map is known. An interesting research question is: given the logical decision structure (i.e., the graph of logical predicates to be evaluated to arrive at a course of action), how best to deliver the requisite information?

Let us look at a toy example to help make the picture more concrete. Consider a scenario from emergency response [26]. Suppose after an earthquake that hits our smart city, there is a shortage of air support, and an emergency medical team needs to transport a severely injured person from an origin site to a nearby medical center for surgery. There are two possible routes to take: One composed of segments A–B–C, and the other of segments D–E–F. We need to make sure that the chosen route is in good enough condition for our vehicle to pass, so we want to retrieve pictures from deployed roadside cameras in order to verify the road conditions and aid our decision-making on which route to take. Our route-finding query can be naturally represented by the logical disjunctive norm form $(viable(A) \wedge viable(B) \wedge viable(C)) \vee (viable(D) \wedge viable(E) \wedge viable(F))$, where $viable(X)$ represents the predicate "route segment X is viable". This expression signifies that at least all segments of one route need to be viable for the transport to occur. In this example, if road segments A, B and C all turn out to be in good condition, then the first route is viable, and there is no need to continue retrieving pictures for road segments D, E, and F. Similarly, if a picture of segment A shows that it is badly damaged, we can skip examining segments B and C, as this route isn't going to work anyway. Instead, we can move on to explore segments D, E, and F.

As is evident from this toy example, exploiting decision structure (represented by the Boolean expression) enables us to take inspiration from heuristics for short-circuiting the evaluation of logical expressions to schedule the acquisition of evidence. Specifically, we can acquire evidence in an order that statistically lowers expected system resource consumption needed to find a viable course of action. By incorporating additional meta-data (e.g., retrieval cost of each picture, data validity intervals, and the probability of each road segment being in good or bad condition), we can compute retrieval schedules that better optimize delivery

resources expended to reach decisions. This optimization, indeed, is the main research challenge in the decision-driven execution paradigm.

This optimization must consider both physical and cyber models. On one hand, models of the underlying physical phenomena are needed to correctly compute inputs such as data validity intervals (how long can one consider measurements of given physical variables fresh), and environmental conditions (e.g., probablities that some measurements not yet acquired will fall into a range that invalidates versus supports a predicate). One the other hand, models of computing and communication resources are needed to understand how much bandwidth and compute power are available for data collection from the physical world.

The latter models can be obtained from network and other resource monitoring. The former are more difficult to obtain. They can be learned over time or derived from the physical nature of the phenomena in question. For example, temperature does not change very quickly. Hence, the validity interval of a temperature measurement could be of the order of large fractions of an hour. On the other hand, state during an active emergency, such as a burning building, can change on the order of minutes. Hence, its validity interval is much shorter. It is also possible for external events to invalidate freshness of variables. For example, the existence of a resource, such a bridge across a river, can be assumed to hold with a very large validity interval. However, a large earthquake or a military air-raid may invalidate such past observations, making them effectively stale and in need of being re-acquired from sensors. The same applies to learned probabilities of conditions. The probability of traffic congestion on some freeway at 11pm on a Monday night might be known. However, a condition, such a nearby large concert that ends around the same time, can invalidate it. In general, a combination of past contextual knowledge, current observations, and invalidations will be needed to operationalize the physical models.

Lowering the data acquisition costs of decisions involves carrying out an optimal collection strategy given the resources available and the underlying physical models, such that a measure of decision correctness is maximized, while cost is minimized. If some contextual information needed for the models is not known, the optimization may proceed without it, but the quality of solutions will be lower, generally entailing a less than optimal resource cost. The sensitivity of decision cost to the quality of models supplied is itself an interesting research problem.

2.2.2 System Abstractions and Components

The decision-driven execution system represents the physical world by a set of labels (names of Boolean variables). These labels can be used in expressions of decision logic

structures. The system maintains tuples of (*label*, *type*, *value*), where *label* is just an identifier (i.e., variable name), the *type* specifies the semantic type of the label (for example, "road condition"), and *value* could be *true*, *false*, or *unknown*. The system can be easily extended to more general types (other than Boolean). More general discrete variables can be implicitly represented by sets of labels, one label for each allowed value of the variable, with the restriction that only one of these can be true at a time. Continuous variables can be supported as long as actions are predicated on some thresholds defined on these variables. For example, the decision to turn the lights on in a smart room can be predicated on the value of an optical sensor measurement dropping below a threshold. This is a Boolean condition whose evaluation result can be stored in a variable labeled, say, *Dim*. The pool of labels itself can be dynamic. New applications can add new labels (and new categories of labels) to the pool and specify sensing modalities needed to determine label values. For instance, in the routing example above, the predicate $viable(X)$ can be represented by the label $viableX$, denoting a Boolean variable of value *true* (if the route segment is viable) or *false* (if it is not). The route selection decision is associated with labels $viableA$, $viableB$, ..., $viableF$.

To determine the value of a label (e.g., whether conditions of a road segment make it a viable candidate), evidence must be collected. An example of such evidence might be a picture of the corresponding road segment. We call such evidence items *evidence objects* or simply *data objects*, where it is clear from context that the data in question offers evidence needed to evaluate a logical predicate in the decision structure.

Evidence objects are data objects needed for deciding the value of labels. Entities that examine evidence in order to determine the value of a label are called, in our architecture, *annotators*. For example, an annotator could be a human analyst receiving a picture of route segment A, and setting the corresponding label, $viableA$, to *true* or *false*, accordingly. Alternatively, an annotator could be a machine vision algorithm performing the same function. In general, annotators should advertise the type of evidence objects they accept as input, and the types of labels they can accordingly compute. Clearly, the same object can be used to evaluate several different labels. For example, a picture of an intersection can be used to evaluate physical road conditions. However, it can also be used to detect specific objects such as individual vehicles, license plates, or pedestrians, or used to estimate values such as length of traffic backup, traffic speed, or congestion level.

Another key component of the decision-driven resource management paradigm is the data sources. Sources that originate data, such as sensors, must advertise the type of data they generate and the label names that their data objects help resolve. For example, a source might offer pictorial evidence of road conditions. Such a source would advertise both its data

type (say, JPEG pictures) and the specific geographic locale covered. In the route discovery example, this source would need to be paired with an annotator that can accept pictures as input and determine viability of road segments within that geographic locale.

Finally, an important component is network storage or caches. The decision on mapping data and computation to network nodes in a distributed execution environment is a classical problem in distributed computing systems. This problem must be solved in the context of a decision driven execution as well. Content (both data objects and annotation labels) should be cached at nodes closer to consumers who might need these objects and labels for their decision-making. Similarly, annotators will need to execute on nodes that are close to consumers needing the annotations. The placement of data and computational modules in the network to minimize decision cost remains an open problem.

The aforementioned architecture effectively changes the query paradigm from specifying *what* objects to retrieve to specifying *why* they are needed; that is to say, how they fit in the logic used to make a decision. This shift is thanks to sharing the structure describing the query's decision logic. Evidence objects are needed to resolve predicates named by labels in that decision logic. The architecture allows the network to be much smarter when answering a query. Being aware of the logical decision structure, the resource management system can allocate resources to seek evidence that helps evaluate the decision expression at the lowest cost. As alluded to in the introduction, we can take inspiration from literature on optimizing the evaluation of logical expressions to determine which labels should be evaluated first and which sources should be contacted for the corresponding evidence. In turn, this determination informs resource allocation, such as policies for scheduling/queuing of object retrieval requests, policies for caching of results, and choices governing invocation of annotators.

2.2.3 A Walk Through the Execution of a Decision Query

Putting it all together, when a user makes a decision query, at a high level, query resolution works as follows. The system first determines the set of predicates (i.e., labels) that is associated with the query from the underlying Boolean expression that describes the decision logic. This is the set of labels whose values need to be resolved. The query source then needs to determine the set of sources with relevant evidence objects. If multiple sources offer redundant evidence, some arbitration is needed to determine who to contact. A scheduling algorithm must decide on the order in which evidence objects must be retrieved to evaluate the different labels.

The system must manage caching. Say, the query source decides to resolve the value of

the label, *viableX*. If the label has already been evaluated in the recent past (because of a prior query), its evaluation may be cached in the network, in which case the resolved value can be found and returned. This is the cheapest scenario. Otherwise, if the evidence object needed to evaluate the predicate has been recently requested (but the corresponding label not evaluated), the requested object may be cached. Such might be the case, for example, when the object was requested to evaluate a different predicate. The cached object needs to be sent to the right annotator to determine the label value relevant to the current query. Otherwise, if the objects is not cached or is stale, the query should be propagated to a source that has fresh relevant objects. The relevant object is then shipped to an annotator that decides label values. Both the object and the computed new labels are cached in the network with a freshness interval that specifies their validity for future use. Next, we outline the research challenges that must be addressed in realizing this architecture.

CHAPTER 3: LOGICAL SHORT-CIRCUITING BASED SUMMARIZATION

In military operations, *Commander's Intent* describes the desired end state and purpose of the operation, expressed in a concise and clear manner. *Command by intent* is a paradigm that empowers subordinate units to exercise measured initiative to meet mission goals and accept prudent risk within commander's intent. It improves agility of military operations by allowing exploitation of local opportunities without an explicit directive from the commander to do so. This chapter discusses what the paradigm entails in terms of architectural decisions for data fusion systems tasked with real-time information collection to satisfy operational mission goals. In our system, information needs of decisions are expressed at a high level, and shared among relevant nodes. The selected nodes, then, jointly operate to meet mission information needs by forwarding and caching relevant data without explicit directives regarding the objects to fetch and sources to contact. A preliminary evaluation of the system is presented using a target tracking application, set in the context of a NATO-based mission scenario, called Anglova. Evaluation results show that delegating some decision authority to the data fusion system (in terms of objects to fetch and sources to contact) allows it to save more network resources, while also increasing mission success rate. The system is therefore particularly well-suited to operation in partially denied or contested environments, where resource bottlenecks caused by adversarial activity impair one's ability to collect real-time information for mission-critical decision making.

3.1 INTRODUCTION

Military operations of the future will utilize fewer human soldiers and more interconnected "things". This applies to sensors and data fusion systems. Future data fusion systems will need to *independently* select the right sensors to meet mission decision needs without humans in the loop to direct the sensor selection. Since military networks may be used in contested or denied environments, where bandwidth is sparse, judicious sensor selection becomes important in order to minimize network resource consumption while delivering the requisite information.

In this work, we develop a framework to incorporate commander's intent[1] into data fusion systems. In this framework, rather than specifying to the retrieval system which sources to contact for which pieces of information, a decision-maker specifies the *goal* (i.e., intent)

[1]Commander's Intent is defined as the desired end state expressed in a concise and clear manner so that every subordinate unit can understand it [27].

from information collection; specifically, a decision to be made. The underlying system then determines the most relevant pieces of information that may impact the decision, as well as the best sources to contact for these pieces. The framework further optimizes information retrieval in order to minimize the cost of information delivery.

The choice of sensors with information that is relevant to a decision may depend on the state of the physical environment. In this chapter, we use the term *environment* in the broader sense to denote everything outside the data fusion system, such as terrain, targets, and weather. For example, deciding where to intersect a fleeing target may depend on observations of terrain, current target location, and current target trajectory. The system must monitor such relevant variables and anticipate their future values in order to perform sensor selection that best informs the decision.

This work is inspired by a novel resource management paradigm, called *decision-driven* execution [28], where all resource consumption is driven by decision needs. A recent implementation of this paradigm is called Athena [29]. Athena expresses a decision as a logical formula that states the available courses of action and specifies the conditions that need to be true for each given course of action to be viable. Since only one of the alternative courses of action will ultimately be chosen by the decision maker, one can imagine these courses of actions as being logically OR'ed. Furthermore, since the viability of a course of action is contingent upon all its prerequisite conditions being true, these conditions are logically AND'ed. The act of making a decision in this context reduces to finding a path in the decision tree that evaluates to true. Evaluating individual prerequisite conditions entails acts of sensing. Hence, there is a cost associated with making a successful decision. As noted in earlier work [28], this decision cost can be minimized using techniques borrowed from optimizing the evaluation of Boolean expressions.

This chapter focuses on one aspect of the decision-centric execution paradigm; namely, the interplay between decision cost and the accuracy of prediction of the state of the environment. The key insight in optimizing the evaluation of Boolean expressions lies in a notion of *short-circuiting*. For example, if a term in a big logical OR expression evaluates to true, there is no need to evaluate the other OR'ed terms. In this case, evaluating a term that is more likely to be true first will reduce the expected cost of overall expression evaluation. This observation ties the cost of decisions (given by a logical expression) to a notion of prediction. More accurate prediction of the state of the environment allows for a better evaluation order of terms in the decision structure, leading to a lower network resource consumption due to better short-circuiting. In contrast, inaccurate prediction entails a higher cost.

Another advantage of expressing decisions as logical expressions lies in its support for prefetching. The framework can fetch objects relevant to the next course of action before

finishing the retrieval of objects relevant to the current one. Hence, if the current course of action is not deemed viable, one can immediately evaluate the next. Prefetching has the potential to reduce delay in expression evaluation at the expense of generally consuming more bandwidth.

We evaluate the framework using a target tracking scenario developed in the context of a NATO activity. The scenario emulates a mission set in a fictitious territory, called Anglova, where a battalion advances to neutralize a threat. The scenario describes node mobility and radio communication bandwidth among nodes. We augment it with an emulation of targets that flee the scene towards one of several hideout points. Our data fusion system orchestrates the acquisition of data from different sensors in order to best inform a decision-maker where each target is going. The system optimizes retrieval cost by short-circuiting decision trees in ways consistent with predicted state of targets. Evaluation results show that our resulting data retrieval scheduler significantly outperforms other baseline algorithms.

The contributions of this work are three-fold. First, we describe a data acquisition system that utilizes predicted state of the environment to control sensor activation in a manner that minimizes decision cost. Second, an implementation on Linux is described. Lastly, the performance of a target tracking application is evaluated in detail.

3.2 DESIGN

The goal of our data acquisition system is to minimize the cost of retrieving information needed for decision purposes. In the following, we describe the query model, then detail the optimization needed for minimizing decision cost.

3.2.1 Query model

A decision maker who wishes to use the system, or a software application acting on their behalf, issues a *query* in the form of a decision tree. The tree has OR'ed nodes corresponding to the alternative courses of action being decided on. Each course of action is viable if a set of prerequisite conditions is true. Hence, in general, a decision tree can be expressed as a logical formula, standardized to a disjunctive normal form (DNF), OR of ANDs. An example is shown in Figure 3.1. The leaf nodes in the tree are the unknown individual conditions to be determined. In this chapter, we consider Boolean conditions. An example condition (e.g., in finding a viable route after a hurricane) might be: "Bridge is not closed" or "Road is not flooded". A condition is evaluated (to true or false) by retrieving objects, such as pictures or videos, that offer evidence supporting one value or the other. If the decision tree

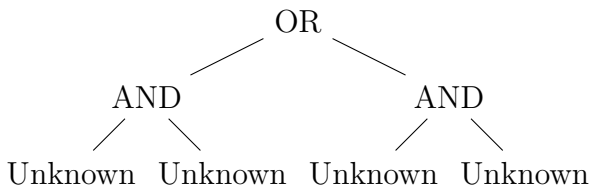


Figure 3.1: Decision tree expressed as a disjunctive normal form (OR of ANDs). A path that evaluates to true denotes a viable course of action.

evaluates to true, a viable course of action exists; namely, the path that evaluated to true in the decision tree.

To evaluate the logical expression efficiently one needs to know the probability of each logical value (true or false) of the unknown at each leaf. This probability informs the order in which the unknowns are to be evaluated. By *evaluating an unknown*, we mean investing in collecting the evidence that establishes the unknown value. Note that, to compute the value of the overall expression, it is not necessary to evaluate all unknowns. For example, in a logical expression given by an OR of multiple terms, once one term evaluates to true, there is no need to evaluate the rest. This is known as *short-circuiting*. It leads to the formulation of a data retrieval scheduling problem to minimize the cost of evaluating the overall expression, or equivalently minimize the data retrieval cost of making a decision.

3.2.2 Data retrieval scheduler

An efficient data retrieval schedule will reduce the amount of data retrieved to make a decision by efficiently exploiting short-circuiting. The job of the data retrieval scheduler is thus to find the short-cuts that resolve the decision with minimal cost. To balance retrieval cost against the short-cut probability, we use a heuristic. Namely, when choosing among OR'ed terms (at intermediate nodes), we compute the probability of each term evaluating to true, normalized by cost, as follows:

$$C_i = \frac{\prod p_{ij}}{\sum c_{ij}}$$

where c_{ij} and p_{ij} are retrieval cost and probability of evaluating to true for the jth condition in the ith term. As shown, it is the product of probability divided by expected cost of the given sub-tree. Expected cost is the sum of retrieval cost for all data needed for evaluating the term in question. Note that, cost of retrieving a data object can be approximated as data size multiplied by retrieval distance (e.g., hop count in routing). That is, heavy sized data from distant nodes is considered to be more expensive. The course of action with highest

probability of success per unit cost is the one that has the highest C_i. The algorithm focuses on retrieving evidence for this course of action first.

Once a term is selected, it remains to evaluate all unknowns in that term. We first evaluate the unknown that has the maximum probability of being false, normalized by cost:

$$\frac{1 - pij}{c_{ij}}$$

This ensures that if the selected course of action is not viable, the failure is determined as early as possible (in expectation).

3.2.3 Name-based data retrieval

Once the scheduler computes the order in which unknowns are evaluated, it begins to retrieve information on each unknown. We use name-based data retrieval, inspired by information-centric networks (ICNs). An advantage of ICNs is in-network caching. Consumers ask for content by name. Any network node that caches a matching named object can therefore return it, leading to more efficient retrieval.

To use the ICN paradigm, a name space must be designed *a priori* as a hierarchical content directory structure in which each type of mission-relevant sensory information has its own "folder" name. Since information is often spatio-temporal, hierarchical naming makes sense, as it allows one to index the folders by space and time. For example, an image of a street in a city might be called `/imagery/city/street/time/imageID`.

Each unknown in the decision tree discussed in the previous section can thus be bound to a folder name; For example, the condition "Main Street not flooded" (today in the city of Anglova) would be bound to the name that denotes imagery of the given street at the given time, such as `/imagery/anglova/main/<current_date>`, since the latter offers evidence that allows to evaluate the unknown at hand. The binding essentially maps from mission-relevant unknowns to available information types. This binding itself is an interesting research problem that is outside the scope of the current work. In the following, the reader may assume that the binding occurs at configuration time, once the types of sensors involved in the system and their locations are determined.

Sensors advertise the part of the name space that they are responsible for sensing. For example, a camera installed on Main Street in the city of Anglova might advertise content of the name prefix `/imagery/anglova/main/`. These advertisements are propagated through the network, setting up forwarding tables that inform other nodes where to forward interests in specific named sub-spaces.

As mentioned above, data consumers express interests in specific content by specifying the corresponding name prefixes (i.e., folder names). Nodes that receive an interest can check whether or not the requested data is already in their cache by name matching. If the name expressed in an interest is a prefix of the name of a locally cached data object, the forwarder regards that the interest matches the data and returns the matching data object. For example, if the name expressed in an interest is `/imagery/anglova/main/2018/May/1st`, then `/imagery/anglova/main/2018/May/1st/image34` is a match, and can be returned. If the forwarder does not have a local match for a received interest, it forwards the interest in the direction of nodes who previously advertized content prefixes that cover the interest. For example, a source that advertizes `/imagery/anglova/main/` would cover an interest in `/imagery/anglova/main/2018/March/21`. Hence, interests in information covered by a sensor can be correctly forwarded to that sensor.

3.2.4 Controllable data prefetching

In preceding sections, we described the scheduling of data retrieval as if retrieval was sequential, resolving one unknown at a time. In reality, one can prefetch evidence ahead of time to resolve multiple unknowns at once. In general, prefetching may result in needless use of bandwidth. For example, resolving multiple unknowns in an OR expression might be wasteful if the first unknown evaluates to true. Nevertheless, bandwidth permitting, prefetching will save time. Should the first unknown in that OR expression evaluate to false, one has already retrieved evidence to evaluate subsequent unknowns.

We allow the application to request prefetching in the query. This request is treated as a hint, leaving the final decision to each node. If a node judges that the network is not congested, it would accept the hint. Otherwise, it would reject it and not propagate the prefetching request to its neighbors. In the evaluation, we also control the prefetching rate that decides the amount of data to prefetch. This allows us to investigate the effect of prefetching on bandwidth and delay in more detail.

3.3 IMPLEMENTATION

The aforementioned data acquisition framework was implemented on top of an information-centric network (ICN) layer. In ICNs, named data entities are top-level abstractions [30, 31], instead of hosts. Hence, rather than routing requests to an IP address, ICN allow routing requests (called *interests*) directly to the requested information, thereby facilitating the implementation of data acquisition systems. As described in Section 3.2, multiple sensors and

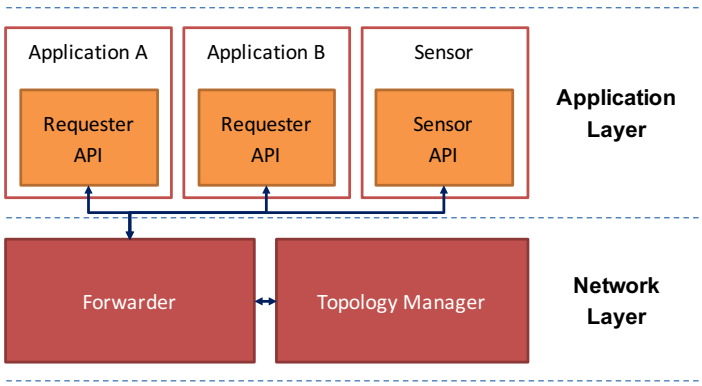


Figure 3.2: Layered architecture of *Athena*. *Athena* applications can be developed through provided APIs that can interact with *Athena* forwarder. The forwarder mainly handles all requests from the application layer and the topology manager provides routing and registered prefix information to the forwarder.

Table 3.1: Sensor APIs

`Sensor (prefix_name, on_interest):`	Constructor for sensor
`on_interest (interest):`	A callback function to get data

applications can run on one Athena node. In order to support such modularity, the overall architecture is layered as shown in Figure 3.2. Each layer is described in detail below.

3.3.1 Application layer

In order to build Athena applications, developers need to implement a requester and a sensor. We provide simple APIs to help developers to build such components as shown in Table 3.1 and 3.2. The *init_query*() is a function to send a query to the local Athena node. This query is processed as described in Section 3.2 and triggers data retrieval. The data request is delivered to sensors and generated data is sent back to the requester. Based on the received data, the requester needs to evaluate the unknowns at the leaf of the decision tree. This is done inside the function *on_label*(). Then, Athena decides the result of the given query using determined values. If the query is resolved, the result is handed to *on_answer*() so that the application can handle the result. If the query is unresolved (i.e., it evaluates to false), Athena makes a next request.

Developers need to define a sensor request handler in *on_interest*(). The sensor registers its content prefix with the local Athena node. This information is advertised to the network by the topology manager. The network forwards matching requests to the sensor, who parses the requested name and sends corresponding data back to the requester.

Table 3.2: Requester APIs

`Requester (on_answer, on_label):` Constructor for the query requester
`init_query (name, tree, deadline):` A function to initiate the query
`on_answer (query name, answer):` A callback function to handle the result of the given query
`on_label (data, labels):` A callback function to set values for the given labels based on the retrieved data

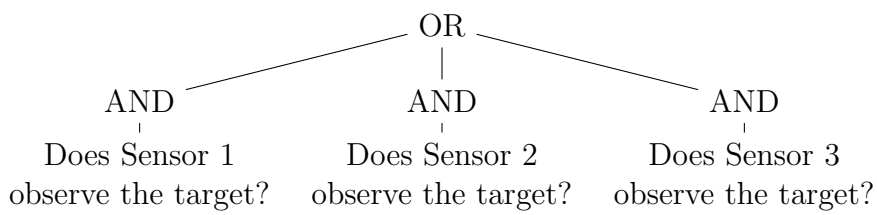


Figure 3.3: Decision Tree for location A. According to the given tree, the target tracking application decides the target location is A if any sensors covering the location (*i.e.*, sensor 1, 2, and 3) actually detects the target.

Figure 3.3 shows a simple decision tree to tell if a target is at location A, covered by three sensors, 1, 2, and 3. If any of the sensors covering location A detects the target, the application decides that the target at location A. The application might first contact the sensor with the highest ratio of target detection probability, p_{ij}, to cost, c_{ij}, of delivering the observation.

3.3.2 Network layer

There are two main components in the network layer: the Athena node and the topology manager. The Athena node is a daemon program that handles all incoming and outgoing messages. Routing information is maintained by the topology manager. It advertises the local prefixes to neighbors, who propagate the information to their neighbors until all information is globally synchronized.

When data requests are received by an Athena node, it forwards them to the next hop node towards the data source. Each node that receives the request checks its cache. If a data name stored in its cache matches the given interest name requested, it returns the cached data. Otherwise, it checks if requests for the same name have been previously received. If so,

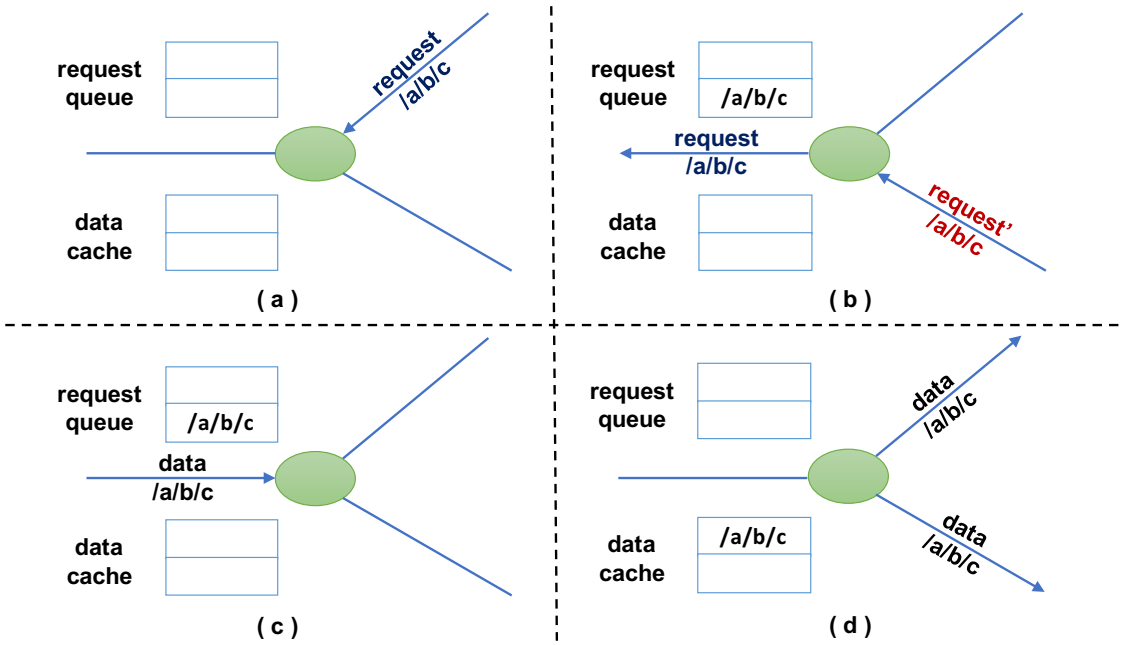


Figure 3.4: Name-based data retrieval. In *Athena*, all requests and data have names. When a request is forwarded, intermediate node stores the request in its queue (a). Although another request with the same name is made, the request is suppressed as the same request has been made before (b). If data is produced and forwarded to this node, the node stores the data to its cache (c). Then, it forwards the data to each destination from which a request came (d).

the requests are consolidated and the node remembers where to forward the replies. Once a request reaches the producer (i.e., sensor) node, it asks the local sensor to produces the data. The produced data would then backtrack along the route(s) traversed by the original requests. This process is visually described in Figure 3.4.

3.4 DOMAIN OF APPLICABILITY

Athena is applicable for a general decision-making problem if the decision can be expressed in a logical formula. Therefore, the application on both the higher and lower echelons of the tactical networks can benefit from Athena. However, strategic decisions such as launching a nuclear missile cannot be automated because their conditions are often related to political and economic conditions that are hard to express in the simple if-then-else form. In this sense, Athena is more suitable to support tactical-level decisions; hence the Tier-1 tactical network would be assumed to be the underlying network systems. That is, remote sensors are connected over a wireless medium, and the decision maker needs to collect data from them to resolve the given decision query. Especially if the given decision involves many data

objects and AI agents that can automate the analysis process, Athena can make the right decision more efficiently than a human.

3.5 EVALUATION

In this section, we describe our experimental setup and results. A target tracking application runs within the Anglova scenario, developed by NATO. Queries decide current target location. Query resolution delay and bandwidth consumption are measured. To penalize for unresolved queries, the delay is also normalized by success rate. That is $d' = \frac{d}{r}$, where d' is normalized delay, d is original delay, and r is query success rate.

3.5.1 The Anglova Scenario

The NATO IST-124 Research Task Group's Anglova scenario provides node mobility and network connectivity data for a realistic battalion-sized military operation consisting of three vignettes designed to highlight challenges related to tactical networks [32]. The scenario depicts an operation conducted by a battalion after receiving information alerting to an attack by insurgent forces against coalition forces in an operation zone.

The US Army Research Laboratory's Dynamically Allocated Virtual Clustering Management System (DAVC) was used to emulate the Anglova scenario on virtual machines. Two different DAVC experimental networks are used to emulate resource-rich and resource-scarce environments by setting a 1Gbps and 10Mbps data rate, respectively. These two extreme network environments provide information on how the proposed architecture would perform under different bandwidth conditions.

For the purposes of this work, a subset of the Anglova scenario is used, where 10 Anglova nodes are deployed and 24 platoon units approach the threat source. All platoon units are connected with a gateway node. Each Anglova node is directly linked to multiple sensors. These sensors jointly cover the city as shown in Figure 3.5. We assume line shaped coverage is by PTZ cameras and rectangular shaped coverage by UAVs. A recent PTZ camera can identify the target if distance is less than 1km during day time. Therefore, if the target is located within that range, we assume that the target is detected. Also, if ground sampling distance (GSD) of the UAV is 6-7cm/pixel (approximately 200m altitude and 170x128 m^2 coverage), image resolution is considered to be reasonable to identify the object. One may ask why not simply have the UAVs track the certain target. This is because they are not only deployed for this particular operation. Instead, they are assigned a certain area of coverage. Sensor specifications are summarized in Table 3.3.

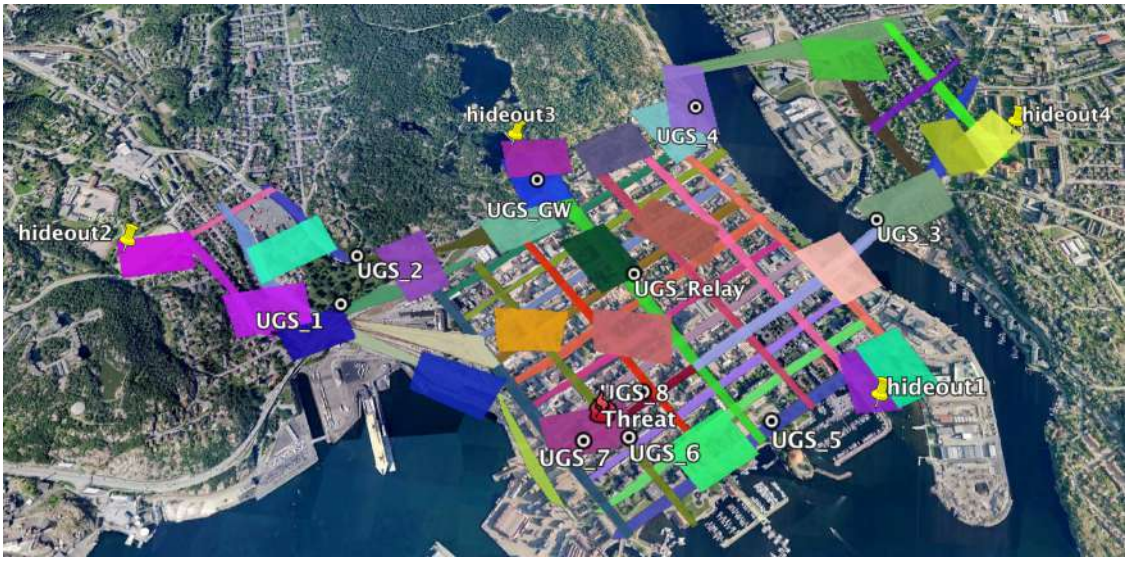


Figure 3.5: Sensor coverage map of the Anglova scenario. The UAV coverage is a rectangular shape and PTZ camera coverage is a line shape. Total 59 sensors jointly cover the city to track the given targets.

Type	Resolution	Compressed file size	Coverage	Number
UAV	2560x1920	2.86MB	170x128 m^2	26
PTZ CAM	1600x1200	1.12MB	1 km	33

Table 3.3: Sensor specifications

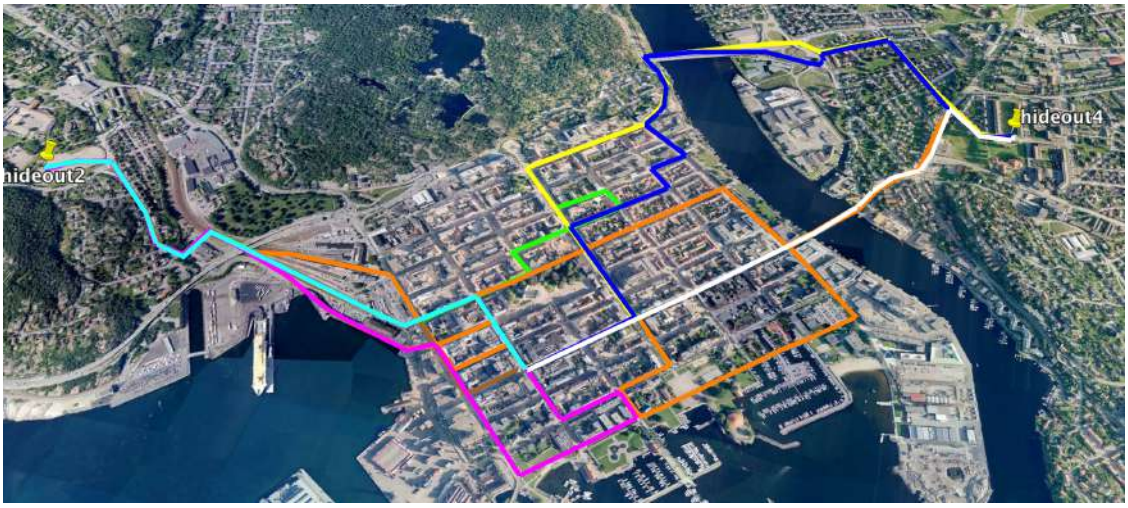


Figure 3.6: 8 targets for validation. Each colored line denotes each target route.

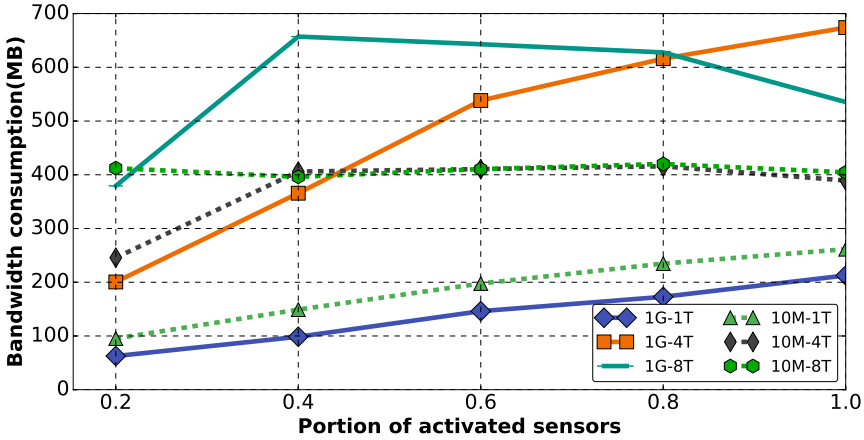
3.5.2 Simulated targets

Targets are simulated based on three parameters: vehicle types, hideout points, and deceptiveness. There are 2 types of vehicles (car and auto-bike), each of which has different speed and route accessibility. For example, an auto-bike can take a byway which is not accessible by a car. Two hideout points are expected, located at the west and east side of the map. Targets would typically flee towards one of the hideout points. Lastly, a target is either naive or deceptive, depending on whether it takes the shortest path to its destination or not. A deceptive target may pretend to head somewhere else at the beginning, then switch path and eventually arrive at its intended destination. A total of 3200 targets are created by configuring combinations of above parameters and are used to train the tracking algorithm. In addition, 8 targets, not included in the training data set, are created for evaluation, as described in Figure 3.6.

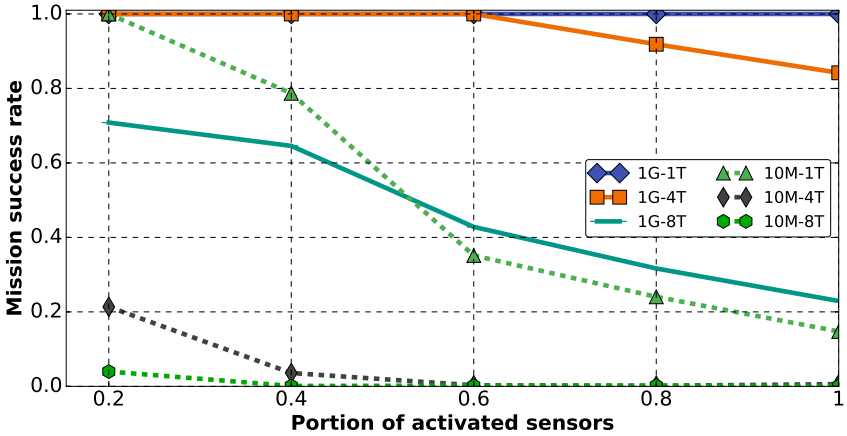
3.5.3 Results

A query about target position results in communication with a subset of sensors. We control the portion of activated sensors and the order in which they are interrogated. Bandwidth consumption and data retrieval success rate are measured as shown in Figure 3.7. Note that, bandwidth consumption denotes received and transmitted data size in MB of one sensor node during the scenario runs (5 minutes).

When there are enough resources, periodically collecting data from all sensors will ensure that targets are tracked at all times. However, this would overload our network. Therefore, data retrieval scheduling becomes important since it is not affordable to retrieve data from

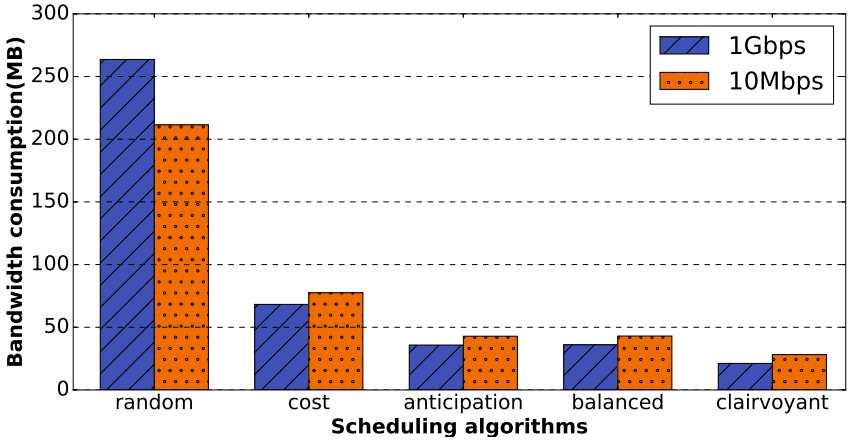


(a) Average bandwidth consumption per sensor node as a function of portion of activated sensors

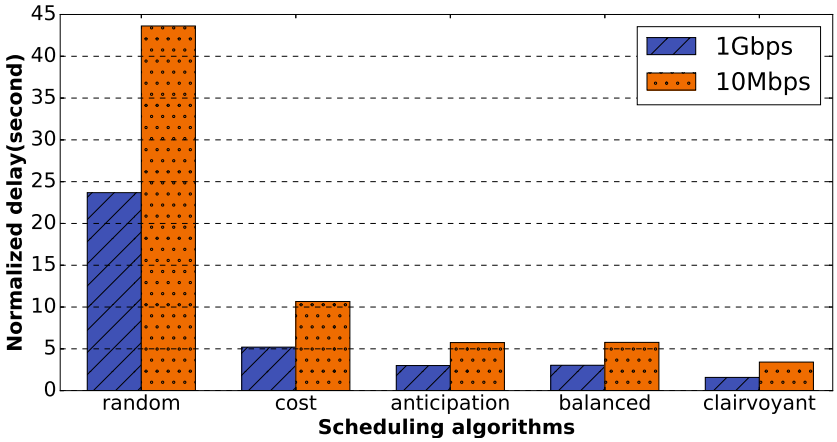


(b) Data retrieval success rate as a function of portion of activated sensors

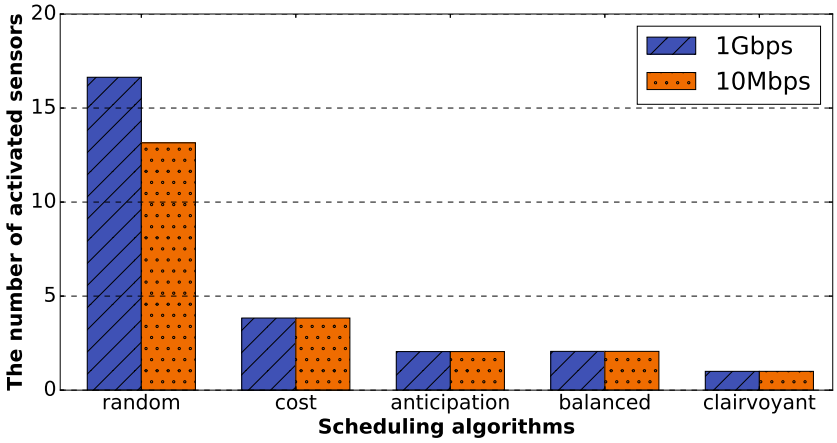
Figure 3.7: Bandwidth consumption (a) and data retrieval success rate (b) for different portion of activated sensors. Each line denotes bandwidth of the given network and the number of targets to track. For example, 1G-4T means 1Gbps bandwidth and 4 targets.



(a) Average bandwidth consumption of the requester node for different scheduling algorithms

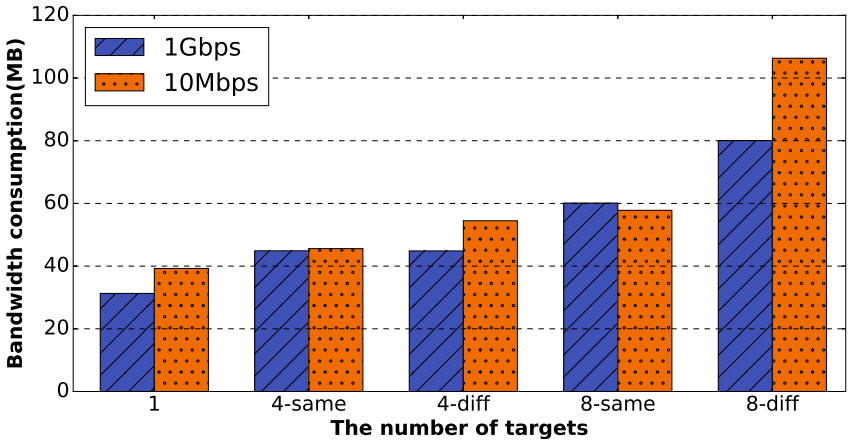


(b) Average query resolution delay for different scheduling algorithms

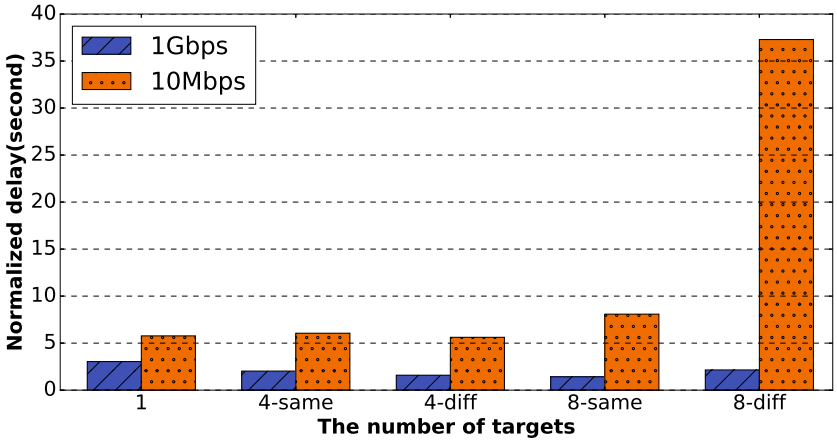


(c) The average number of activated sensors for different scheduling algorithms

Figure 3.8: Bandwidth consumption, query resolution delay, and the number of activated sensors for different scheduling algorithms



(a) Average bandwidth consumption per sensor node for different number of targets



(b) Average query resolution delay for different number of targets

Figure 3.9: Bandwidth consumption and query resolution delay with different number of requesters. n-same denotes that n nodes track the same target whereas n-diff denotes that each of n nodes tracks a different target (*i.e.*, n different targets).

all sensors. Second, data caching is necessary as different applications may request the same data.

We implement two baseline algorithms, called *random* and *cost*; *random* retrieves data from randomly selected sensors and *cost* is retrieves the least expensive data first. Prediction-based algorithms are *anticipation* and *balanced*; *anticipation* is only based on short-circuit probability whereas *balanced* considers short-circuit probability per unit cost as described in Section 3.2. Lastly, *clairvoyant* retrieves data from exactly one sensor at a time; namely, the sensor that happens to be at the target's location.

In a given run, four naive targets and four deceptive targets are created and the applica-

tion independently tracks them. Measured bandwidth consumption of the requester node, normalized query resolution delay, and the average number of activated sensors are shown in Figure 3.8 for the different algorithms. The result shows that the prediction-based scheduler consumes about 10 times less bandwidth than *random* and 2 time less bandwidth than *cost*, respectively. In terms of delay, the prediction-based scheduler can resolve the query much quicker than baselines, as shown in Figure 3.8b. This result is mainly because the prediction-based scheduler only activates a minimal number of sensors as shown in Figure 3.8c. In this experiment, *anticipation* and *balanced* show about the same performance.

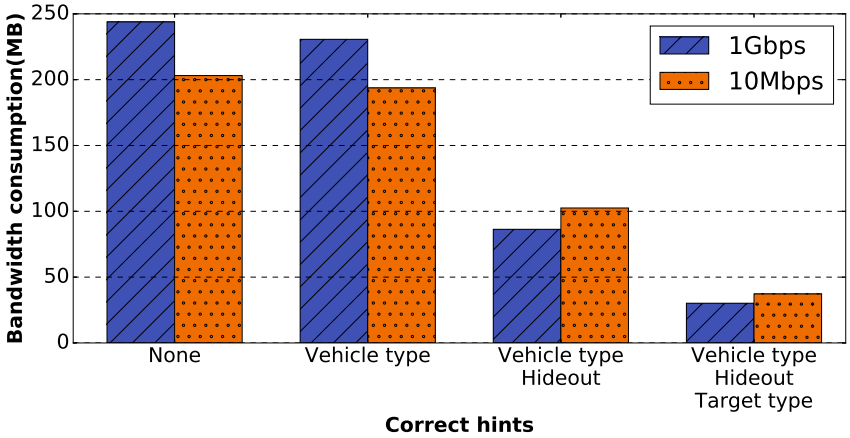
Note that, *balanced* is an adaptive solution that balances prediction accuracy and cost. It considers short-circuit probability per unit cost. Thus, if prediction is correct, its performance would be close to *anticipation*. Otherwise, its performance would be close to *cost*.

The next experiment examines effectiveness of caching in name-based data retrieval with multiple targets. Figure 3.9 shows bandwidth consumption of one sensor node and query resolution delay during the scenario as a function of the number of targets when applying the balanced algorithm. The result shows that bandwidth consumption and delay do not linearly increase with the number of targets. Concurrently tracking four targets consumes less than double the bandwidth consumption of tracking a single target. Interestingly, the average delay of tracking multiple targets is even less than that of tracking a single target because of caching. Specifically, with more targets, more queries end up retrieving data from a cache, making the average delay shorter.

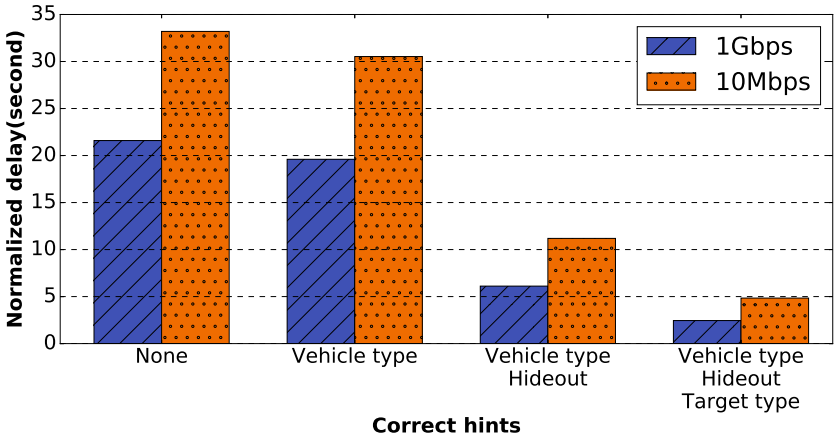
When there are eight different targets, however, bandwidth consumption and delay greatly increase. This is because of onset of overload.

The next experiment explores the relation between performance and accuracy of available side information, such as vehicle types, hideouts, and target types. Figure 3.10 shows bandwidth consumption of the requesting node and normalized delay during the scenario. To this end, we first intentionally provide three *incorrect* hints to the application, specifying an incorrect vehicle type, hideout location, and target type. This is to test impact of poor side information on performance. We then correct these hints one at a time. The x-axis in Figure 3.10 specifies which hints are corrected. For example, if the target drives a *car* to the *East* hideout point along the *shortest path*, then the initial incorrect set of hints would be that they are on an *auto-bike*, seeking the *West* hideout, and being *deceptive*. The hint about the vehicle type is then corrected to *car*. Next, the hint about the hideout is corrected to *East*. Finally, the hint about the target type is correct to *shortest path*.

As shown, if the hints are incorrect, network resource consumption is high as the algorithm would not be able to quickly find the target given the wrong side information. When the

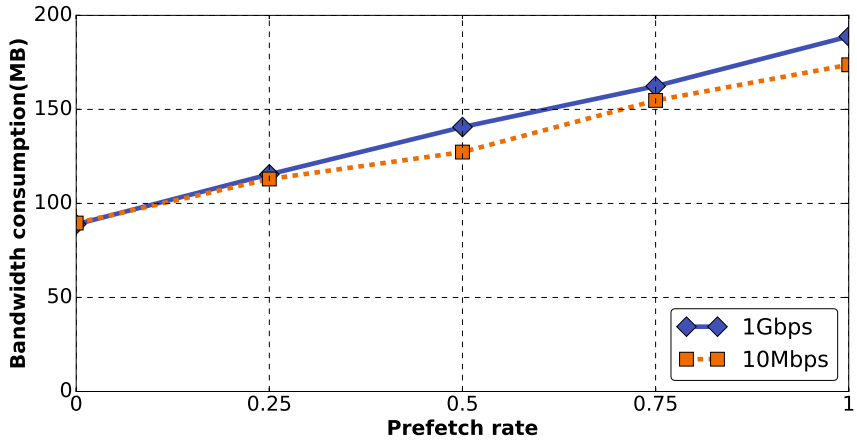


(a) Average bandwidth consumption of the requester node for different numbers of correct hints

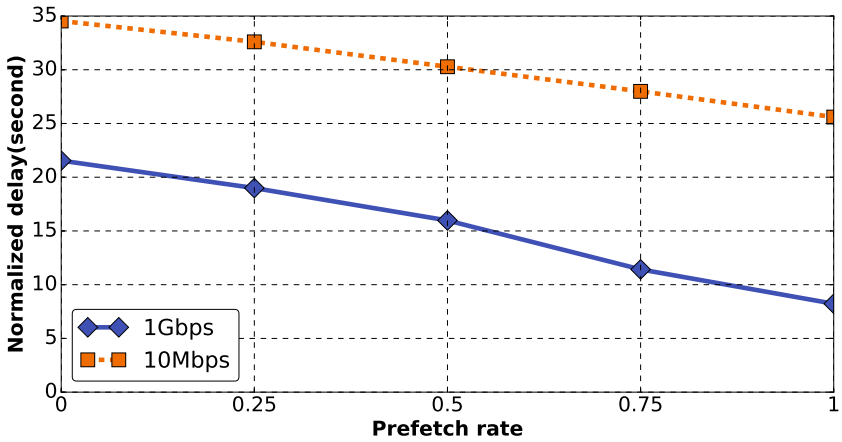


(b) Average query resolution delay for different numbers of correct hints

Figure 3.10: Bandwidth consumption and query resolution delay for different anticipation accuracy. X-axis denotes the number of correct hints. The right side has more correct hints than the left side.



(a) Average bandwidth consumption per sensor node as a function of prefetch rate



(b) Average query resolution delay as a function of prefetch rate

Figure 3.11: Bandwidth consumption and query resolution delay as a function of prefetch rate.

vehicle type information is correct, bandwidth and delay are slightly reduced. With the correct hideout point information, resource consumption is greatly decreased because one can eliminate more sensors from consideration. Lastly, if all information is correct, bandwidth consumption and query resolution delay are the lowest. This result indicates how side information (e.g., available from external sources) can be exploited to affect network resource consumption in this framework, and what the impact of its accuracy is on performance.

The relation between prefetching, delay and bandwidth is shown in Figure 3.11. The x-axis refers to the fraction of objects relevant to a query that are being prefetched. As the prefetch rate increases, bandwidth consumption increases accordingly. In return, query resolution delay generally decreases. With full prefetching, bandwidth consumption is almost

double that of no prefetching. Therefore, this option should be carefully used as it increases network bandwidth consumption, which may affect other operations.

3.6 CONCLUSIONS

This work describes a data fusion system that adopts a command-by-intent paradigm. The system allows a decision maker to specify what decision they are trying to make and automates the retrieval of information needed for that decision. In doing so, it activates a minimal number of sensors to satisfy the information needs. A key idea in optimizing sensor selection is the notion of short-cutting Boolean expression evaluation, that depends on predictions of the state of the environment. We implement this idea on the Athena framework. As an application of this framework, target tracking is demonstrated and evaluation on an Anglova mission scenario. The evaluation results show that accurate prediction can greatly decrease network bandwidth consumption and delay by activating the minimal number of sensors. The authors are currently working on developing more Athena applications in the military context.

CHAPTER 4: REDUNDANCY BASED SUMMARIZATION

The advent of social networks, mobile sensing, and the Internet of Things herald an age of data overload, where the amount of data generated and stored by various data services exceeds application consumption needs. In such an age, an increasingly important need of data clients will be one of data sub-sampling. This need calls for novel data dissemination protocols that allow clients to request from the network a representative sampling of data that matches a query. In this chapter, we present the design of a new transport-layer dissemination protocol, called InfoMax, that allows applications to request such a data sampling. InfoMax exploits the recently proposed named-data-networking (NDN) stack that makes networks aware of hierarchical data names, as opposed to IP addresses. Assuming that named objects with longer prefixes are semantically more similar, InfoMax has the property of minimizing semantic redundancy among delivered data items, hence offering the best coverage of the requested topic with the fewest bytes. The chapter discusses the design of InfoMax, its experimental evaluation, and example applications.

4.1 INTRODUCTION

This chapter develops a new transport-layer communication abstraction and underlying data dissemination protocol that allow a consumer to request a *sampling* of a producer's data at a *configurable granularity*. The protocol is implemented on top of a named-data-networking (NDN) stack. It represents an example of a new transport paradigm centered around lossy information transfer that aims to substantially reduce transmitted data volume, while meeting application summarization needs.

The work is motivated by the proliferation of sensors that are connected to the Internet, and the advent of social networks that democratize information broadcast, propelling us into a world of data overload at an accelerating rate. Indeed, over 90% of the world data was produced within the last decade [33]. As the amount of digital data grows exponentially, a gap arises between the over-abundant supply and actual application demand. The growing disparity between the amounts of data generated and what suffices to meet application needs suggests that an increasingly important function in the future will be one of data sub-sampling (as a means of summarizing large data).[1] Different data uses might have

[1] In this chapter, we use the word "sampling" or "sub-sampling" in the conventional general sense of selecting representative examples of objects from a larger set of similar objects. For instance, in a tourism application one might want to select examples of good and bad reviews of some hotel. In a building management application, one might want to select examples of afternoon temperature readings in a building.

different sub-sampling needs for the same data set. The current Internet architecture places the sub-sampling burden on the information client. The chapter shows that *information centric networks* significantly change the equation by facilitating the development of generic lossy transport, that has the effect of sub-sampling data in a manner that achieves minimum information loss, while reducing information volume in a client-controlled fashion. We expect that this protocol will be of increasing importance to a growing range of data-centric applications.

Our protocol is implemented on top of the *named data networking* (NDN) protocol stack [31]. NDN, an instance of information-centric networking, is a network layer that assigns hierarchical names to data objects instead of hosts. This architectural decision has two important implications from the perspective of InfoMax design. First, the network and transport layers are aware of application-level object boundaries. This is because objects have explicit names that the network is aware of. Second, since a hierarchical naming scheme is used, inferences can be made about semantic overlap between named objects based on the distance between the corresponding names in the name tree. These two insights allow us to develop an information-loss-minimizing protocol for data sub-sampling.

Unlike the IP paradigm, which is *push-based* (where a sender decides what a receiver gets), InfoMax lies atop of NDN, which is *pull-based*. The receiver pulls data from the network by name. (Note that, in NDN, since data are named, it does not matter who the sender is.) Hence, InfoMax offers a primitive for requesting a data set specified by a name prefix. The prefix names the root of a content tree with the understanding that a sampling of the content tree is to be transmitted to the requesting node. The requesting node does not need to know what content is stored in the tree and does not know the tree name space. It just wants the data under the tree at a configurable level of summarization (i.e., sub-sampling). Since it is the receiver of the data that controls the amount of information transfer, the receiver can pull less content or more content, until the receiver's content needs are satisfied. Hence, besides specifying the name prefix that decides the data set, the receiver controls the degree of summarization by specifying how much information is actually pulled from the named tree.

The novelty of InfoMax lies in the order that objects belonging to this tree are transferred. InfoMax assumes that objects that share a longer prefix within the tree have more semantic overlap. Hence, the marginal utility of sending an object that shares a longer prefix with a previously transmitted one is *less* than the marginal utility of sending an object that shares a shorter prefix. This leads to a *shortest shared prefix first* transmission order for objects in the requested tree. This order is shown to maximize the marginal utility of transmitted objects and hence the total accumulated utility for a given transmitted data volume. Said

differently, a transmission can be stopped at any time, short of transmitting the whole tree, while ensuring the property that the information loss resulting from early termination is minimized over all possible object transmission orders. Since, in a pull-based architecture, it is the receiver that controls when transmission stops, the receiver gets to decide when they've had data at a fine-enough granularity. The idea is reminiscent of receiver-driven multicast [34], implemented in an application-independent fashion.

InfoMax extends a previous transport layer abstraction, called the *Information Funnel* [35], proposed earlier by the authors. While the latter was geared for data *collection* by a single consumer from multiple producers, InfoMax is geared for the complementary case of data *dissemination*, where multiple consumers pull data from the same producer, possibly at different levels of summarization. Together, the Information Funnel and the InfoMax dissemination protocol may constitute the two building blocks of many data-intensive services. The funnel will collect data into repositories from multiple sources, whereas InfoMax will serve it to multiple eventual consumers at different degrees of detail. We also significantly simplify the data ordering algorithm, compared to the algorithm used by the Information Funnel, while ensuring that it maximally benefits from network caching within the NDN Layer. The protocol was implemented and tested on the NDN testbed. At the time of writing, the testbed includes more than 15 institutions, spanning three continents (although our tests were performed in the continental US). Two applications are illustrated; one from the domain of tourism and the other from the domain of social network search. Results demonstrate the advantages of using InfoMax in terms of achieving a configurable receiver-decided trade off between data volume received and degree of detail. We also show that the scheme offers a relatively low overhead and that it leverages native NDN in-network caching.

4.2 DESIGN AND IMPLEMENTATION

In this section, we discuss the InfoMax abstraction, design decisions, and implementation on top of the named-data-networking stack.

4.2.1 The InfoMax Information Summarization Abstraction

The InfoMax protocol is designed to sample data of applications that store and serve significant amounts of largely homogeneous, and partially redundant data. An example application class that fits this category is social networks. For instance, Twitter serves data (collected from a large population) as JSON objects; each object is a tweet together with its metadata. Similarly, Instagram and Flickr serve images; each image includes a picture file

and EXIF metadata. These data sets are large and syntactically homogeneous, but amenable to hierarchical clustering based on semantic content. For example, recent work suggested a hierarchical clustering framework for tweets [36], that significantly speeds up indexing to retrieve tweets that match arbitrary keywords. The hierarchy arises because some tweets are more similar than others, as can be measured by various text similarity metrics. At the top of the hierarchy are tweet clusters that have no text overlap. Subsequent levels introduce tweets with progressively more overlap with their siblings. If a name space echoed the above hierarchy, tweets whose names have a longer shared prefix would be more similar.

Cyber-physical systems offer another application class that gives rise to syntactically homogeneous data objects that semantically fall into hierarchical structures. For example, a campus building management system might collect data on temperature in different campus spaces. Such data may be organized hierarchically by building, floor, room, and temperature sensor [37]. Clearly, data from sensors in the same room would be more similar than data from different floors or buildings. If the data were named hierarchically as described above, items that share a longer name prefix would generally be more correlated (being from the same or close physical spaces). With the rise of projects such as the Internet of Things, and the increased ubiquity of sensory instrumentation, the amount of generated sensor data is going to increase making cyber-physical data sets more widely available on networks.

The InfoMax protocol is designed as a general protocol for requesting a sampling of such data sets that summarizes them at a receiver-driven degree of granularity. Name space design is clearly a key consideration when using InfoMax, in that it determines the output of summarization. The examples above illustrate that the name space in important categories applications lends itself naturally to a hierarchical organization that is suitable for InfoMax. The main API calls provided by the InfoMax producer and consumer, along with their uses, are summarized in Table 4.1 and Table 4.2, respectively.

4.2.2 A Note on Optimality

The InfoMax protocol was designed to (i) minimize overhead, (ii) make the best use of the underlying NDN stack, and (iii) offer a general sub-sampling function that is application-independent. By sub-sampling, we do not mean returning an arbitrary subset of requested objects. What is the interesting about InfoMax is that it retrieves the subset that *minimizes* information loss (compared to receiving the entire set), given the fraction of retrieved objects.

Definition 3-1. The name space assumption: The fundamental assumption that InfoMax makes regarding an application's name space, is that data objects have hierarchical

Table 4.1: InfoMax Producer APIs

`InfoMaxProducer (Name prefix):` Constructor for the InfoMax producer object, `producer`. It is called by an application to initialize the producer and specify a name prefix associated with this producer. The prefix specifies a content name space that the producer should make available on the network.
`add (Name postfix, Data data):` A method of the producer, `producer->add` to add data objects to its tree. The call specifies the data object that needs to be added along with its position in the tree.
`delete (Name postfix):` A method of the producer, `producer->delete` to delete the specified tree branch.

Table 4.2: InfoMax Consumer APIs

`InfoMaxConsumer (Name prefix):` Similar to the InfoMaxProducer, this is a constructor for the InfoMax consumer object, `consumer`. A prefix is specified that the consumer is interested in. It creates the receiving end for all content that matches this prefix.
`get ():` This method, `consumer->get`, waits to receive the object from the network that match the prefix requested by the consumer (or timeout). The method may be called multiple times to receive the set of matching objects. The InfoMax protocol continues to retrieve objects until the application stops calling this method. The application at the receiver can continue to retrieve objects until the desired level of detail is met.

names that respect the following property: The longer the shared name prefix between two objects, measured in the number of shared name segments (i.e., tree levels), the lower the marginal utility of receiving the second object.

In accordance with the above assumption, objects are retrieved in a *shortest-shared-prefix-first* manner. Below, we offer a proof sketch of why this retrieval policy minimizes information (utility) loss.

Theorem 3.1: *Shortest-shared-prefix-first retrieval order minimizes receiver's information utility loss.*

Proof Sketch: Let us define *information utility* of an object at a receiver as a measure of value that the receiver gets from receiving the object. Consider a name prefix, advertised by a producer, that names a tree, T, containing a total of N data objects. Each object, $D_i \in T$

has an information utility, U_i, when received. Given the *name space assumption* mentioned above, U_i is a function of the length of the shared prefix between D_i and previously received objects. Let us denote the length of the largest such shared prefix by S_i. Hence, $U_i = f(S_i)$, where f is a non-increasing function. Shortest-shared-prefix-first implies that the sent object will minimize S_i over the remaining (unsent) object set, $R \subset T$. Hence, it will maximize $f(S_i)$. Since $f(S_i)$ is maximized for each sent object, it is possible to prove by induction over the number of received objects, K, that the sum $f(S_i)$ over the K objects received in InfoMax order is maximum among all possible sums of K objects. Hence, the utility sum over the remaining (unsent) object set, R, is minimum, which is defined as the *information utility loss* should transfer stop after K objects. Hence, InfoMax minimizes information loss for any K. □

Two observations should be made here. First, the above proof hinges on the assumptions that (i) U_i is *only* a function of S_i, denoted $U_i = f(S_i)$ in the proof, and (ii) the function f is non-increasing. The optimality result does not hold for more general utility functions. For example, it does not hold when marginal utilities are a function of the *identities* of received objects. Second, it is notoriously hard to figure out utility values in practice. Hence, it may be counter-productive to consider more nuanced utility models. Instead, we follow the Occam's Razor principle. Accordingly, the above proof sketch develops intuitions regarding optimality in the *simplest case* that is consistent with the name space assumption (labeled **Definition 1**, above).

4.2.3 The InfoMax Protocol

The InfoMax protocol [38, 39] is implemented on top of NDN. NDN offers an API to request single data objects by name. This API is called an *NDN interest*. A consumer can send an NDN interest to retrieve a named object from the network. NDN will route that interest to the nearest node (router or host) that has a matching object. The object is then returned following the reverse route. Routers on the reverse path cache objects that they see. InfoMax uses this mechanism to retrieve a content tree by repeatedly using NDN interests to retrieve the individual objects. The key contribution of InfoMax is its implementation of the shortest-shared-prefix-first retrieval order for objects in the requested tree. In this chapter, we assume that content has a *single producer*. However, this content can be cached at multiple places in the network.

The first design question is: who should maintain the InfoMax shortest-shared-prefix-first order? When requesting a content tree that matches a prefix, the InfoMax consumer does not know what content exists in the tree. Hence, the consumer cannot request objects in the

tree in the correct order. It is therefore the producer's responsibility to compute a retrieval order for the tree.

Native NDN allows consumers to send interests in partially specified name prefixes (e.g., the root of the desired content tree) with an exclusion filter that specifies objects (matching the prefix) that the consumer already received. Unfortunately, this approach does not specify which object to retrieve next from the tree.

A trivial design would have the producer keep track of how many objects were delivered so far to each consumer. When the next request arrives from a consumer, the next object matching that consumer's interest would be delivered. The design would require producers to keep per-client state, which undesirable. Furthermore, any design that requires recipients of consumers' NDN interests to know the *next* object to send in InfoMax order (e.g., based on previously received objects) would not work in the presence of network caching. Caches (unless they run InfoMax code as well) will not know the correct InfoMax order and hence may not serve the correct next object. To keep caches from having to implement InfoMax, consumers should always ask for the *full* object name, as opposed to a partially specified name prefix.

The requirement that producers maintain the shortest-shared-prefix-first list for their content (since they know their content best), together with the requirement that consumers ask for objects by their exact full name lead to a simple design where:

- *At the producer:* The transport entity sorts objects in the tree (under the exported name prefix) in a shortest-shared-prefix-first delivery order. This sorted order is recorded, broken into smaller objects, called *lists*. These lists are numbered and made available by the transport entity to be retrieved sequentially by consumers.

- *At the consumer:* Given a tree prefix, say *root*, to retrieve objects from, the transport entity requests the aforementioned numbered lists using default names, such as *root*/*list*/1, *root*/*list*/2, etc. (In our implementation, the default list names contain special characters not used in naming data objects, in order not to clash with data object names.) Once a list is retrieved, the transport entity at the consumer parses it for data object names, and requests the listed objects by exact name in the order they appear on the list.

The approach allows for both list objects and data objects to be cached. The cache need not run InfoMax code. It simply responds to interests in named list and data objects, either serving them or forwarding the interest further towards a producer.

The size of the lists turns out to be an important design parameter that can significantly impact communication performance. A small list size could lead to frequent exchange of lists in order to determine which data objects to retrieve next, thus leading to more network traffic. A large list size can increase overhead if the consumer does not need that many data objects in their tree summary. Since the network-layer packet in NDN is 8KB long, we fixed our list size to 100 object names, which is roughly 8KB in size. Hence, list objects take one full NDN packet. This is suitable for transfer of large numbers of data objects, which is the common case in the data intensive applications that motivate InfoMax. Whenever the consumer needs more data objects, it can ask for the next list and will get names of the next 100 objects.

Another important parameter to consider is the interest pipelining window size. That is to say, how many objects from a list should the consumer's transport entity request in parallel? This parameter is exported by the NDN layer. Its effect is empirically determined in the evaluation section.

The approach mentioned above is suitable for retrieving primarily static content. If the tree at the producer is updated often, two problems occur. First, extra overhead is paid in recomputing the lists that maintain the sorted order of objects in the tree. Second, if the lists are updated while some consumers are retrieving content, retrieval of inconsistent list versions may result in an incorrect data object retrieval order.

To reduce the overhead of recomputing the lists (as well as the number of list versions generated and hence potential cache pollution with these versions), the protocol controls the frequency at which the lists are recomputed. For example, the transport entity at the producer may recompute lists when a new object is added or 10 minutes after the last recomputation, whichever is *longer*. This imposes a minimum list update period and hence bounds overhead.

To prevent inconsistent list versions from jeopardizing delivery order to a consumer, a limited number of past versions may be maintained at the producer's transport entity. The transport entity at the consumer notes the version number of the first list it receives, $root/list/1$ and, in the rest of the data transfer, requests lists of the same version number.

Having said the above, we should note that InfoMax envisions data that does not change often. For example, once sensor data objects are collected, they are stored for a long period of time without modification. Similarly, once users issue tweets or images, they live in the blogosphere without modification until obsolete. The most common case of updates is to add new objects or new branches to the tree. We recommend that, in applications featuring frequent updates, the first branch under the consumer's prefix, *root* should refer to a time window. Hence, each window, a new tree is constructed from objects in that window. That

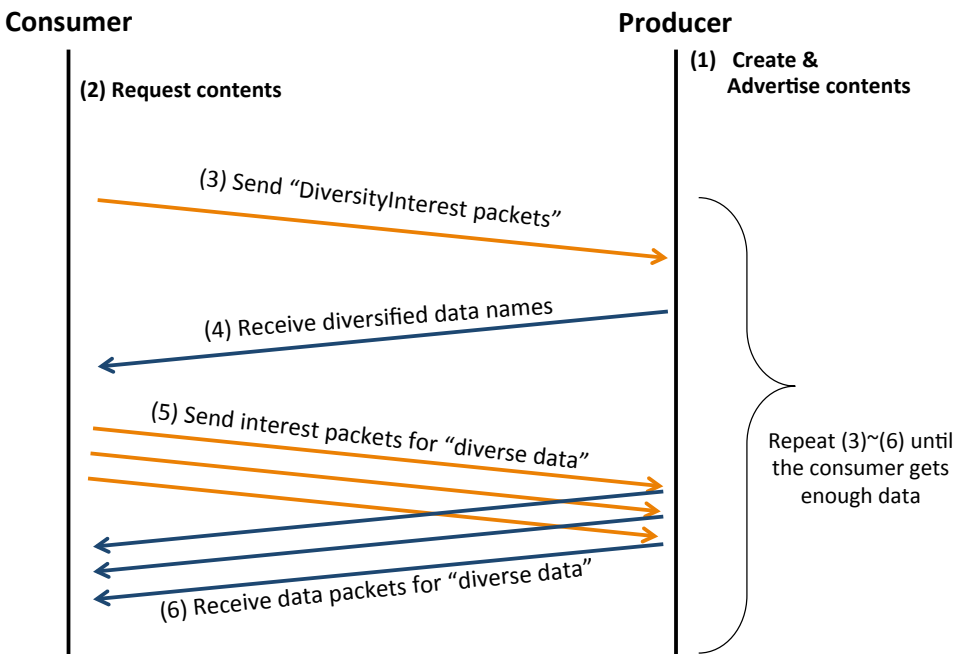


Figure 4.1: Communication flow of InfoMax

tree then remains fixed for the rest of time, once a new window starts. A consumer can request specifically the window(s) that they need under *root*.

Note that, name space design, is in general an important application-level issue for applications that serve data over information-centric networks. Application designers must consider other design implications as well, such as security and ease of routing. Those implications may be orthogonal to the naming hint suggested above, as they tend to affect the composition of the prefix *root* itself, more so than the topology of the descending tree under it.

Figure 4.1 depicts an example flow of the InfoMax protocol. In step 1, an application offers a new name space by creating an InfoMax producer. In step 2, a consumer is created who would like to receive a sampling of this content. In step 3, the consumer's transport entity sends an NDN interest in the first list object (called a "diversity interest" in the figure). The interest is routed by NDN to whoever has this list. Unless the content is already cached somewhere, the NDN interest packet makes it to the producer. On receiving this packet at the producer, the NDN layer invokes a callback to the InfoMax transport entity. The entity sends back the first list with the names of data objects that the consumer should ask for (step 4). On receiving this list, the consumer requests the data objects (step 5). These requests are fulfilled by whoever has the content. In the figure, the content is fulfilled again by the producer (step 6). The consumer may request more content, repeating steps 3-6 until satisfied or until all content under the tree is collected.

4.2.4 An Approximate Transmission Ordering Algorithm

An efficient implementation of the shortest-shared-prefix-first delivery algorithm is needed to reduce InfoMax overhead. In prior work, the authors proposed an optimal algorithm for diversifying data objects based on name similarity [35]. However, this algorithm is relatively slow. Instead, we developed a simplified algorithm that works as shown below.

Algorithm 1: Approximate least-shared-prefix algorithm

```
simplePrioritizer(Node root, Node[] list)
while unvisited leaf nodes in tree do
  | list.insert(getNextNode(root));
end

Node getNextNode(Node currentNode)
currentNode.count += 1;
if currentNode == leafNode then
  | return currentNode;
end
for child = currentNode.children do
  | if child has the smallest count among siblings with unvisited leaf nodes then
  |   | getNextNode(child);
  | end
end
```

The algorithm starts with all branch counts initialized to zero, then traverses the tree from the root node taking, at each level of the tree, the least visited branch and increment its count. If there is a tie, the leftmost branch (of those involved in the tie) is taken. Once a leaf node is reached, the object at the leaf is returned and its name appended to the output list. The traversal starts again from the root for each subsequent object. The algorithm terminates when all leaves have been visited. The list generated in this process approximates shortest-shared-prefix-first. This is because an object that has the shortest shared prefix with previously listed ones must descend from some branch, B, not yet traversed, whose sibling have been traversed already. The count of that branch would therefore be zero. When the algorithm reaches the parent node at branch, B, say, node P, it will take branch B because it has the smallest count (specifically, its count is zero). Hence, the algorithm will correctly choose the branch leading to a shortest-shared-prefix object. The problem here is that the greedy nature of the algorithm might cause it to take a different branch earlier on, hence never reaching node P. This occurs when there is a tie in the counts somewhere closer to the

Figure 4.2: Top-level query – World disasters in 2017

root of the tree, causing the algorithm to descend into a different subtree (i.e., away from P). We investigate the efficacy of this algorithm in the evaluation section.

As a simple example, we created an disaster aftermath application. It allows consumers to browse pictures of world events in 2017 at destinations of choice. The pictures are organized hierarchically first by event (e.g., "disaster", "protest"), then by country, then by sub-region that belong to each category, and finally pictures of each protest.

Suppose a consumer, a analyst, queries for the world disasters. The client software issues an InfoMax request to download pictures under the "/Disaster/World" prefix. Figure 4.2 shows the first 6 images retrieved. We observe that the pictures are spread across various branches in the content tree, including California wildfire, Iran earthquake, and other regions. They give a diverse view of world disasters overall. A random retrieval of these pictures would likely yield much redundancy. Instead, by following diverse top level branches first, the shortest-shared-prefix-first algorithm automatically retrieved a diverse set of pictures. No application semantics were needed to achieve this diversification.

Next, the consumer wishes to take a closer look at the disasters in United States. The client software queries InfoMax for "/Protest/Disaster/US". The results are shown in Figure 4.3 Again, they come from different branches at this level of the tree, showing different type of disasters, wildfire and hurricane. Finally, the consumer wishes to see the hurricane images. The software queries InfoMax for "/Protest/World/Disaster/US/Florida". At this level only hurricane Irma pictures struck Florida are retrieved as shown in Figure 4.4.

Two points are notable in this example. First, the retrieved pictures, in each case, constituted a sampling of a data set that contained *orders of magnitude more matching entries* to the consumer's specific query, most of which were partially redundant. Among these

Figure 4.3: Second-level query – U.S. disasters in 2017

Figure 4.4: Query on the Florida disasters in 2017s

matching entries, an appropriately diverse small sampling was returned. Second, such diversification of retrieved information occurred with no use of application-specific information. It was simply an artifact of following the generic shortest-shared-postfix-first retrieval order by the InfoMax transport entity.

4.3 DOMAIN OF APPLICABILITY

InfoMax mainly targets the high echelons of tactical networks, a large-scale wire communication such as a division. This design choice is because redundancy minimization can be more beneficial for big data applications, where there are too much data to send everything. For example, the command center may have a large volume of information that is hard to disseminate. In this case, the command center first needs to organize the collected data in the least-redundant order so that subordinate units can sample them until situation awareness has been achieved. This example can apply for the low echelons of the tactical networks, a small-scale wireless network such as a platoon, while voice messages can be more efficient in this scale.

4.4 EVALUATION

In this section, we evaluate InfoMax performance. This includes evaluating the overhead, investigating the impact of network caching, and assessing the degree to which the algorithm approaches the optimal shortest-shared-postfix-first transmission order. The experiments were done on a nation-wide testbed implementing the NDN protocol. There are 17 institutions participating in the NDN testbed in the world. The testbed map can be found in the NDN site [40]. These nodes are created for research purposes. Hosts can be connected to NDN routers by configuring and executing an NDN forwarding Daemon [41]. We used the above testbed for following experiments.

4.4.1 Transmission overhead

In order to measure end-to-end delay between a producer and a consumer, we created a producer in the Midwest (connecting to the UIUC NDN router) and a consumer in California (connecting the UCLA NDN router) as shown in Figure 4.5.

To evaluate the extra overhead of InfoMax, we then created an additional consumer (also at UCLA) that is "clairvoyant" in that it knows exactly the topology of the producer's data tree. This consumer therefore simply requests data objects one at a time by name using

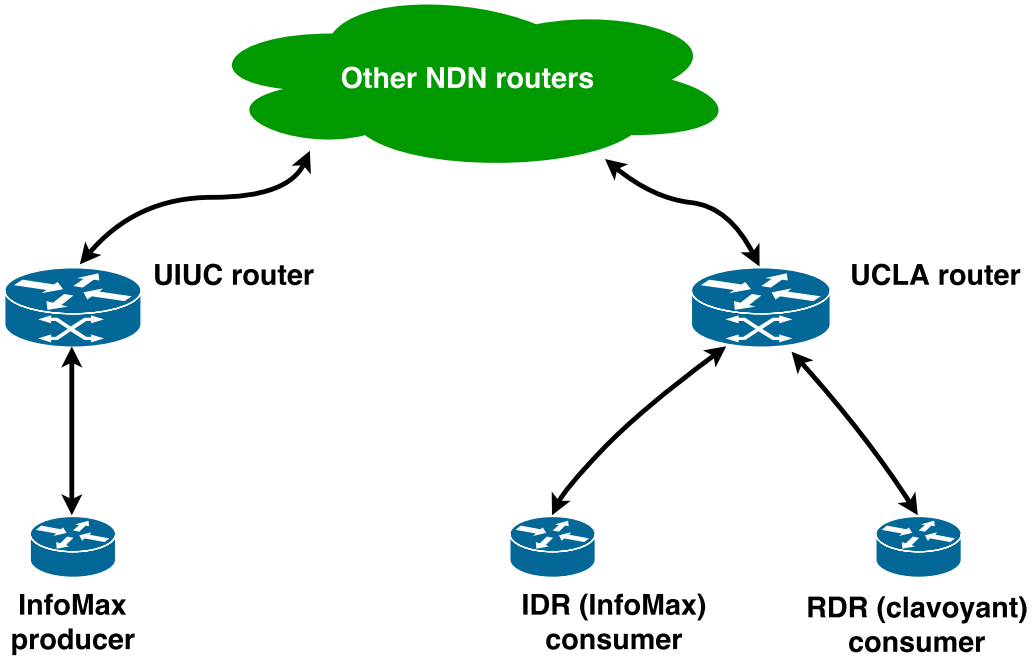


Figure 4.5: Topology for measuring end-to-end delay

RDR, bypassing the InfoMax transport. This clairvoyant consumer is thus expected to show the best achievable performance in terms of latency and throughput.

We perform the experiment varying the number of requested data objects. In Figure 4.6, the X-axis denotes the number of data objects requested and the Y-axis denotes latency per one object. Considering that InfoMax exchanges additional packets and parses the list packets for data object names, we expect the base (clairvoyant) client to outperform InfoMax. However, the added latency is not significant. Moreover, this extra latency could mitigated by successful NDN caching as will be shown in the following experiment.

4.4.2 Scaling delivery

A key benefit of NDN is in-network caching. InfoMax is carefully designed to leverage the NDN router cache. To demonstrate, we measure transmission rate at the producer side while increasing the number of InfoMax consumers. If NDN caching works well, the load on the consumer should not change. It should remain the same as with only one consumer. Figure 4.7 shows the network topology used for this experiment. As before, the InfoMax producer is in the Midwest (connected to the UIUC NDN router). Four consumers are logically connected to different NDN routers across the testbed (UCLA, University of

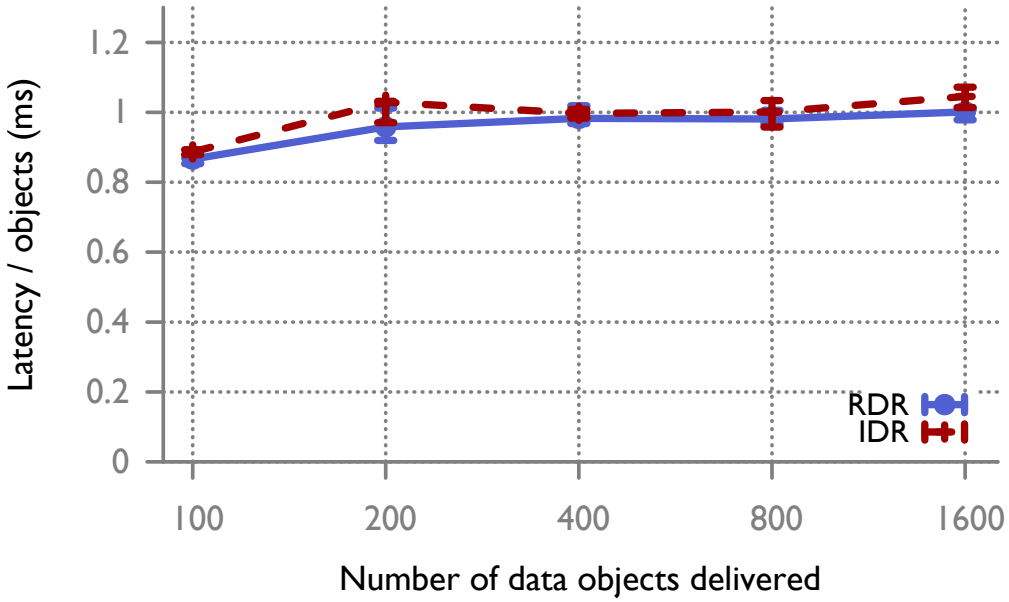


Figure 4.6: Amortized per-object end-to-end delay as a function of the number of objects

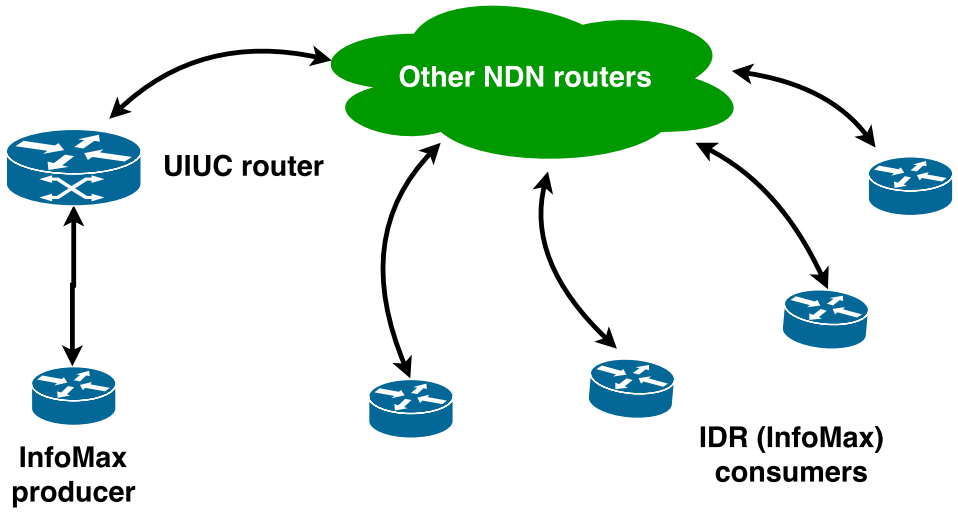


Figure 4.7: Topology for testing NDN caching with multiple consumers

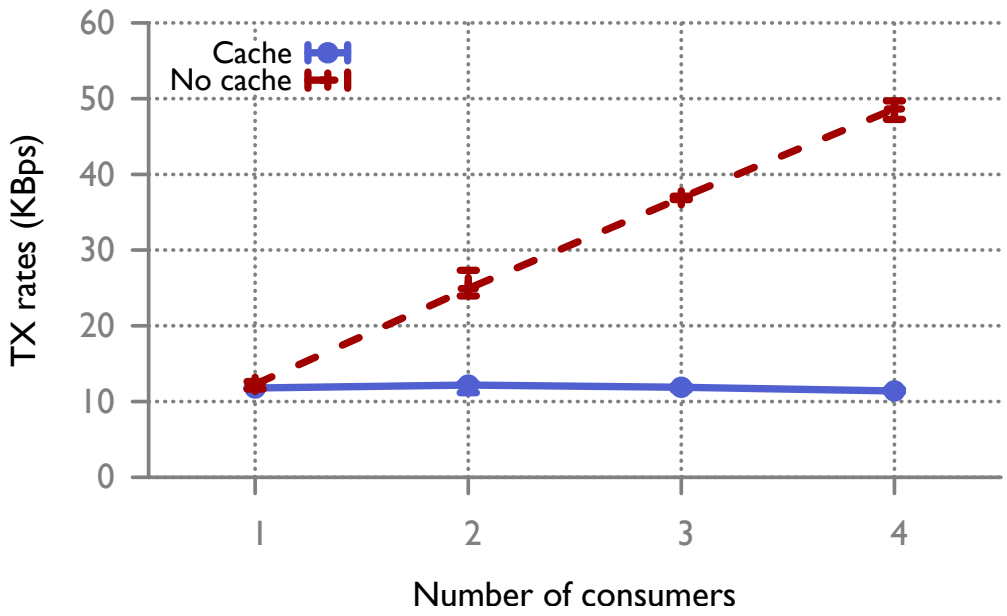


Figure 4.8: Transmission rate of a producer with multiple consumers

Michigan, Colorado State University, and VERISIGN lab).

Each consumer is programmed to request 100 data objects, repeatedly. We compare what happens when cache is enabled and disabled, respectively. The resulting transmission rate of the producer is shown in Figure 4.8. In the figure, the X-axis and the Y-axis represent the number of consumers and the total transmission rate at the producer, respectively. The solid line in Figure 4.8 shows the producer's transmission rate in the presence of caching. Note that, it remains almost same as the number of consumers increases. This is because of successful NDN caching. Next, we disabled the cache function by setting cache time-to-live to be very short (1ms). As can be seen from the dotted line in Figure 4.8, the resulting transmission rate of the producer increases almost linearly with the number of consumers in this case. The experiments show that InfoMax does not increase the producer's transmission bandwidth needs.

4.4.3 Shortest-shared-postfix-first Ordering

InfoMax provides an approximate implementation of the shortest-shared-postfix-first order that approaches optimal behavior with low overhead. In order to show it, we first devise an experiment to measure how close InfoMax comes to an optimal shortest-shared-postfix-first algorithm. The optimal algorithm was described in the author's prior work and is used as the comparison point [35]. For this experiment, a random tree generator is developed to create a diverse set of trees by controlling the total number of nodes, N, and the maximum

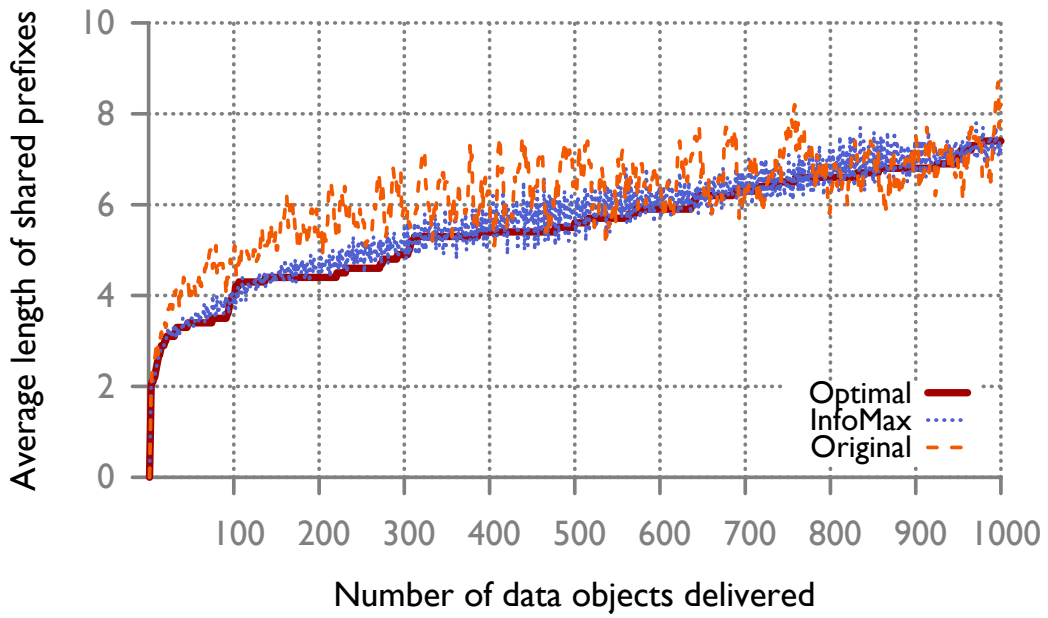


Figure 4.9: Average length of shared postfix (in number of shared name segments) as a function of objects transmitted

number of children, C, under one parent. Different trees are created (with $N = 1600$ and $C = 20$), and the output of an optimal implementation of shortest-shared-postfix-first is compared to the approximate Infomax algorithm. Figure 4.9 and 4.10 shows the result of this comparison, where the X-axis denotes the number of objects transmitted, and the Y-axis denotes the average number of shared segments (tree levels) between the current object name and previously sent data.

As can be seen in the graph, the number of shared segments in the name postfix increases monotonically as more objects are transmitted, indicating that diverse objects are transmitted first (i.e., those with a smaller postfix overlap) followed by progressively more redundant objects (with a higher postfix overlap). In application terms, the degree of redundancy increases as more objects are sent. Note also that the InfoMax curve is very close to the optimal, whereas the regular NDN curve (which, for the purposes of this experiment, corresponds to a leftmost child first traversal) is significantly higher meaning that more redundant objects are sent earlier, thereby reducing transmitted information value.

We perform the same experiments with two different sets of trees created by our generator. Both sets have the same number of nodes in a tree (N=1600), but different maximum number of children (C=5, 100). Trees having smaller C are deeper and those having bigger C are wider and shallower. Figure 4.11 and 4.12 shows that deeper trees gain more from our algorithm in that the separation between the InfoMax and NDN curves is larger. This is because leaf nodes in a deeper tree have longer names, so the impact of a shortest-shared-

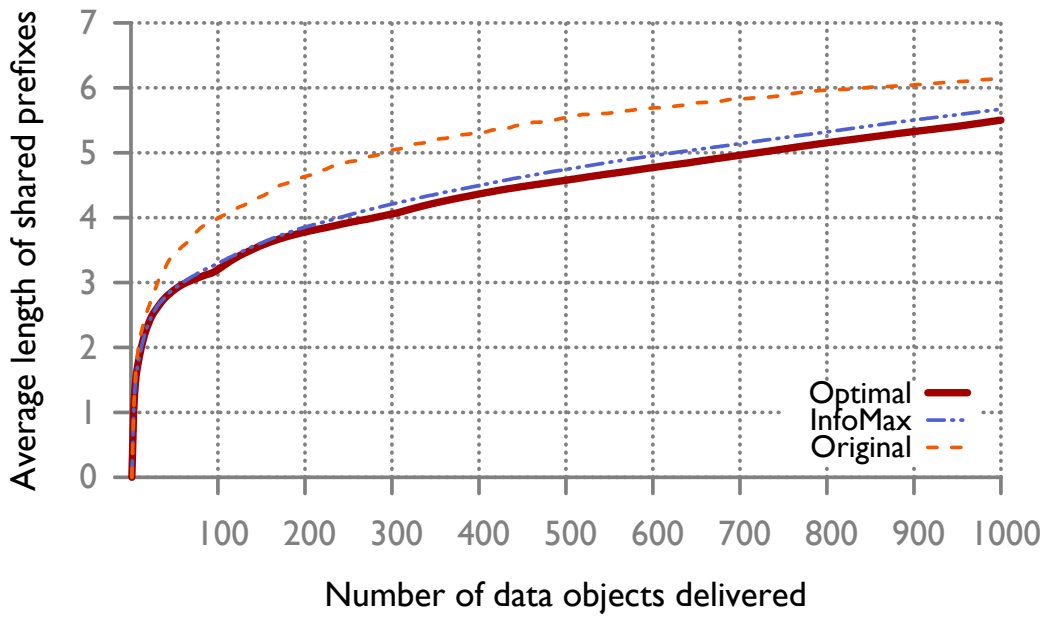


Figure 4.10: Smoothed curve for Figure 4.9

postfix-first strategy is more pronounced. In the limit, if the tree depth is equal to 1, there would be no difference among the algorithms.

As discussed in the previous section, we provide two types of algorithms for diversification; a top-down and a bottom-up. Figure 4.13 shows computation time of both algorithms for a different number of data objects. Computation time for the top-down algorithm linearly increases with the number of leaf nodes (N) in a tree as its complexity is $O(H \cdot N)$, where H is the height of a tree. Compared to this, the bottom-up algorithm grows slightly faster because its complexity is $O(N \log N)$ inherited from merge sorting. The bottom-up algorithm, however, generates ordered lists for all the sub-trees at once, whereas the top-down algorithm generates a list for only one tree. In this sense, the bottom-up algorithm could be better because a tree with N leaf nodes can have at most $N - 1$ sub-trees by induction. Hence, if an application needs samples for all sub-trees, the bottom-up algorithm can actually outperform the top-down algorithm.

4.5 CONCLUSIONS

In this chapter, we proposed a new transport layer protocol to sub-samples big data sets. This protocol is built on top of NDN and exploit its hierarchical name structure. By observing that data objects with a longer shared prefix have more semantic similarity, InfoMax produces (an approximation of) the shortest shared prefix order among transmitted data ob-

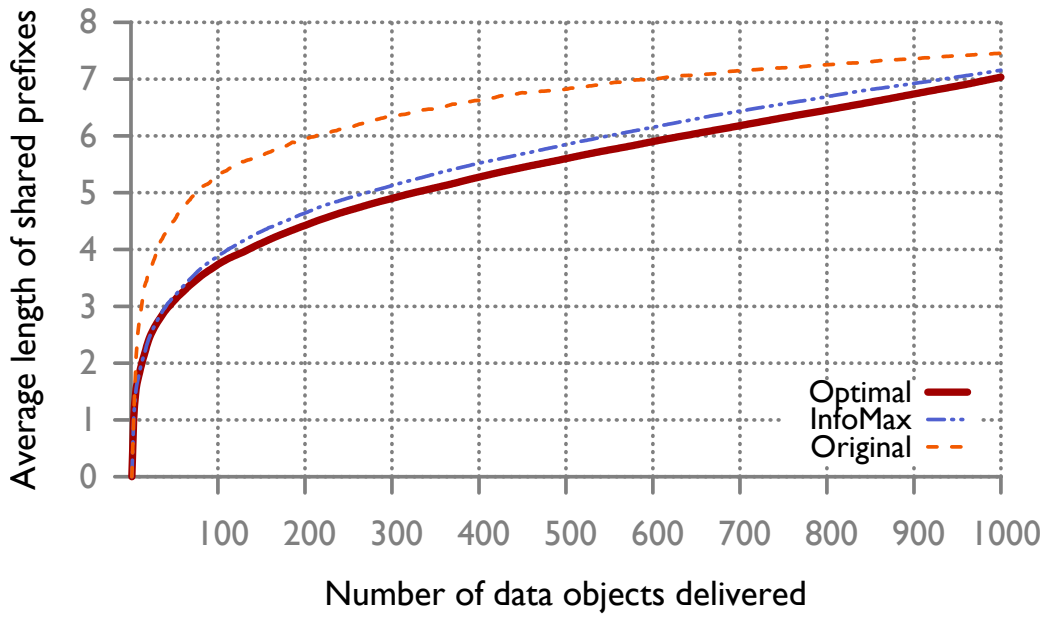


Figure 4.11: Average length of shared postfix (in number of shared name segments) as a function of objects transmitted for deep trees

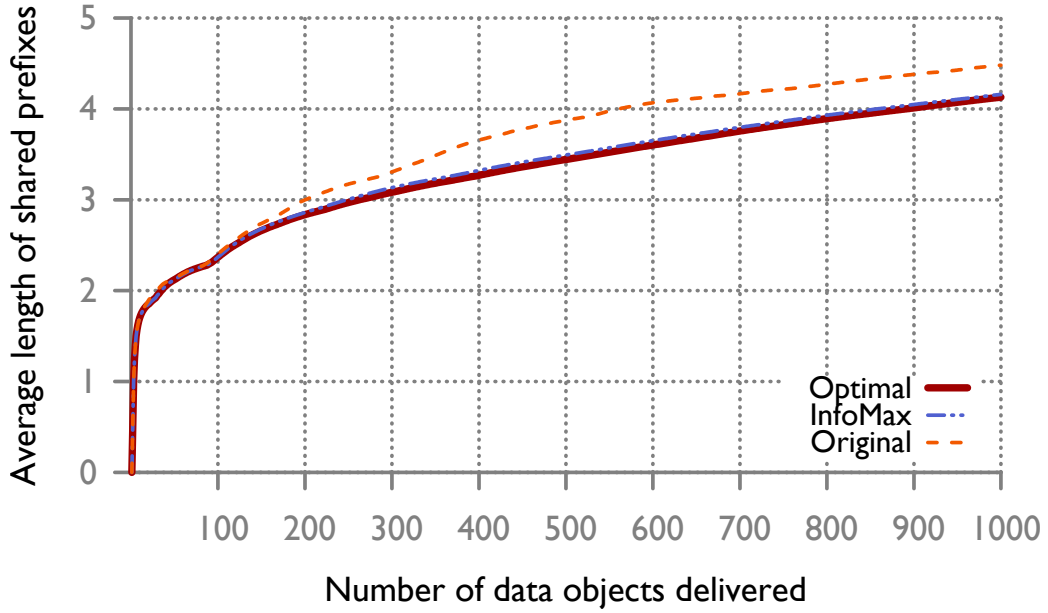


Figure 4.12: Average length of shared postfix (in number of shared name segments) as a function of objects transmitted for wide trees

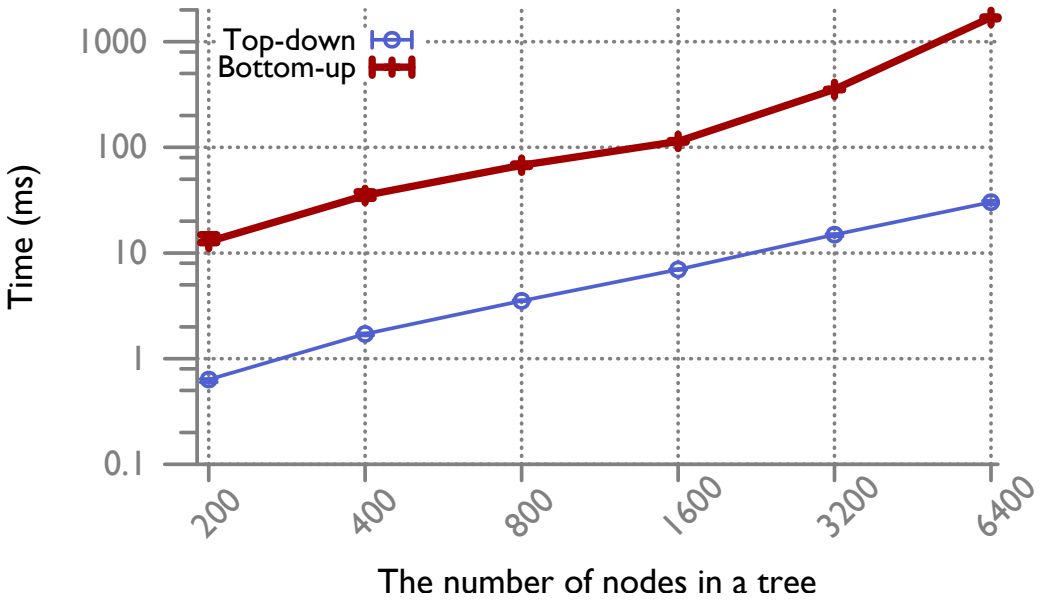


Figure 4.13: Computation time for each prioritization algorithm

jects to minimize data redundancy. It allows multiple consumers to retrieve data at different configurable levels of detail from one producer while maximally leveraging NDN caching to minimize producer overhead. InfoMax has been deployed on a nation-wide testbed. Analysis of long-term experiences with this deployment is deferred to a subsequent publication.

CHAPTER 5: AUTOMATED DATA NAMING SERVICE

Recent work suggested that, in the age of data overload produced by sensors, social media, and IoT devices, a key new type of network transport protocols will be one that offers *representative summaries* of requested data, retrieved at a consumer-controlled degree of granularity. Given the over-abundance of data, consumers will seldom need *all* data on a topic, but rather will increasingly favor an appropriate *sampling* for summarization purposes. The chapter explores such sampling as a novel service enabled by information-centric networking paradigms that name data objects, not hosts. By naming data objects, it becomes possible to selectively retrieve them, but the properties of the resulting sampling depend on the naming scheme. This chapter describes an automated object naming service, called Espresso, that facilitates content sampling over information-centric networks. We show how Espresso, combined with a trivial retrieval policy, translates the sampling problem into a naming problem, and customizes the naming to different applications' sampling needs. Experimental results show that the computational overhead of automated naming is affordable. The service is first evaluated in simulation, demonstrating a higher sampled-data utility to the consumer, while balancing retrieved data importance and diversity. Social network applications are then introduced, where naming is produced by Espresso. Results demonstrate the advantages of Espresso, compared to baselines, in terms of retrieving meaningful media data summaries.

5.1 INTRODUCTION

This chapter describes *Espresso*, a data naming service for the age of sensing, social media, and the Internet of Things. The goal of this service is to automatically name content in a manner that facilitates efficient retrieval of extractive *summaries*[1] over information-centric networks (ICNs). Recent work on ICNs argues for use of machine-generated names [42], because users are less interested in remembering cumbersome identifiers (even when they are human-readable). In this chapter, names are automatically generated to support *content summarization* at different degrees of granularity.

The work is motivated by the view that modern sensing and social media applications will increasingly need content sampling services to combat data overload [43]. Data users will require well-selected samples of available data, as opposed the entire set. This is already

[1]Extractive summarization refers to summarization achieved by extracting samples of the data set being summarized, as opposed to generating a brief high-level description.

true of social networks, for example, where users who search for content on a topic (e.g., via Twitter or Instagram) do not really need to see all matching tweets and pictures, but rather a representative selection thereof – a well chosen subset that summarizes the key issues. Thus, services are needed to support sampling or summarizing content.[2]

Espresso adopts a producer-consumer model proposed in recent literature [43]. The producer exports a data set of interest. Different consumers want to retrieve samples (*i.e.*, summaries) of the data set or parts thereof, at a consumer-specified degree of granularity.

We consider summarizing data sets of *homogeneous objects*, such as sets of microblog entries, pictures, news headlines, or product reviews. An ideal sampling method should select objects in a manner that maximizes some notion of information utility of the retrieved summary. Intuitively, an object should be included in the summary if it is both (i) more important in some application-specific sense and (ii) less redundant with previous objects already included in the summary.

ICNs can map the problem of *content sampling* to one of name-space design if the naming can allow retrieving objects matching a query in an order that meets the two aforementioned criteria. *Espresso* is such a naming service, developed for ICNs with hierarchical name spaces.

Espresso names data hierarchically, based on an application-specific content similarity metric, such that more similar data are closer together in the name space. It allows consumers to retrieve content summaries by traversing the name space in a particular generic order. Moreover, applications can zoom in and out to retrieve content at different degrees of detail. We describe the design and implementation of *Espresso* as well as its evaluation both in simulations and in the context of a Twitter-based application.

Simulation results demonstrate that *Espresso* achieves a higher utility for retrieved data than compared baselines. An empirical evaluation of a Twitter-based news application supports the same observation. The evaluation shows that consumers can control the level of summarization (of news extracted from Twitter) by zooming in or out on the topics.

5.2 NAMING, RETRIEVAL ORDER, AND SUMMARIZATION

To explain the relation between naming, retrieval policy, object retrieval order, and data set summarization, consider the hierarchical name tree shown in Figure 5.1. We shall use it below to illustrate how the problem of representative sampling can be translated into one of name space design in information-centric networks that support hierarchical names.

[2]In the rest of this chapter, we use the term *sampling* and *summarization* interchangeably.

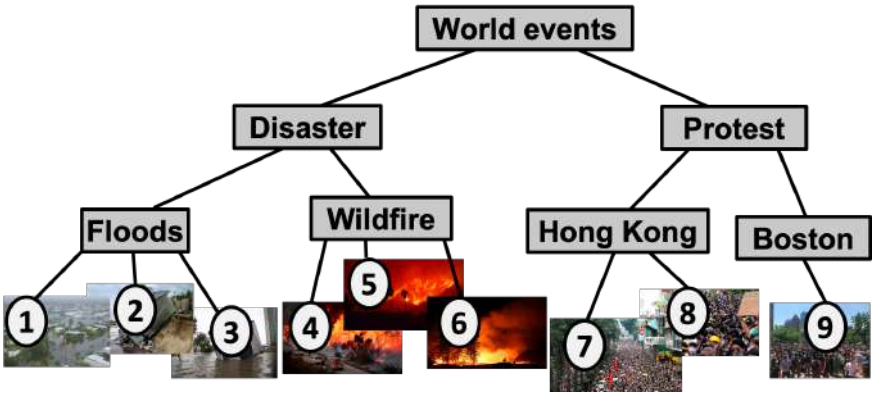


Figure 5.1: World event data set

5.2.1 Representative Sampling

Let us first informally discuss what we mean by representative sampling. In general, representative sampling is a sampling that preserves the desired statistic of the sampled phenomenon. Hence, what constitutes a representative sampling of data differs from one application to the next depending on what statistics are sought. If the goal is to determine average traffic speed on different streets, then a roughly equal number of samples from cars on each street may suffice, independently on the total number of cars on each street. However, if the goal is to tally (an approximate histogram of) good and bad reviews of a product, it is more representative if the samples of each opinion category were chosen proportionally to their prevalence, as opposed to independently of the number of reviews in the category.

In a later section, we show how *Espresso* allows applications to customize their own notion of representative sampling. For the purposes of the illustrative example below, let representative sampling refer to a sampling that groups data into categories (of partially redundant objects), and returns a roughly equal number of representatives of each category, chosen in the order of quality, as defined by the application. Below, we show a naming scheme, then explain how it leads to representative sampling.

5.2.2 Naming

Consider the tree shown in Figure 5.1. The data objects are images of various events in the world from the online application. All data objects are *leaves* of the tree. The rest of the tree comprises a hierarchical name space, where each leaf has a UNIX-like name. For example, the name `/Event/Disaster/Floods/1` refers to the leftmost picture; an aftermath image of floods.

We call the set of objects at the leaves, the *data set*. Note that, the data set is partially

redundant. Pictures are grouped into categories, each representing the same event. There are multiple pictures of each event. Furthermore, in the figure, pictures of the same event are organized left-to-right in decreasing degree of some application-specific quality metric. The metric chosen here is popularity (namely, the number of likes in the social network application). More popular image for the same events are to the left of their siblings, whereas less popular images are to the right. This is just one illustrative application-specific organization for sibling nodes by quality. If data freshness is more important, the most recent images can go to the left. Other ways to define quality are possible. In general, let siblings be organized left-to-right by decreasing value of some application-specific quality metric we call *weight*, w_i, defined for each object, O_i. (For now, let us apply this order only to the leaves.)

Finally, note that the tree is organized such that *more related events share a longer name suffix*. For example, all events related to disasters start with `/Event/Disaster`, and all pictures of floods start with `/Event/Disaster/Floods`. In this work, each data name is logically divided into two: a prefix and a suffix. The prefix is a common name of all the data within the data set, whereas the suffix denotes an individual data item followed by the prefix. Hence, the name tree has the property that objects that share a longer name suffix are potentially more redundant and similar.

5.2.3 Retrieval Policy and Retrieval Order

Intuitively, a representative sampling of the above tree would be to retrieve at least one picture of each event or higher-level event category (depending on desired granularity). We call the set of rules that algorithmically define how to choose the next object to retrieve from a name space, the *retrieval policy*. The actual sequence of objects selected when the policy is applied is called, *retrieval order*. Let us consider the following simple retrieval policy:

1. First, associate a counter with each branch in the tree (to count how many times it was traversed) and initialize it to zero.

2. To retrieve an object, start at a specified vertex, Q, and recursively descend down the branch with the lowest counter value (or the leftmost branch in case of a tie). As you traverse a branch, the counter of the traversed branch is incremented.

3. When you reach a leaf, select the object for retrieval and go to step 2 again to find the next object.

We shall call it the InfoMax policy, referring to the work where it was first presented [43]. Starting at the root of the tree shown in Figure 5.1 (i.e., Q =`/Event`), the above policy retrieves the pictures in the order 1, 7, 4, 9, 2, 8, 5, 3, 6.

5.2.4 Properties of Summary

For our example tree, the policy, mentioned above has two favorable properties. Namely, (i) it samples representatives of each cluster in decreasing granularity, and (ii) selects pictures of a given event in decreasing order of quality.

Specifically, starting with node Q =`/Event`, it first selects a picture from each event category (picture 1 from `Disaster` then picture 7 from `Protest`), then it continues the selection to include a picture of each individual event (retrieving 4 and 9 that cover the wildfire and the Boston protest after having retrieved 1 and 7, covering the floods and the Hong Kong protest). This is a consequence of the hierarchical organization of the tree by content similarity. Finally, it retrieves subsequent pictures of each event in decreasing order of quality. This is a consequence of the tie-breaking rule (that favors the leftmost branch), combined with our tree organization that orders sibling objects left-to-right by decreasing weight.

The above favorable properties are independent of the starting node, Q. For example, starting with node Q =`/Event/Disaster`, the algorithm retrieves pictures in the order 1, 4, 2, 5, 3, 6. Note how all disaster images are covered first, then additional lower weight images of each event follow.

It is important to note that the above favorable properties are not inherent in the InfoMax retrieval *policy*. Rather, they arise from interaction of the policy with the *specific way the name tree of is organized*. The example illustrates that an appropriate hierarchical naming scheme, combined with a simple and generic retrieval policy, allow retrieving objects matching a query (that specifies a starting node, Q) in a manner that achieves representative sampling.

In the presented example, the name tree was manually generated. The question addressed in this chapter is: how to do so automatically, such that the above favorable summarization properties hold for the resulting retrieval order?

5.3 DESIGN

In this section, we describe the design of *Espresso*, illustrating automatic generation of name spaces that lead to representative sampling when retrieved using the simple rules

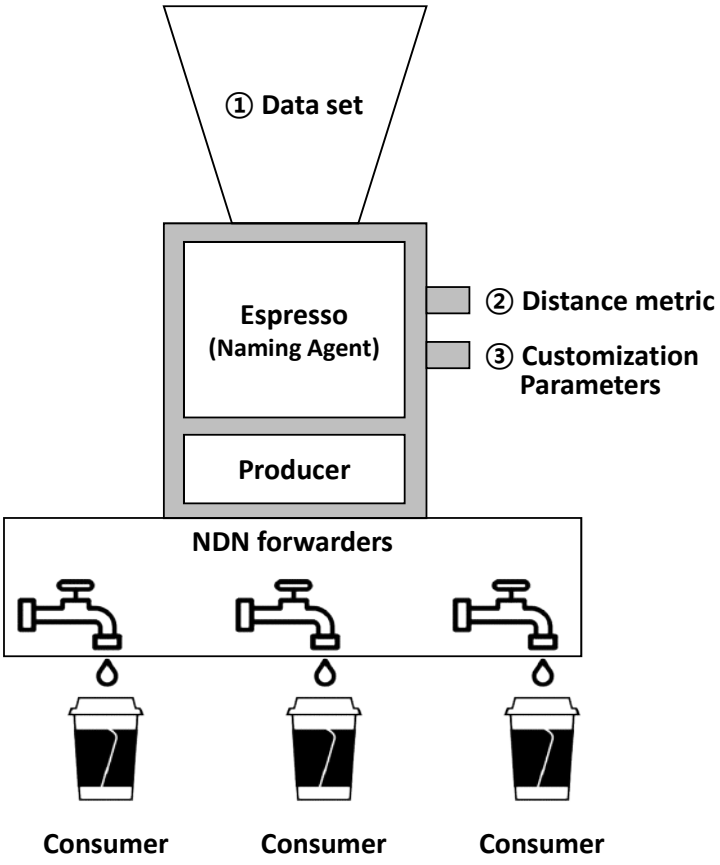


Figure 5.2: Overall architecture of *Espresso*

comprising the InfoMax policy [43].

5.3.1 Overview

The overall architecture of *Espresso* can seen in Figure 5.2. *Espresso* runs on the producer side. To name objects, it takes three inputs from the producer application: a (pointer to a) data set, a distance metric (to customize the application-specific notion of object similarity), and configuration parameters that customize the notion of representative sampling. The first two inputs customize the generation of names by the *naming engine*. The last input customizes the retrieval policy implemented by the *prioritizer*.

5.3.2 Automated Naming

The purpose of the naming engine is to build a hierarchical name space for a given data set. Note that, an automated name is a *suffix* and *Espresso* generates a full name by concatenating it with a given *prefix*. In ICNs, routing is only associated with the *prefix*. This property nicely decouples naming concerns related to routing efficiency from naming concerns related to efficiency of summarization. This paper is concerned with the latter only.

Espresso requires an application-specific distance metric that defines the notion of redundancy from the application's perspective. For example, if data objects comprise sensory data, the distance function might simply be the difference in value. If objects refer to pieces of text, a distance function might be the cosine similarity, a commonly used metric for deciding how similar two pieces of text are. If objects comprise images taken, a distance function might be the color correlogram, or might depend on metadata such as time and location of images, and associated image tags. In our Twitter application example, the Jaccard index, a text similarity metric [44], was used as a distance metric for tweets.

Inspired by the approach described in SocialTrove [36], the overall procedure is illustrated in Figure 5.3. The example shows how unstructured image data is converted into a hierarchical name space. A detailed explanation for each step follows.

Graph Constructions

The first step is to build a graph from a data set. The graph consists of nodes and edges, where a node represents a data object and an edge denotes distance between two data objects calculated by the given distance function. In this particular example, the distance metric can be defined as location distance. That is, images taken at nearby places will have a shorter edge. For example, two Berliner Dom images in Figure 5.3 may have the shortest distance among all since both images are taken from the approximately same place.

Hierarchical Naming

The second step is to build a tree from the graph in the previous stage. To this end, a hierarchical clustering algorithm is applied to group similar objects. One naive and expensive algorithm is to calculate a pair-wise distance among all data, which takes $O(n^2)$. Considering the massive volume of data being generated in this age, however, it is simply not affordable. Instead, we exploit an efficient approximation of the K-means clustering algorithm. Nodes are recursively grouped into K disjoint sets based on distance. Each set is further subdivided

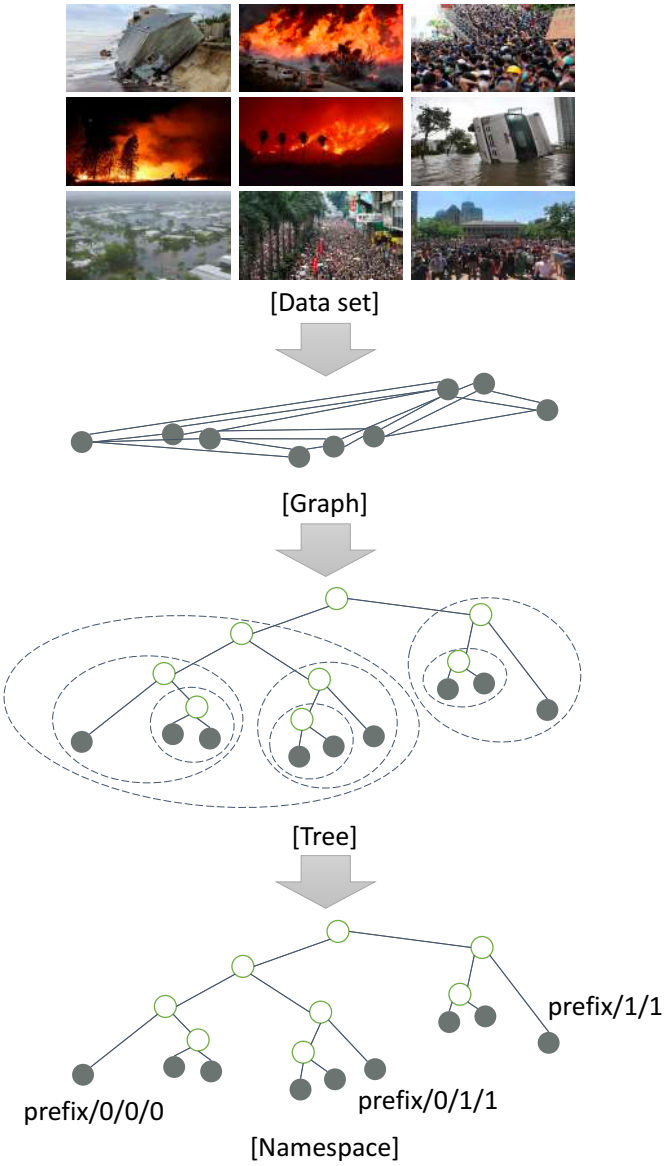


Figure 5.3: Procedure to convert a data set to a hierarchical name space

until single objects are reached. Figure 5.3 illustrates the process, where K is 2. Siblings are ordered according to the application-supplied weight metric, and leaf objects are named by concatenating the branch IDs that lead to the leaf from the root. In the case of a binary tree, a branch can have an ID 1 or 0. Our implementation currently supports only binary trees following the example of SocialTrove [36]. Figure 5.3 shows the names assigned to each leaf node.

Although the K-means clustering algorithm is known to be NP-hard, there are several heuristic algorithms that can accelerate the running time. For example, time complexity of Lloyd's algorithm is $O(wnkd)$ for k centers, n vector points in d dimensions, and w iterations until convergence [45]. Note that, w may vary depending on data points, but the Lloyd algorithm often has polynomial smoothed complexity as shown in [46] except few extreme cases. If the complexity of K-means clustering is linear, the complexity of hierarchical clustering described above would be somewhere between $O(n \log n)$ and $O(n^2)$ depending on balance of the binary tree. That is, if the tree is well-balanced, then the algorithm would be closer to $O(n \log n)$. Otherwise, it would be closer to $O(n^2)$. In practice, the time complexity of *Espresso* is even close to $O(n)$ as shown in the evaluation section.

Indeed, a clustering algorithm that is not dependent on a pre-defined number of clusters, K, would be able to construct more adaptive name trees according to data distributions. However, it is known to be hard and itself could be an interesting research topic [47]. Instead, *Espresso* allows an application to control prioritization parameters to support various sampling requirements. More details can be found in the following.

5.3.3 Customizable Prioritization

Once a tree is constructed, applications may configure how the tree is traversed. The following parameters are defined to offer configurability:

- *Weight*, w_i: defined for each object and denotes its priority among its siblings.
- *Aggregation*: a function that specifies how the aggregate weight of a node is computed given the weights of each of its children. Examples include sum or max.
- *Merge order*: a function that defines how sibling branches are multiplexed given aggregate branch weights and individual object weights within each branch.

Logically, leaves are organized in decreasing order of weight, w_i. Intermediate nodes are organized in decreasing order of the output of the aggregation function, applied to each

Retrieval patterns	Weight	Aggre-gation	Branching order
Diversity only	*	*	Least visited
Proportional to aggregates	1	Sum	Proportional to sum
High aggregates	From application	Max	Least visited, High aggregates
Above threshold	From application	Max	Least visited, Above threshold

Table 5.1: Variations of *Espresso*

sibling. To generate a retrieval order, branches are merged bottom up. When merging sibling branches, nodes of different branches are multiplexed. For example, their nodes may be strictly interleaved in a round-robin fashion, or they may be merged proportionally according to their branch weights.

In all cases, the output is a sequence that gives rise to an ordered list of objects associated with each node in the tree, leaves in the subtree hanging from that node. Below are example orders produced by the aforementioned customization:

- *Diversity only*: It samples branches equally in a round robin fashion, regardless of weights.
- *Proportional to aggregates*: This order selects more samples from the bigger clusters, proportionally to their size.
- *High aggregates*: It is the default *Espresso* order that samples branches equally, most important first.
- *Above threshold*: This order samples only those branches that have higher weights than a threshold.

Each order above can meet different application sampling needs.

The retrieval policy used is modified version of merge sorting. Intermediate nodes merge leaf nodes from bottom to top and sort them in the desired retrieval order. The process terminates after every data object gets sorted.

5.3.4 Handling dynamic updates

Espresso is primarily designed for organizing static data. If data is dynamically updated, *Espresso* needs to re-construct the hierarchy. This will introduce extra overhead especially when data changes frequently.

In order to prevent these problems, *Espresso* uses time windows. A name tree is constructed for data residing in the same window, once the window is closed (i.e., the time corresponding to the window range elapses). Hence, the generated tree never changes. New objects are put in a new window. *Espresso* allows applications to control the size of the time window in accordance with their data freshness needs, essentially balancing freshness and tree construction overhead. For example, in our Twitter news application, we have set a 1-hour time window to refresh news-feeds. Hence, generated summaries are at most one hour old.

The main reason to use the above approach is the data immutability principle of ICNs [48]. That is, data packets are uniquely named and they are immutable after being published. Deleting the existing content or updating objects would violate this principle. Naming with a version may help solve this problem, but it may cause cache inefficiency as one data item will have multiple names. The time-window-based approach guarantees that data is uniquely named, and thus complies well with the principle. Moreover, increasing the number of trees does not cause a serious scalability problem. This is because the constructed trees are considered to be small as they only contain pointers to content, not actual content.

5.4 IMPLEMENTATION

We implemented *Espresso* on top of the named data networks (NDN) code-base; specifically, the producer and consumer context. The overall procedure of exchanging sampled data objects between a producer and a consumer is sequentially outlined as follows.

Espresso names individual data objects, added by a producer, and creates a hierarchical name space for a given data set and uses it to generate a retrieval order associated with each node in the name space (for objects in its subtree) using the *Espresso* prioritizer. The retrieval order for each subtree is cut into lists of object names. These lists themselves are treated as objects having the same prefix as the objects named on the list.

A consumer application requests samples of a subtree by issuing a request (interest) with a specific prefix and a list number (starting with list 1). Upon receiving this request, the producer sends the corresponding list. Next, the consumer application requests the actual data objects in the order they appear on the list. If all the named objects in the received

list are consumed, the consumer may request the next name list to retrieve more samples. This process is iterative until the consumer decided that it has seen enough data.

Espresso is piggybacked on *InfoMax* to implement the above protocol. *InfoMax* is implemented as one of the data retrieval protocols in the consumer APIs of NDN [49]. It is designed to provide more representative samples first by transmitting data in the least-shared-suffix order. The underlying assumption is that more similar content shares a longer name suffix. In this sense, *Espresso* is easy to plug into *InfoMax* since the automated namespace naturally satisfies this requirement. Also, the *Espresso* prioritizer exploits *InfoMax* name lists to support sampling customizability. That is, the sampling order decided by the prioritizer can be not only the least-shared-suffix, but also one of several variations that meets the application needs.

5.5 DOMAIN OF APPLICABILITY

Espresso is proposed to automate the naming process and provide a customizable sampling order for InfoMax. Therefore, the domain of applicability for this work is still the high echelons of the tactical networks, the same as InfoMax, because there is more room to minimize redundancy in big data applications. Different than InfoMax, it can customize the sampling order by configuring tree-traversal parameters. InfoMax traverses the sub-branches in a round-robin fashion to select the representative samples. However, this does not always serve the application needs. If the application needs to have more samples for the urgent theater of war, more samples from the theater would be required. Espresso can meet the various kinds of sampling needs to achieve situational awareness. In this work, we exploit social media data rather than military data in the evaluations because they become in large volume and are easy to obtain.

5.6 EVALUATION

This section first evaluates latency of automatic naming mainly caused by the hierarchical clustering algorithm. It then presents functional performance analysis of *Espresso*. The following subsections describe each experiment in detail.

5.6.1 Latency of automatic naming

The main purpose of this experiment is to understand the overhead of *Espresso*. To this end, we have measured the running time of automated naming for a set of tweets with

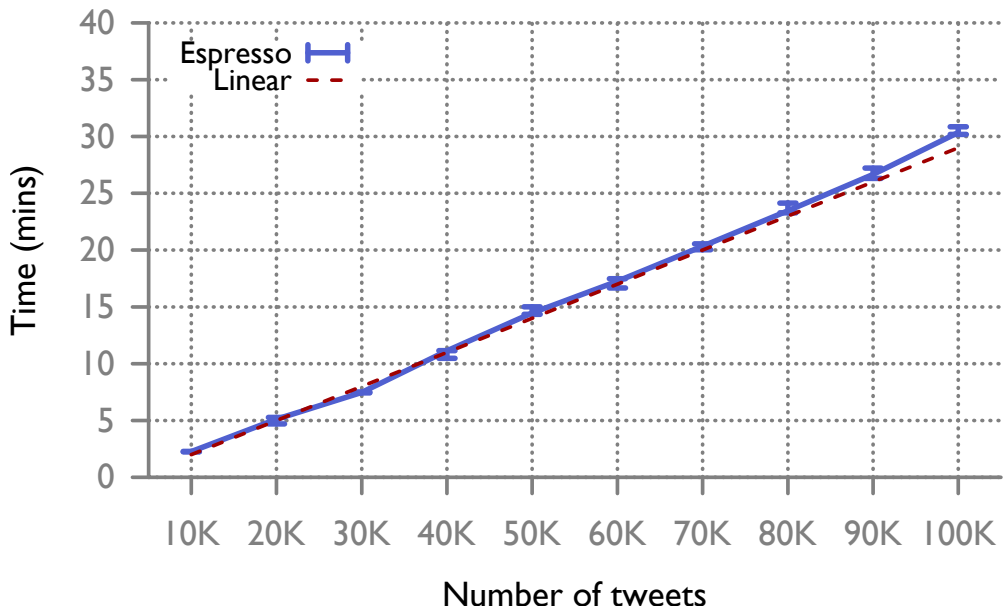


Figure 5.4: The running time to construct a hierarchical name space with increasing the number of tweets

increasing the number in order to evaluate performance of *Espresso*. Note that we have prepared 10 different data sets with the same size to avoid the data-dependent result. The result is based on average and standard error for all measurements. The measured latency includes the cost of graph construction and hierarchical naming for the given number of tweets. As we discussed in the design section, the main cost of *Espresso* is latency induced by hierarchical clustering. We have shown that the iterative 2-means clustering method would be approximately $O(n \log n)$ in the average case and we will support this argument with the following experiment result.

Figure 5.4 shows that latency of *Espresso* increases slightly faster than the linear trend-line. We believe that this successful result is due to several optimization efforts such as exploiting the triangle inequality to avoid unnecessary distance calculation as described in [50]. In practice, we could not find overhead problems in the Twitter news application when setting 1-hour time window because the number of tweets crawled for a certain topic (keywords) during the specified period mostly does not exceed 10,000.

The latency can be further optimized by the distributed manner. Since divided groups are independent to each other, they can be executed simultaneously. Therefore, with multiple workers, it would be able to further reduce the currently measured latency. The assumption of multiple workers is reasonable because *Espresso* is run at the producer-side which is often considered to have rich computing resources. Further optimization technique can be studied in future.

5.6.2 Simulation for functional performance

We have designed a simulator that can generate various data sets, where objects are separated in some customizable manner. Specifically, the simulator can randomly distribute object metadata in a two-dimensional Euclidean plane, then compute Euclidean distance as a distance metric. Each point gets assigned a random weight based on the normal (Gaussian) distribution with specified mean (μ) and standard deviation (σ).

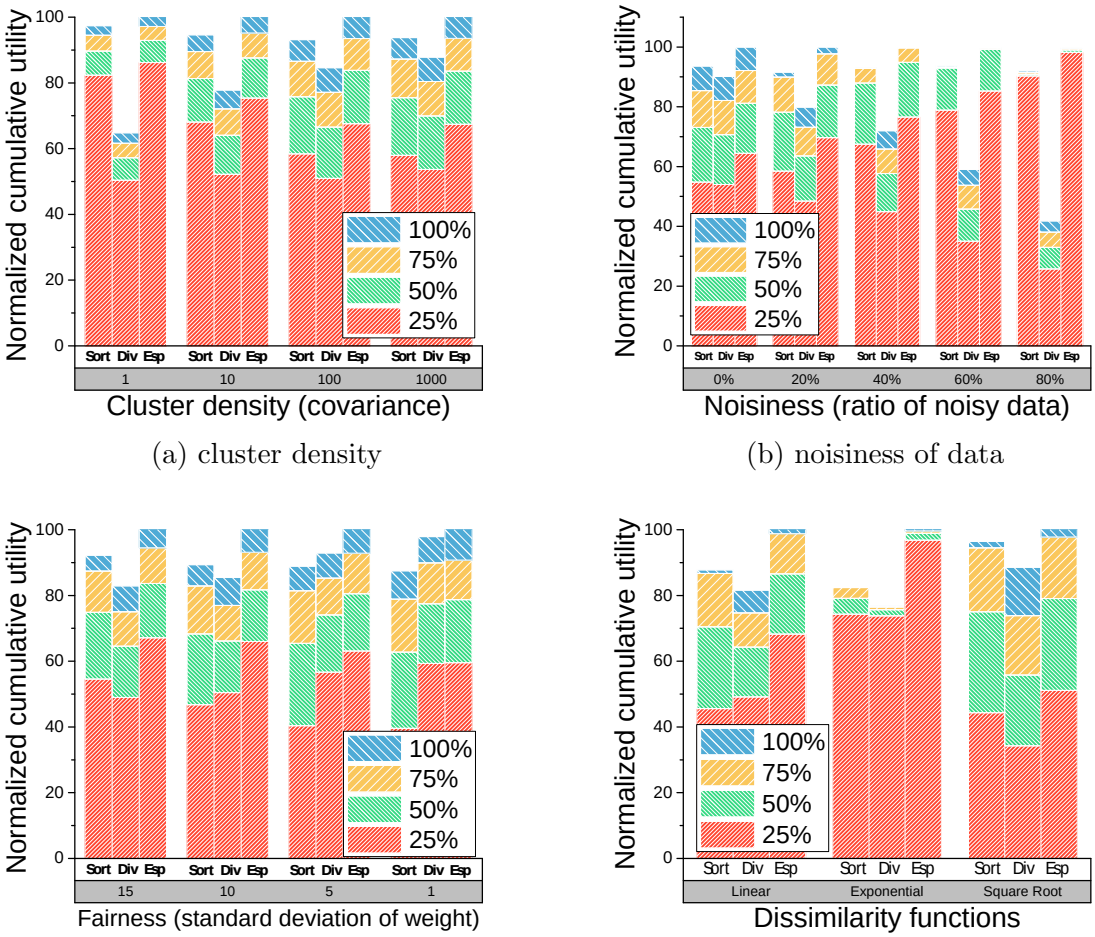
We assume that the utility of receiving an object O_i is:

$$U_i = W_i \times D_i \ (0 \leq D_i \leq 1)$$

where U_i, W_i, and D_i denote ith object's marginal utility, weight, and dissimilarity. D_i is a function of distance, where longer distance means less redundancy. The dissimilarity function is non-decreasing function, and there can be many ways to define it because notion of redundancy can vary depending on applications. The simulator can choose three types of dissimilarity functions, *linear*, *exponential*, and *square root*. The *linear* dissimilarity function is defined as follows: $D_i = \min(\frac{d_i}{t},\ 1)$ where d_i denotes the minimum distance of the ith object from any of the previously transmitted items. For example, if (0, 0) and (0, 5) are previously sent, then the minimum distance of (0, 1) from either of them is 1. This is because redundancy would be decided by the most similar data (*i.e.*, the closest data). The parameter, t, denotes the threshold that distinguishes two objects as non-overlapped. That is, if the distance between two objects is longer than t, they are considered to be different.

The simulation results are summarized in Figure 5.5. The graphs show cumulative information utility, ΣU_i, at 25%, 50%, 75%, and 100% transmission of a data set. As can be seen in the figures, marginal utility is diminished as getting more and more data objects. For obvious comparison among different algorithms, ΣU_i is normalized to 100, the maximum score of each experiment. Note that the utility value is computed from the utility function defined above. Three algorithms are compared, *sort*, *diversity*, and *Espresso*. *Sort* is the sorted order by weight, W_i, from the largest to the smallest. *Diversity* is the order of maximizing dissimilarity, D_i. *Espresso* considers harmony between the two and is configured to maximize utility for received objects. As described in 5.3, Espresso can adjust branching (or merging) function according to application-specific utility function. In this case, it first divides aggregates of nodes by visit counts and selects the highest value. In this manner, *Espresso* achieves the highest marginal utility than two baselines by pursuing high U_i, not simply W_i or D_i.

The main observation that can be found from Figure 5.5 is customizability of *Espresso*. Other monotonic algorithms have certain pros and cons in accordance with data charac-



(a) cluster density

(b) noisiness of data

(c) weight distribution fairness

(d) dissimilarity functions

Figure 5.5: Normalized information utility in accordance with different cluster density, proportion of noisy data, weight distribution fairness, and dissimilarity functions

teristics. For example, *diversity* is favorable for unfairly weighted clusters whereas *sort* is not. *sort*, on the other hand, would be preferable for highly noisy data over *diversity*. *Espresso* always achieves the best utility among all even when different dissimilarity functions are applied. This is because *Espresso* allows applications to customize the branching (or merging) order to their utility functions. This fine-tuning customization only requires a few lines of code that define selection orders for given aggregates and counts. The properly defined function shall allow *Espresso* to inherently achieve the best marginal utility than other baselines.

5.7 CONCLUSIONS

In this work, we proposed *Espresso*, a generic data naming service for ICNs. The primary benefit of this service is to create an automated namespace for an unstructured data set in a manner that facilitates representative sampling. It allows customizing retrieval policies, balancing notions of diversity and importance, to offer representative summaries to meet application needs. It further supports data zoom-in and zoom-out functionality, based on the name hierarchy. The authors are currently working on developing applications that exploit advantages of *Espresso*.

CHAPTER 6: SUMMARIZATION FOR STREAM CONTENTS

In the age data overload and scenarios that require fast distributed situation understanding, we envision that content summarization services will become a critical capability of the underlying networked systems. Previous work, called InfoMax, proposed such a service in the transport layer to minimize semantic redundancy of transmitted content and maximize information coverage. We extend this work in three dimensions. First, we adapt summarization to the needs of *streaming* content and develop a corresponding publish-subscribe protocol with on-the-fly extractive summarization (Pub/Sub-Sum) of continuous content streams (as opposed to extractive summarization of fixed data sets). Next, we support many-to-many communication between publishers and subscribers, as opposed to InfoMax, which is designed to disseminate data from one producer to multiple consumers. Lastly, we introduce a new type of congestion handling mechanism that adaptively controls the level of summarization by considering available network bandwidth. We conducted experiments for functionality and performance on Mininet (a network emulator) and on a real device testbed. Evaluation results indicate that the new protocol summarizes data appropriately to available network resources, offering an improved compromise between received data quality and resource consumption.

6.1 INTRODUCTION

Our work is motivated by the need to enable better team situation understanding in dynamic scenarios involving large data streams originating from multiple sources that contain information critical to the team. An example might be a disaster response team that aims to keep up with an evolving threat, such as a flood, via multiple camera feeds from the affected neighborhood(s). Arbitrary contention among different data streams may result in unpredictable consequences such as disproportionate data loss in streams transmitted over longer distances or in streams originating across tighter bottlenecks. In such scenarios, the underlying network services must be able to summarize content in a manner that provides a more balanced situation understanding.

There exist broadly two techniques for data summarization: namely, *abstractive* [51] and *extractive* [52]. The former technique aims to "understand" the data and report an abstraction that summarizes the key parts. An example would be to paraphrase the gist of a longer text article. The latter technique extracts representative data samples, such as video clips that summarize the most important moments of a longer event (such as a sports game). We

adopt extractive summarization in the current chapter due to its relative simplicity (being a data sampling/selection problem) and independence from the details of the data format itself.

Our work extends InfoMax, proposed by a subset of the authors [43, 53] to summarize static data sets from a single producer. Infomax operates on top of information-centric networks (ICNs), where data has names that can be resolved for query routing purposes [30]. It adopts a hierarchical name-space. Accordingly, it logically organizes (i.e., names) data in a hierarchical manner such that higher content similarity is translated into a longer shared name prefix. The run-time protocol then prioritizes data so that the next data item to transmit is semantically less redundant with the previously transmitted data based on this simple naming convention. More specifically, it transmits the data item with the smallest shared name prefix (with previously transmitted items) next. The protocol is implemented on a named-data network (NDN) stack. NDN is a representative of ICNs [31].

Our work overcomes two key InfoMax limitations. The first limitation of InfoMax is that it is strictly a one-to-many summarization scheme. Only one data source is assumed. Redundancy among multiple sources (e.g., cameras monitoring the same building from overlapping viewpoints) is not addressed. The second limitation is that InfoMax targets static (or slowly evolving) content, such as images of key tourist landmarks or social media feeds that are aggregated into static bins (say, every 6 hours) so they can be statically summarized. This limitation is problematic especially for streaming data such as video. The higher tempo involved in continuous video summarization precludes solutions that work on an individual object (e.g., frame) level. Instead, more efficient solutions must approach summarization at a higher level, such as the level of sensor selection (e.g., selection of cameras that have the best and most complementary vantage points at a given time).

Pub/Sub-Sum handles both of the above limitations. First, by thinking of the content name space as a space for organizing named data *streams* (not individual objects), we are able to choose a subset of streams to share in a way that maximizes a notion of situation coverage. Second, we extend the approach to handle the multi-producer case. To this end, locally registered stream prefixes are propagated across the network to share prefix information among the nodes. The prefix propagation can be done by the routing protocol or a separate application. In the context of NDN, named-data link state routing protocol (NLSR) [54] or automatic prefix propagation [55] can perform the described job. After getting the prefix information from the network, the protocol constructs a current prefix tree that then constitutes the basis for summarization.

Finally, in moving from static data sets to streams, a new problem is introduced; namely, one of congestion handling. Since the summarization service persists over the duration of

the stream, one must adapt the subset of streams that constitute the summary in a manner that is responsive to the current level of network resource availability. Traditional congestion handling mechanisms such as TCP control the sending rate of individual sources without impacting what content to send. We suggest a complementary congestion control algorithm on top of TCP that adapts the level of summarization, which translates into selection of one or more sources to be part of the current summary. Other Pub/Sub-Sum nodes back-off. The algorithm assumes that Pub/Sub-Sum nodes implicitly agree to be cooperative as might be the case with first responders trying to maximize their overall situation understanding.

Our protocol is implemented on top of a lite ICN framework that sits in the application layer thus making the work easily portable. In our application-layer implementation, nodes send packets over TCP connections. As in other ICNs, our protocol has two types of packets: interest and data. When a consumer issues an interest packet, the network forwards it to the producer that hosts the matching data. Once the producer generates data for the interest, the system transmits the data back to the consumer. Pub/Sub-Sum operates on top of this producer and consumer model. When the subscriber wants to start a subscription, it first requests a name catalogue to get name information for the set of available content. The subscriber can consume actual content by requesting it from the catalogue. Similarly, the publisher produces the name catalogue to advertise its content. Once an interest comes in a named object, it puts the actual named content to the network.

We first evaluate the protocol in the Mininet simulator by comparing it to a simple baseline with no summarization cabaility. The evaluation results indicate that Pub/Sub-Sum retrieves more balanced content than the baseline, thus increasing situation understanding. We also evaluate the protocol on a Raspberry PI testbed. Testbed experiments confirm that Pub/Sub-Sum functionally works on real devices and the results match with the Mininet simulations.

6.2 DESIGN

This section describes the protocol design. There are three design considerations. First, we discuss how to handle streaming content. Second, we present our algorithm for handling multiple sources. Lastly, we describe how to manage congestion in the pub-sub protocol, jointly with the underlying system's congestion handling mechanisms.

6.2.1 Handling Streaming Content

Prior work on network services for content summarization assume the existence of a relatively static set of objects that need to be shared. For example, InfoMax [43] prioritizes all data and generates a single list of available named objects that can be broken hierarchically by topic and subtopic. Let us call it the *name catalogue*. Consumers request a piece of the name catalogue on a given topic/subtopic, called *page*, containing a fixed number of names. Once a consumer receives all content of the retrieved page, it may request the next page if more detail is needed. In this manner, consumers can control the amount of content retrieved until their summarization needs are met.

The problem with this method is that the name catalogue is difficult to update once published. This complicates support for real-time streaming applications, where new objects are constantly generated and where summarization has to be done on current (as opposed to archived) content.

To remedy this problem, we focus on individual *streams* as the units comprising the summary. The name space breaks down the set of available streams in a hierarchical manner such that more redundant streams (e.g., feeds from overlapping cameras) share a longer name prefix. Given a query on a specific topic (expressed by a prefix in the same space), the tree of streams under that prefix is summarized by considering only a subset of its branches in the summary; namely, the least redundant branches (i.e., branches whose names share the least prefix). As we show later, the size of the subset is controlled depending on network conditions. Hence, in effect, subscribers subscribe to streams from a different number of branches in the query range that jointly offer the best coverage of that range (for the selected number of streams).

In addition, each publisher periodically creates a sub-list, called *volume*. An example of a resulting catalogue can be found in Figure 6.1. A name catalogue consists of several volumes containing different content streams. As before, each volume has pages of contents names, where each page has a fixed number of names. The update period decides the number of volumes and pages. Thus, it has a trade-off between update frequency and overhead. The shorter the period, the quicker the update, but the higher communication overhead.

Let's assume that the publisher produces content every second for n seconds. Then the total number of volumes would be $\frac{n}{p}$, where p is the update period. If p becomes smaller, the publisher generates more volumes that contain fewer pages. Otherwise, there will be fewer volumes having more pages. For instance, the publisher may want to have smaller p for the faster update in the example of live video streaming. On the other hand, if a pre-recorded video needs to be streamed out, p can be large since the publisher knows the whole size of

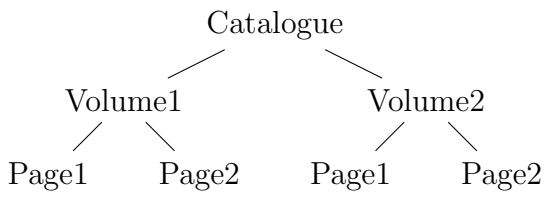


Figure 6.1: An example of name catalogue tree with 2 volumes having 2 pages for each

the video ahead of time.

6.2.2 Supporting Multiple Sources

Our algorithm supports multiple sources by allowing them to exchange name prefixes of data they carry. If an NDN-like routing protocol is running, it may exchange locally registered prefix information. Specifically, in the context of NDN, the named-data link state routing protocol (NLSR) should be able to perform this function [54]. NDN also provides an automatic prefix propagation method, as described in [55]. Otherwise, an application-layer prefix broadcast can do the job.

Once the prefix information is shared, a subscriber can construct prefix (or topic) tree as shown in Figure 6.2. The example shows four publishers, whose names are /Street-A/Block-1, /Street-A/Block-2, /Street-B/Block-3, and /Street-B/Block-4, respectively. Each of these publishers is a street camera that monitors traffic congestion status and periodically generates available content names so that subscribers are aware of the existence of newly created content. If the network resources are sufficient, the subscriber can receive streams from all of the cameras. Otherwise, the subscriber needs to select a subset of the publishers to constitute its data summary. An effective strategy would be to minimize the redundancy of content. In this example, two cameras on the same street are more redundant than others. Thus, one camera of each street can be removed from the summary (unsubscribed) to save some bandwidth.

One extra benefit of having the prefix tree is that subscribers can zoom in or out the topic if necessary. The higher level of nodes in the tree denotes a more comprehensive view of the topic. In contrast, the lower level of nodes represents a more specific topic. For example, if the subscriber specifies '/' as interest, then it gets videos from all cameras, if the network is capable. If the subscriber wants to zoom in, say '/Street-A/Block-1', then it gets videos from only one camera. If the subscriber wants to zoom out again, say '/Street-A,' then it gets streams from two cameras in the street, network conditions permitting. In short, the subscriber drives what range of content is to be summarized.

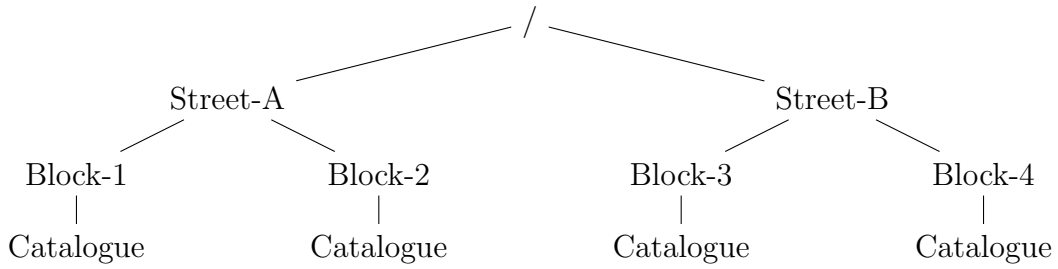


Figure 6.2: An example of the prefix tree

6.2.3 Summarization-Based Congestion Control

The idea of summarizing content across several sources motivates a new direction in congestion control, namely, summarization-based congestion control. Traditionally, congestion control mechanisms adaptively control the sending (or retrieving) rate of individual sources according to network conditions. There have been several efforts studying congestion control mechanisms in ICNs. For a representative set of examples, recent work [56] provides a comprehensive survey of the state-of-the-art in NDN congestion handling. Current approaches are categorized into three groups: Receiver-based, Hop-by-Hop interest shaping, and hybrid approaches. Receiver-based algorithms allow the data consumer (terminal) to shape the interest issue rate, whereas hop-by-hop algorithms do so in the network layer. Hybrid approaches combine the two. Pub/Sub-Sum complements prior solutions using an application layer technique.

In this work, we combine our application-level congestion control mechanism with one of the receiver-driven approaches, called ICP [57]. This algorithm works similarly to the current TCP congestion control algorithm. It maintains a congestion window that defines the maximum number of interest packets in transit. Initially, it doubles the window size until congestion is detected, which is the slow-start phase. When the congestion is first detected, it halves the window size and switches to the AIMD (additive-increase-multiplicative-decrease) phase. That is, if congestion is not detected, the window size increases gradually. If congestion is detected, the window size decreases quickly. Whenever a consumer issues a new interest packet, it triggers a timer to measure the packet round-trip time (RTT) between the interest issue and the data arrival. If the RTT delay is longer than the expected timeout delay, it regards that congestion occurs in the network.

If the network gets congested, the consumer signals to the subscriber so that it can adjust its data volume to retrieve; otherwise, the subscriber assumes that the network is sufficient to manage the current volume of data requests. Based on this algorithm, we develop an enhanced congestion handling solution as follows. A subscriber interested in data under some tree prefix first starts by retrieving data from only one of the branches in the tree (one

branch subscription). If the network has no congestion for a certain period, the subscriber increments the number of branches from which data is requested by picking a branch that is least redundant with the current subscriptions (i.e., that has the shortest shared prefix with them). We say that a new branch subscription is thus added. The retrieved summary is collected from all subscribed branches of the tree defined by the original prefix. The algorithm for incrementing branch subscriptions is merely a tree traversal that always selects the least-visited branch first with a left-most tiebreaker rule. In Figure 6.2, if /Street-A/Block-1 is subscribed, then the algorithm picks /Street-B/Block-3 next. On the other hand, if network congestion is detected, it decrements subscriptions.

Discussion about which branches need to be cancelled is an interesting new challenge. One solution is to unsubscribe to the most redundant branch. However, this method may be problematic if the least redundant publisher is the primary cause of the delay. Therefore, another strategy could be to unsubscribe to the suspected cause of congestion. Note that, the subscriber knows which data gets delayed by how much from the underlying consumer. This offers opportunities for manipulating branch subscriptions in a way that balanced content redundancy within the current summary against retrieval delay.

How to set algorithm parameters is another discussion topic since algorithm performance is dependent on these settings. For example, if the algorithm is too sensitive to bandwidth changes, the number of selected sources (branches in the summary) would be fluctuating. In this work, we do not discuss what parameters are optimal and how to configure those values automatically. Instead, we leave the auto-configuration of the congestion control algorithm for future work.

6.3 IMPLEMENTATION

This section describes implementation details for the design mentioned above. We provide a library package, called *Athena*, that can effectively schedule data retrievals according to the specified application needs [58].

Athena provides a thin layer placed between applications and communication systems. Among other things, it offers APIs for applications to define their data requirements.

6.3.1 Built-in ICN framework

We built a light-weight communication protocol that offers ICN abstractions in the application layer. This implementation is inspired by the NDN software stacks provided in [59]. It operates as an overlay network, where every packet is transmitted over a TCP connection.

An network layer equivalent (implemented in the application layer) focuses on the basic primitives of ICNs, where every data has a name, and the network forwards packets based on the longest prefix matching rule. Also, this implementation includes a simple routing functionality that computes a route from the given source and destination. To this end, it periodically exchanges local connectivity information with other nodes such as link state messages. The network layer emulation offers mainly three APIs to the above transport layer: 1) issue an interest packet, 2) register a prefix, and 3) put a data packet to the network.

The transport layer emulation uses these APIs to transmit an application data unit. The producer first segments the whole message into smaller pieces and name them sequentially (e.g., /prefix/suffix/1, /prefix/suffix/2). When the first interest packet issued by the consumer reaches the producer, it puts the first data packet to the network along with meta information including the total number of data packets. When the consumer receives the first packet, it requests the next data packets. Similar idea can be found in [49, 60]. Figure 6.3 reveals the network and transport layer that exchanges interest and data packets among nodes.

As mentioned earlier, a congestion window defines the maximum number of interest packets on the fly. This window size is to control congestion so that the consumer makes parallel requests as many as the network can manage. Several works pointed out that receiver-driven timeout-based algorithm would not fit into multi-source and connection-less featured ICNs [61, 56]. However, our implementation uses the naive algorithm since the main focus of this work is to develop an algorithm that reduces the injected data volume into the network via adaptive summarization.

6.3.2 Pub/Sub-Sum Protocol

Figure 6.3 describes overall function blocks of the Pub/Sub-Sum protocol on top of the built-in ICN protocols described in the previous subsection.

The Pub/Sub-Sum protocol is an Athena layer component placed between the application and transport layers. It offers APIs to publish and subscribe contents under a specific prefix (topic). When an application asks to subscribe to the prefix, the subscriber collects the matching prefixes from the network. Then, it prioritizes the collected prefixes in the least-redundant order based on names and starts subscribing to the very first one in the list.

It first asks the meta information about the served contents set including the volume number and its page number as described in Section 6.2.1. Depending on the application type, it can select either the first volume or the last one depending on application needs. If

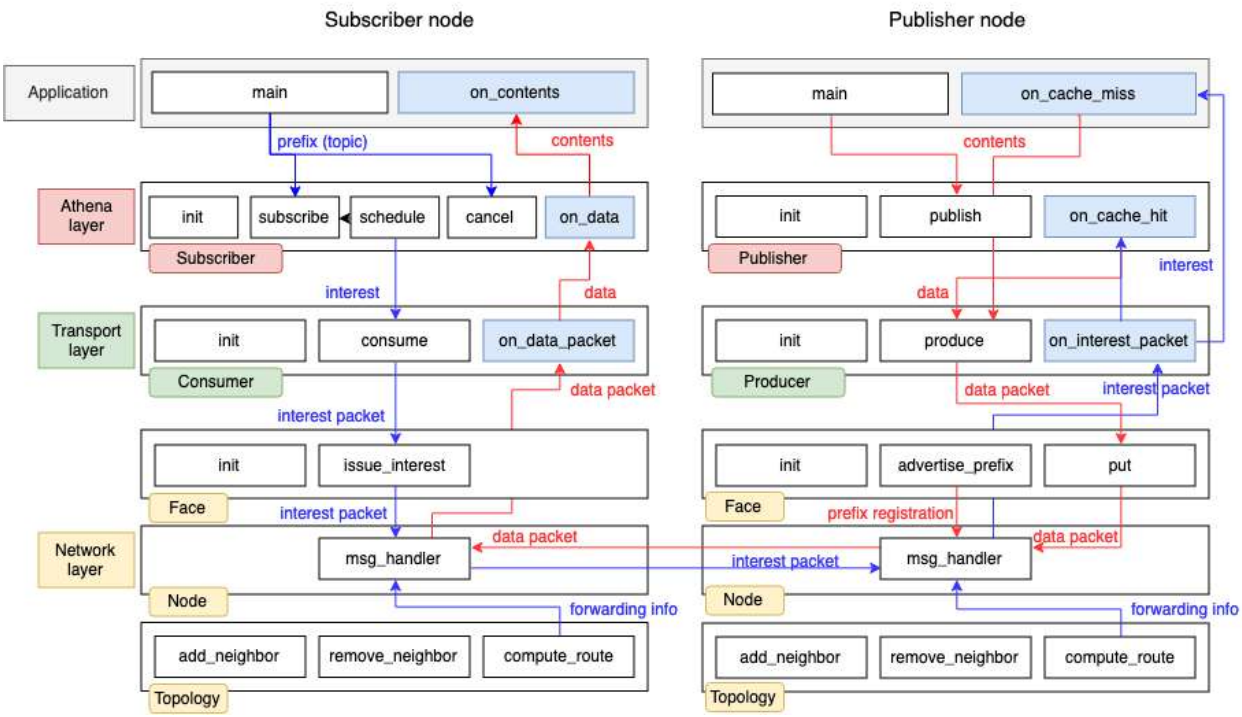


Figure 6.3: Pub/Sub-Sum protocol and underlying ICN function blocks

an application does not care about historical data, it will choose the latter. Once the volume number is decided, it retrieves the first page of the volume to get a prioritized contents list. Finally, the subscriber can request actual contents based on names in the given page. This process is iterative until the application cancels the subscription. The visualization for this process can be found in Figure 6.4.

Given the prefix of a content tree, the protocol starts with activating one branch subscription, then gradually increases the number of branch subscriptions if no congestion is detected for a certain period. If the subscriber gets congestion signals from the underlying consumer, it starts a timer to measure the round-trip time (RTT) between interest packet departure and data packet arrival. If the RTT is longer than the expected, it re-issues an interest packet up to some threshold. This retransmission algorithm is the same as TCP as described in RFC-6298 [62]. If any time-out occurs, the consumer signals to the subscriber so that it can know that congestion is suspected. Upon receiving this signal, the subscriber decreases the number of active branch subscriptions adaptively until congestion is gone.

6.4 DOMAIN OF APPLICABILITY

InfoMax mainly supports a case where multiple subordinate units retrieve sample information from the command center. Pub/Sub-Sum extends InfoMax and works for many-to-

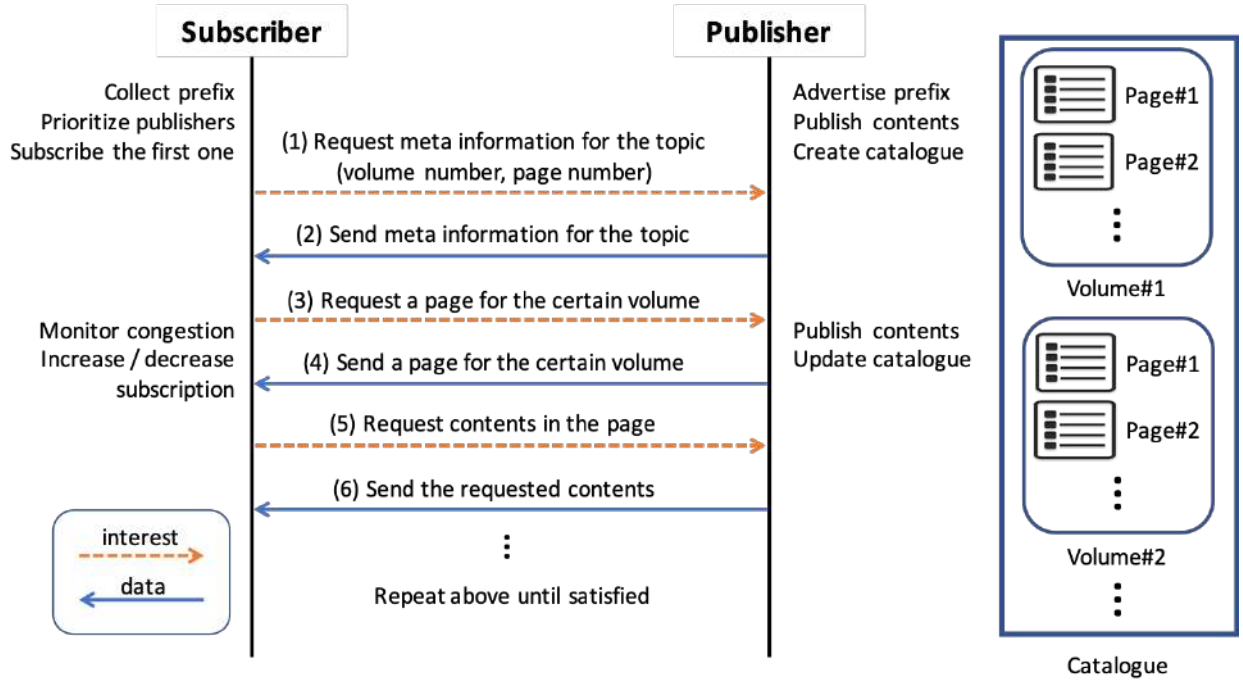


Figure 6.4: Control flow of Pub/Sub-Sum protocol

many communication. Therefore, it supports the reverse communication pattern that allows the command center to collect information from subordinate units. This difference makes Pub/Sub-Sum suitable not only for the high echelons of the tactical networks, but also for the low echelons. For example, the platoon leader may need to get video streams from each squad. Pub/Sub-Sum can support the platoon leader by subscribing to each squad video camera. Of course, each squad can subscribe to the platoon leader as well to update the current situation.

6.5 EVALUATION

This section evaluates the system in two different environments. The first environment is Mininet, a widely used network simulator that enables one to flexibly configure the network environment including link speeds and network size [63]. The second one is a real device testbed using Raspberry Pis. The purpose of the testbed is to show that the protocol can run on actual devices and that empirical results match simulations. Detailed setup and experiment results are described in the following subsections.

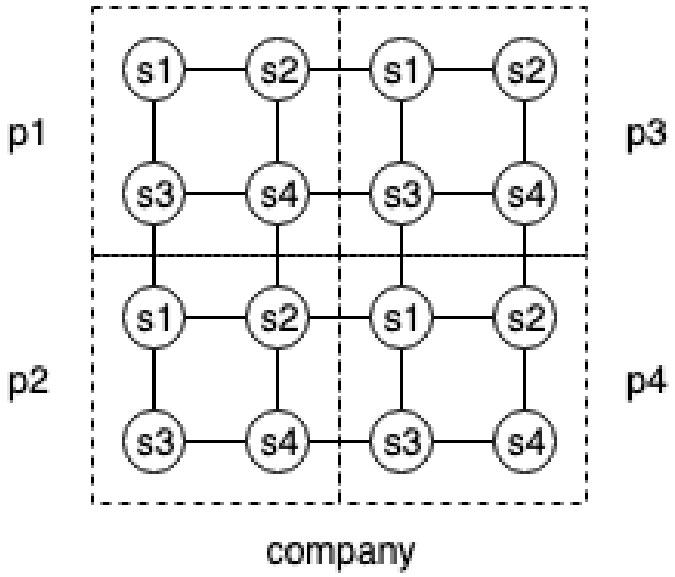


Figure 6.5: A grid topology with 16 nodes in Mininet simulation

6.5.1 Mininet

In this experiment, we have composed a hypothetical scenario for better understanding of the use of the Pub/Sub-Sum protocol. The scenario is about company reconnaissance, where a company is searching a particular area, and the company commander wants to have periodic reports from sub-units. For convenient naming, let's say every 4th unit is a headquarter (e.g., 4th platoon is the company HQ). Every squad has a mobile camera and streams images (1MB each) every second for 30 seconds to the company commander. To simulate this, we create the grid topology with 16 nodes as shown in Figure 6.5. A name for each node is hierarchically composed such as /company/platoon-1/squad-1 for the top-left node. Every squad publishes contents with the sequence number under its prefix. Then, the company commander, located at the bottom-right corner, collects reports from the sub-units by simply specifying '/company' as a topic to subscribe.

Two different algorithms are used in the experiments. The baseline subscribes to 16 publishers all the time while Pub/Sub-Sum adaptively controls the number of subscriptions by considering the network conditions. Figure 6.6 describes the number of active subscriptions over time for both algorithms. Two different network speeds are tested: 100 and 10 Mbps. Observe that subscriptions reflect the volume of *requested* (as opposed to *delivered*) content. It is a measure of input load imposed on the network, where a higher load is not necessarily better.

The number of subscriptions (i.e., requested flows) remains the same for the baseline algo-

rithm. In contrast, Pub/Sub-Sum dynamically changes the number of subscriptions according to network conditions. When the network speed is 100Mbps, the number of subscriptions monotonically increases until 16, since the network can handle all of the subscriptions. However, subscriptions fluctuate around 8 in the 10Mbps case, because 10Mbps is not sufficient to get streams from all sources; thus the protocol tries to limit the number of subscriptions to accommodate the bandwidth constraint.

Perhaps counter-intuitively, despite the lower number of subscriptions, Pub/Sub-Sum *delivers* more content than the baseline (in the constrained network case) as shown in Figure 6.7. The figure shows that Pub/Sub-Sum delivers 170 images without any drops while the baseline transmits only 128 images. The remaining images are either dropped due to delivery failures or never make it out of their producers' queues. A delivery failure means that a data packet in the message timed-out multiple times (more than the pre-defined retransmission threshold). The high number of message drops in the baseline algorithm is due to network overload that ends up causing more congestion. In the case of the faster network, both algorithms eventually transmit all content.

Next, we consider non-redundant content items. In this experiment, content generated by the same platoon at the same time slot is considered redundant. In other words, since the squads in the same platoon are geographically close to each other, their reports can be considered as redundant. Non-redundant items are those generated at different times or by different platoons. In this case, the difference between Pub/Sub-Sum and the baseline becomes more dramatic. Figure 6.8 shows that the number of collected non-redundant content items (by the above definition) from each platoon over time. The result denotes that Pub/Sub-Sum collects non-redundant reports faster than the baseline in both network environments. Notably, in the constrained network, Pub/Sub-Sum achieves more than 75% information coverage (i.e., collects more than 75% of all non-redundant items) at the end while the baseline attains lower than 50%. This result expresses the main strength of Pub/Sub-Sum; quickly summarizing content by requesting less redundant data objects first.

We consider non-redundant content items. In this experiment, content generated by the same platoon at the same time slot is considered redundant. In other words, since the squads in the same platoon are geographically close to each other, their reports can be considered as redundant. Non-redundant items are those generated at different times or by different platoons. In this case, the difference between Pub/Sub-Sum and the baseline becomes more dramatic. Figure 6.8 shows that the number of collected non-redundant content items (by the above definition) from each platoon over time. The result denotes that Pub/Sub-Sum collects non-redundant reports faster than the baseline in both network environments. Notably, in the constrained network, Pub/Sub-Sum achieves more than 75% information

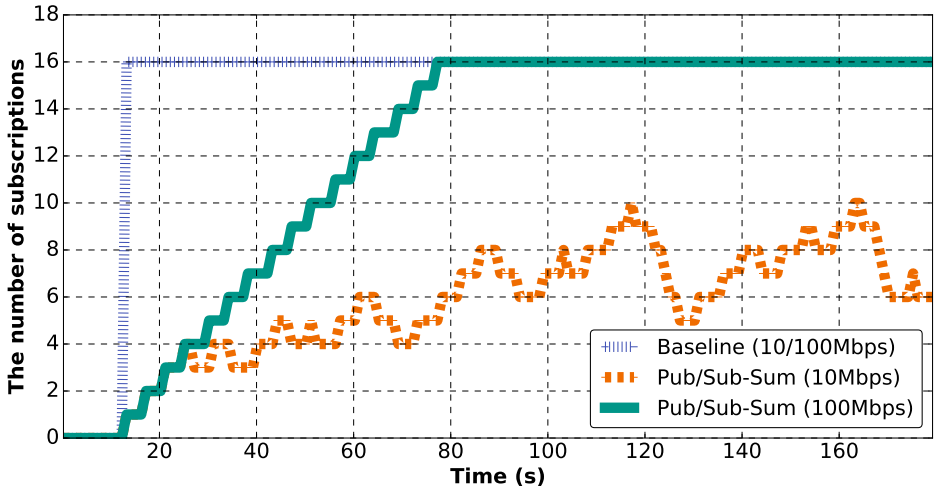


Figure 6.6: The number of active subscriptions over time for the baseline and Pub/Sub-Sum in 10/100 Mbps networks

coverage (i.e., collects more than 75% of all non-redundant items) at the end while the baseline attains lower than 50%. This result expresses the main strength of Pub/Sub-Sum; quickly summarizing content by requesting less redundant data objects first.

The next part of the evaluation explores a dynamic network environment, where the network speed starts with 100Mbps but drops after 30 seconds to 2Mbps (e.g., due to an attack such as jamming). The congestion period is 2-minute long followed by 30 seconds of normal operation. We simulate such an environment by controlling link bandwidth through Mininet APIs.

Figure 6.10 shows how the adaptive algorithm behaves for the denied network. Until the congestion detected, it linearly increases the number of subscriptions. This number drastically drops during the congestion period then stays around 3. After the congestion is gone, the algorithm quickly increases the number of subscriptions because no congestion is found. The count of successfully delivered images is displayed in Figure 6.11. In contrast, Figure 6.12 shows the counts of the non-redundant reports from platoons. As before, Pub/Sub-Sum achieves better information coverage than the baseline especially during the congestion period.

We first measure bandwidth consumption using *bmon*, a rate estimator for Linux systems, to verify the bandwidth changes. Bandwidth usage as a function of time for publisher and subscriber is presented in Figure 6.9. As can be seen, a subscriber is the most massive bandwidth consumer; thus the subscriber mainly gets choked when the link speed suddenly decreases. Different than the baseline subscriber that rapidly consumes bandwidth from the beginning, the Pub/Sub-Sum subscriber starts slowly but quickly catches up with bandwidth

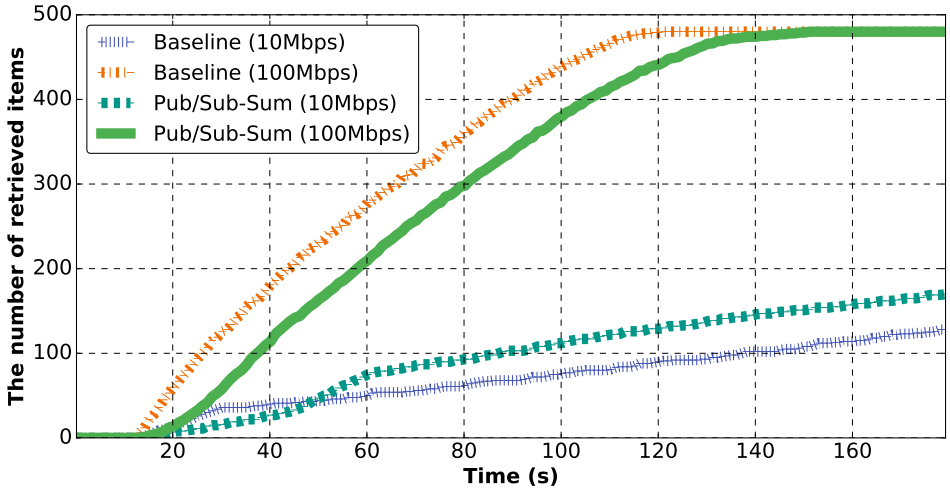


Figure 6.7: The number of retrieved items over time for the baseline and Pub/Sub-Sum in 10/100 Mbps networks

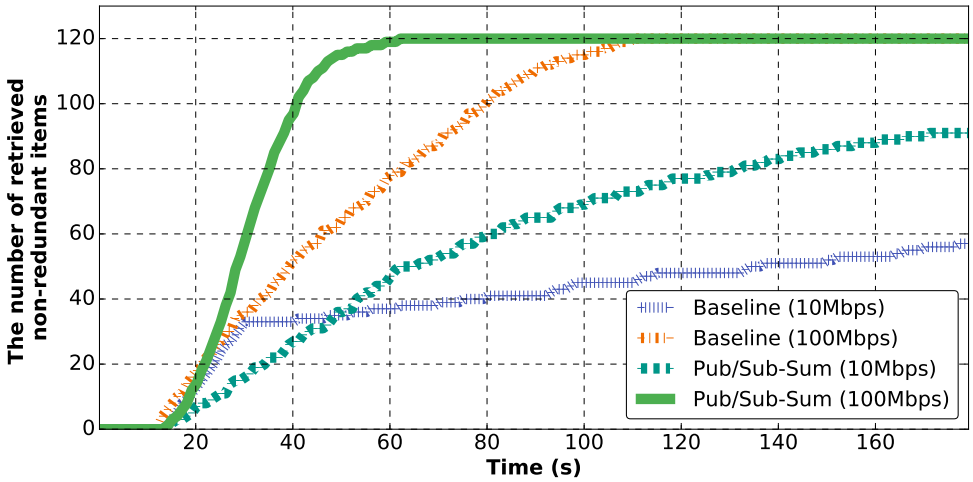


Figure 6.8: The number of retrieved non-redundant items over time for the baseline and Pub/Sub-Sum in 10/100 Mbps networks

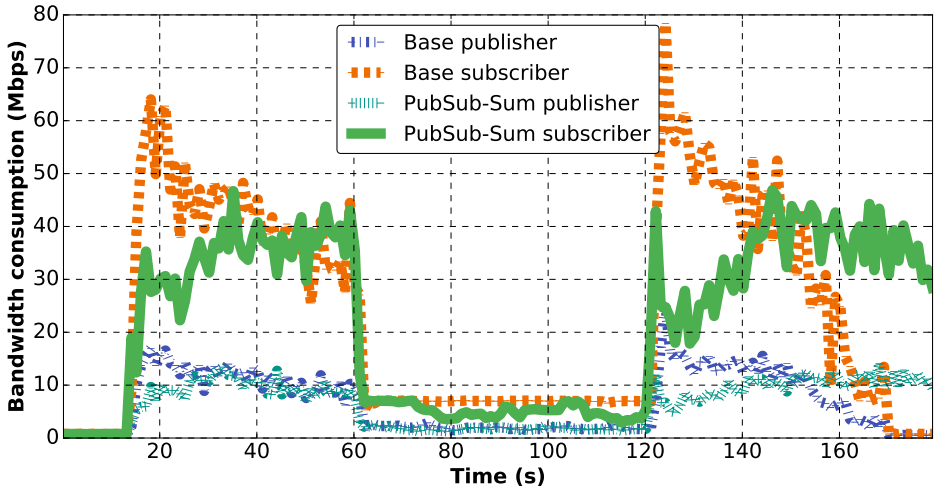


Figure 6.9: Bandwidth usage over time for the baseline and Pub/Sub-Sum in the congested network

consumption of the baseline due to the adaptive control algorithm.

6.5.2 Raspberry Pi Testbed

The Mininet simulation shows that the implementation works well as designed. In the next experiment, we want to verify that it operates on real devices as well and to see whether the simulation results match with the real testbed case. To this end, we have constructed a simple Raspberry Pi testbed as shown in Figure 6.13. We use 4 Raspberry Pis as publishers and a desktop as a subscriber. Raspberry Pis equipped with cameras publish a snapshot image every second. Two of them see one side of a scene, and the remaining two see the other side as can be inferred from the node names in the figure. The desktop, which is the subscriber, then collects the published images and display them on the screen whenever being updated. The screen-shot of such program can be seen in Figure 6.14. The left side image shows that Pub/Sub-Sum selects less redundant sources when the network is constrained. Otherwise, all streams are available as in the right picture.

Using the testbed, we test a congestion scenario similar to the previous experiment. Since the testbed currently supports 100Mbps full duplex for Raspberry PIs and 1Gbps full duplex for the desktop, we need to manually downgrade the speed to simulate congestion. One way to do so is to use *ethtool*, a Linux utility to modify parameters of network interface controllers. Using the tool, we could downgrade the speed to 10Mbps half duplex as shown in Figure 6.15. Different than the Mininet method, *ethtool* first disconnects the network to reset the rate. That is why there are two valleys between congestion and non-congestion zones in the graph.

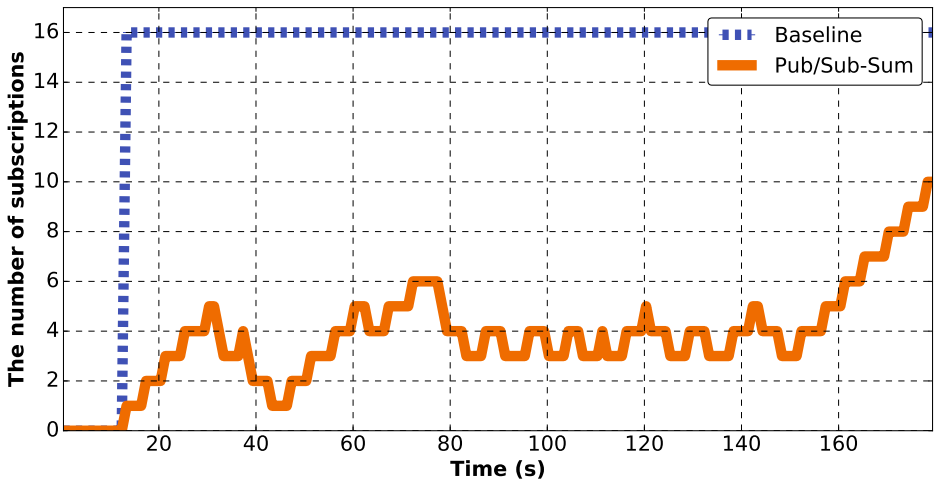


Figure 6.10: The number of active subscriptions over time for the baseline and Pub/Sub-Sum in the congested network

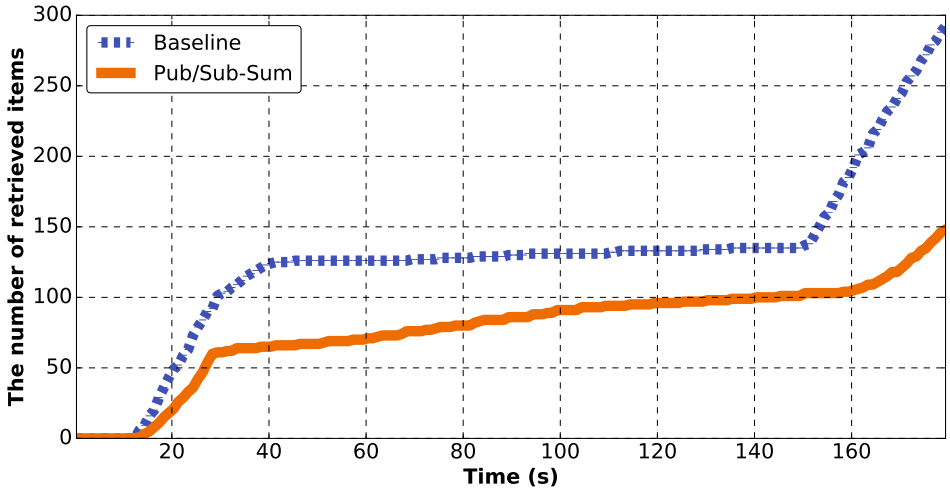


Figure 6.11: The number of retrieved items over time for the baseline and Pub/Sub-Sum in the congested network

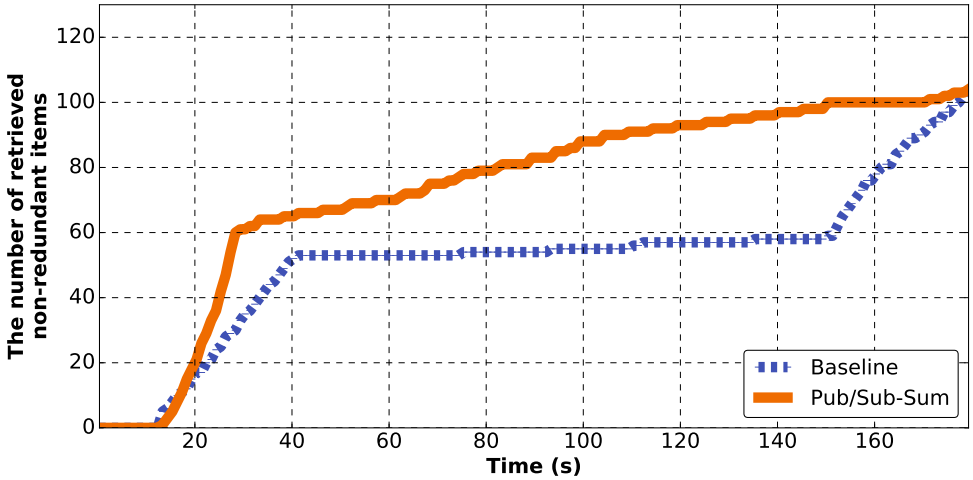


Figure 6.12: The number of retrieved non-redundant items over time for the baseline and Pub/Sub-Sum in the congested network

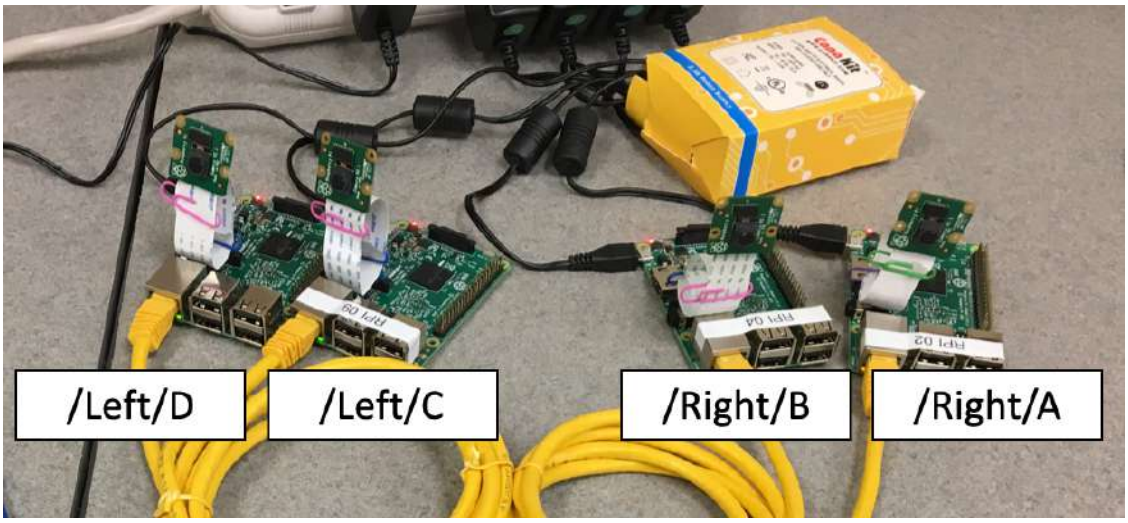


Figure 6.13: Raspberry Pi testbed

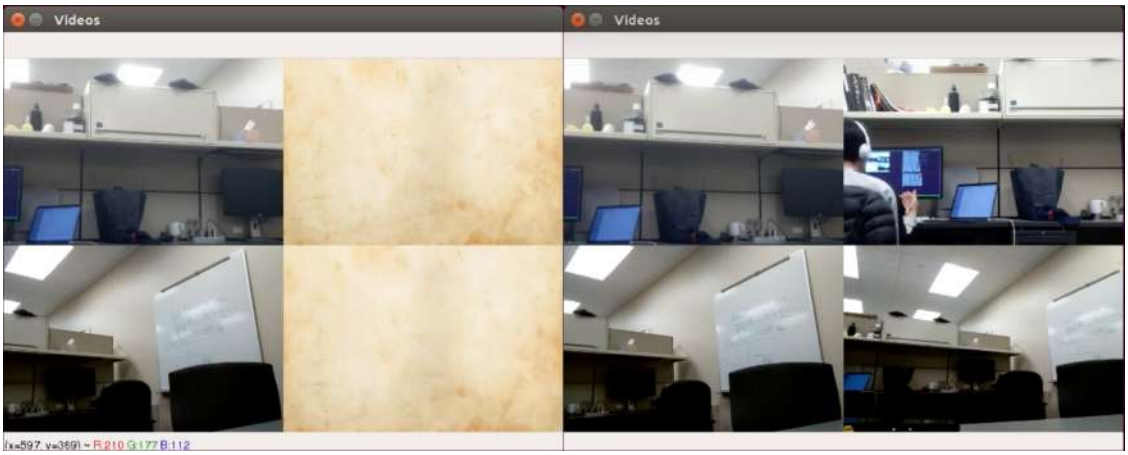


Figure 6.14: Application snapshots when the resource is constrained (left) and sufficient (right)

Using the testbed, we test a congestion scenario similar to the previous experiment. Since the testbed currently supports 100Mbps full duplex for Raspberry PIs and 1Gbps full duplex for the desktop, we need to manually downgrade the speed to simulate congestion. One way to do so is to use *ethtool*, a Linux utility to modify parameters of network interface controllers. Using the tool, we could downgrade the speed to 10Mbps half duplex. Different than the Mininet method, *ethtool* first disconnects the network to reset the rate. That is why there are two valleys between congestion and non-congestion zones in the graph.

The figure explains that the subscriber reaches the maximum network capacity when the network gets congested. However, the congested network is still able to get streams from all four cameras as in Figure 6.16. When *ethtool* changes the network speed the number of subscriptions drop, but it recovers to 4 quickly when the network is reconnected. The number of retrieved images is higher in the constant phase while the speed gets a bit slower during the congestion period.

6.6 CONCLUSIONS

This work describes Pub/Sub-Sum, a content sampling-based pub-sub pattern on ICNs. It selectively subscribes to the available publishers in the network by considering source redundancy. This work extends the previous work, InfoMax, in that Pub/Sub-Sum diversifies not only content for a particular source but also sources themselves. It also describes how to handle network congestion by adaptively controlling the number of subscriptions, jointly working with the traditional approach that shapes the interest rate. In the future, we plan to extend this work to satisfy the real-time constraints from applications so that the algorithm optimizes the network resources while meeting the given constraints.

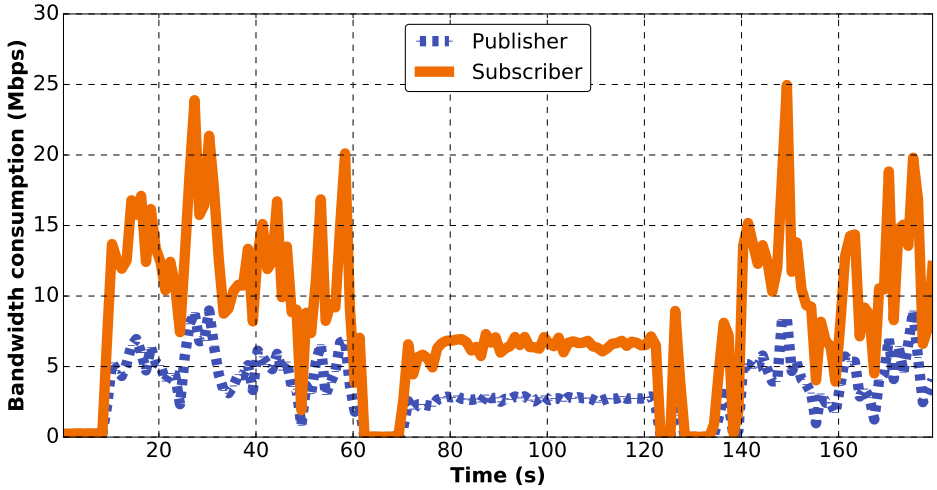


Figure 6.15: Bandwidth usage over time for the baseline and Pub/Sub-Sum in the Raspberry PIi testbed when the network is congested

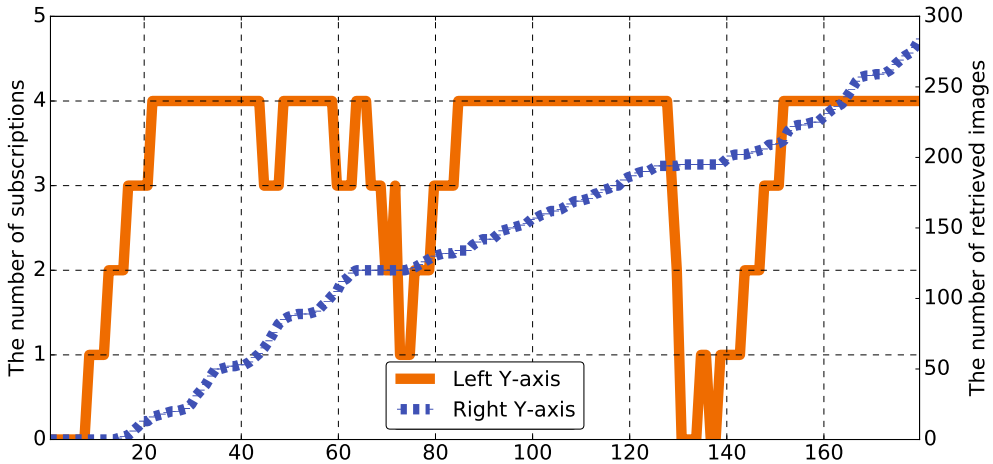


Figure 6.16: The number of active subscriptions and retrieved items over time for the baseline and Pub/Sub-Sum in the Raspberry Pi testbed when the network is congested

CHAPTER 7: APPLICATION EXAMPLES

This chapter introduces the applications implemented on the Athena layer. There are two representative applications, and both of them are in the military context. The first one is target tracking, where the target is blended into the regular traffic in the city, and the mission is to track the target with minimal data consumption. The next one is situation awareness. In the military, reporting to the commander is a critical mission, and we want to collect the report in the least semantically redundant manner possible so that the commander can understand the situation well with minimal data usage. Details about the applications can be found in the following sections.

7.1 TARGET TRACKING

The majority of the world population today live in cities. This fact suggests that future military operations are more likely to occur in city environments, as opposed to remote areas. In one scenario, a friendly city government might offer access to deployed local sensors (such as traffic and security cameras) to assist in army operations. Traffic in the city might follow typical patterns, while targets of interest might attempt to blend in. In this section, we investigate the efficacy of tracking such targets using sparsely deployed sensors in a city, such as security cameras at intersections.

Ideally, intelligent vision algorithms on the cameras would recognize targets of interest automatically. However, if the cameras belong to the local city infrastructure, they might not be equipped with target recognition software, or the mission might require that identities of targets of interest not be shared externally with gray sensors. Hence, camera footage might need to be sent instead to a remote location for inspection by a trusted AI agent, or a human operator. In either scenario, efficiency considerations dictate the footage transmitted to be minimal. For example, in an environment where communication bandwidth is limited, sending less footage without losing the target can entail significant bandwidth economy.

In the following, we assume that the sensors offer an interface that allows them to start/stop streaming data on demand. We call these operations to turn on (wake up) and turn off the sensor, respectively. The tracking system must automatically determine (i) which sensors (i.e., cameras) need to be turned on, and (ii) when (i.e., when the selected sensors need to start/stop streaming their footage), such that targets of interest appear on tape.

The work attempts to understand the natural entropy that exists in such a tracking system.

How predictable is normal target behavior? How well can one predict the next sensor that the target will cross and the expected time when it will do so? To answer these questions, we use a data set collected by Kestrel, a video analytics system from the University of Southern California [64]. Kestrel tracks paths of vehicles across campus cameras using YOLO, a real-time object detector [65]. The system then indicates the target (bounding box) location in the camera image and a timestamp for every image frame. Hence, every sensor maintains a {target ID: {timestamp: (x-pixel, y-pixel)}}, where target ID is locally assigned by each sensor. For validation, the data set is further annotated to assign global target IDs such that sightings of the same target at different cameras have the same ID.

Our prediction algorithm consists of two parts: sensor prediction and delay prediction. Once a target leaves the field of view of one camera, sensor prediction selects the cameras to activate next, to keep tracking the specific target. It is possible to train a classifier to associate the last camera ID that observed the target, as well as the direction of motion, with predicted next sensors on the target's path. Once after identifying the next sensor, the next question is when to turn on the selected sensor. The camera need not start capturing data immediately but may want to wait until the target arrives. We can learn the delay distribution between two sensors and use it to predict the wake-up delay for the next sensor so that it can minimize its uptime without missing the target.

We compare different algorithm regarding sensor uptime, precision, and recall. Precision measures are tracking efficiency. It refers to the ratio of time that a target of interest was in the field of view of a camera compared to the total uptime of the camera. Recall measures are tracking accuracy. It refers to the ratio of the time that a camera was able to observe the target to the time that the target was present within a range of the camera. For the data set at hand, compared to a scenario where all cameras are on all the time, the results show that one can reduce sensor uptime 238 times at the expense of missing 15% of the target/camera encounters.

There exists a trade-off between sensor uptime and missed targets. Two parameters are provided to allow users to control this trade-off; the desired confidence level in sensor prediction and the desired confidence level in delay prediction. When increasing these parameters, the algorithm selects more sensors and wakes them up earlier, thereby catching targets with a higher probability. For example, the evaluation result shows that the detection error rate could be only 1% with a proper configuration while sensor uptime is still 2.7 times saved.

7.1.1 Data-centric target tracking algorithm

In this section, we first describe the dataset captured by USC campus cameras. An algorithm is then proposed to minimize sensor uptime and detection errors. The algorithm consists of two parts: sensor prediction and delay prediction. The following subsections describe each algorithm.

Figure 7.1 shows the USC campus camera map. There are more cameras, but this work only uses 9 of them. We collected the footage from these cameras for a contiguous interval of 15 minutes during which 284 clear targets were observed. Every camera generates an image at its frame rate. Kestrel, the video analytics system of USC, enables users to search for specific events such as target arrival and departure [64]. It uses YOLO [65], the real-time object detector, to track vehicles across campus and represents the target position as (x-pixel, y-pixel) in the image.

The information extracted from Kestrel can, therefore, be represented by the tuples (sensor ID, target ID, timestamp, position), denoting which sensor saw which target when and where. Since targets spend a finite amount of time in the field of view of each camera, multiple consecutive sightings are reported. We summarized the data set by transforming it into (target ID, sensor ID, arrival timestamp, departure timestamp, out-direction), describing when the target arrived and left each sensor, and what the direction was when the target disappeared. The moving direction is calculated from target positions in the two consecutive images from the raw data. Note that this is ground truth data trained by the system to predict the next sensor and delay for the requested target, which will be described in the following subsections.

When the target leaves one sensor, the system should decide the next sensor to turn on to keep tracking the target. We introduce two algorithms: Frequency-based and Out-direction classifier.

The first algorithm selects the most frequently visited next sensor first. From the target data set described above, during training, one can count the number of targets a sensor sees. One can then compute the probability that a target shows up at a given other sensor next. For example, if sensor A sees 100 targets of which 80 are then seen by sensor B and 10 by C, then the probability of observing a target at B after it passes A is 0.8 and the probability of seeing a target at C after it moves A is 0.1. The algorithm selects the most likely next sensor, which is B in this case.

The above algorithm works well for one-way streets. If more than one traffic direction is possible, that information further enhances prediction. The second algorithm, therefore, exploits target out-direction information. To this end, we use a classification algorithm that

Figure 7.1: Camera placements in USC

automatically learns the next sensor based on the given out-direction and current sensor. It can thus predict the next sensor when a target leaves in a particular out-direction.

Once the sensor prediction algorithm selects the next sensor, the next question is when to wake up the sensor. The delay prediction algorithm handles this question as follows.

The first algorithm is to turn on the selected sensor immediately and wait for a certain amount of time. If the target does not appear during this time, the sensor is automatically turned off. The main benefit of this method is that it is hard to miss the target if the timeout period is long enough. On the other hand, if the target does not show up, the sensor uptime is needlessly high, which ends up wasting resources.

The second algorithm bases the experienced delay from the training dataset. Since the dataset contains the times of target arrival and departure at each sensor, delay of targets moving from sensor to sensor can be easily computed. Therefore, it is possible to construct the delay distribution for targets traveling between each pair of sensors. This distribution can be used to select a delay and uptime that capture a specific percentile of targets. For example, if 80% of targets take 3-5 seconds to move from sensor A to sensor B, then turning on sensor B after 3 seconds (from target departure from A), and turning it off after 5 seconds, will capture 80% of targets traveling from A to B.

There exists a trade-off between resource savings and detection probability. A sensor that is always on may consume more resources (bandwidth to stream data to a human or an AI agent for analysis) while never missing targets. On the other hand, a sensor that turns on only briefly may sometimes miss targets. The parameters described below control this trade-off.

The sensor prediction algorithm can compute the probability of target detection by subsequent sensors. If the system only selects one sensor, it would choose the sensor with the highest chance to see the target next. However, it is not always a good idea to pick only one sensor. Instead, users can specify a confidence level, C_s. The system iteratively selects the sensors in decreasing order of their probability (i.e., odds to be next on the target's path) until the sum of such probabilities exceeds C_s. The algorithm returns a set of sensors. If C_s increases, the set becomes larger. A larger C_s would decrease target miss rate while increasing total sensor uptime.

A control parameter for delay prediction, C_d, is to control the uptime duration for which the next sensor is turned on. If C_d is large, the wake-up time for selected sensors would be quick and the timeout period would be extensive. On the other hand, the selected sensors are only active for a short amount of time.

The delay prediction algorithm returns two values: a turn-on time, T_{on}, and an off time, T_{out}. We compare two variations of this algorithm. In the first, T_{on} is immediate and C_d is proportional to timeout. If the second algorithm is selected, T_{on} and T_{out} are computed from a delay distribution. The percentile function is used to find T_{on} and T_{out} to capture a percent of target, C_d, as follows:

$$T_{on} = percentile(L_{delay}, \frac{(1 - C_d)}{2})$$
$$T_{out} = percentile(L_{delay}, \frac{C_d + (1 - C_d)2}{})$$

where *percentile* is a function to pick the value from the given sorted list at the input percentile. For example, if C_d is 0.5, then T_{on} is at the 25th percentile and T_{out} is at the 75th percentile of the delay distribution. If C_d is 1.0, the minimum and maximum delay would be selected.

7.1.2 Evaluation

This section first studies the accuracy of various algorithms when predicting the best next sensor(s) and their uptime. We then explain the metrics used and compare different algorithms. In all experiments, data are broken into a training set and a test set. Namely, we train with all targets while excluding one (the test target), then alternate the excluded target. Finally, we compute the average over all cases.

To predict which sensor a target is going to visit next, we selected six different algorithms to train our predictors. Two of them are linear algorithms: logistic regression (LR) and

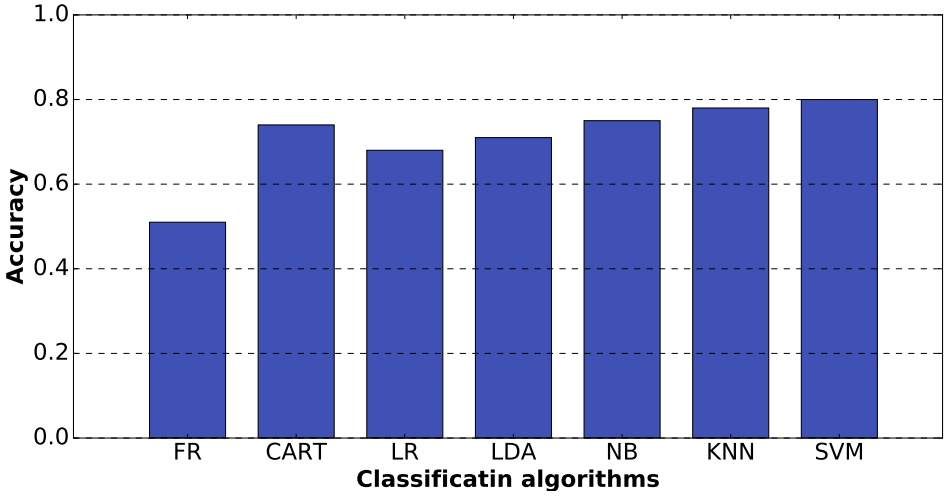


Figure 7.2: Accuracy of various classification algorithms for predicting the best next sensor

linear discriminant analysis (LDA). The rest are non-linear algorithms: K-nearest neighbors (KNN), classification and regression trees (CART), Gaussian naive Bayes (NB), and Support Vector Machine (SVM).

Figure 7.2 shows the average accuracy of each algorithm when predicting the next sensor for all targets given the current sensor and out-direction. We also compare, FR which denotes the frequency-based prediction algorithm that considers frequency only. The result shows unsurprisingly that out-direction is highly correlated with the next sensor as can be seen from the poor accuracy of FR compared to algorithms that use out-direction. Also, non-linear algorithms generally outperform linear algorithms. SVM shows the best accuracy. In the rest of the evaluation, SVM is used to predict the next sensor.

There are three metrics used in the evaluation. The first is sensor uptime, denoting how long sensors remain on. Algorithms with lower uptime are better. The second metric is precision, the ratio of time that a target is in view of a camera to the total time the camera is on. The third metric is recall, the ratio of time that the target is actively monitored by a camera (i.e., it is in the field of view, and the camera is on) to the time it is in the camera's field of view (when it is either on or off). Lower recall denotes that the sensor was not asked to stream data although the target was detectable.

Table 7.1 summarizes five experimental settings (or operation modes). Note that, the system sends an initial signal to the right sensor when the target first appears in the system. Thus, we assume that the initial location of the target is given.

We first demonstrate the impact of the sensor prediction algorithm by comparing operation modes 1, 2, and 3. These modes are same in delay prediction, while each has different sensor

Mode	Sensor prediction (selected sensors)	Delay prediction (turn on, timeout)
1	All sensors	(immediately, end of scenario)
2	Frequency-Based (FR)	(immediately, end of scenario)
3	Out-direction classifier	(immediately, end of scenario)
4	Out-direction classifier	(immediately, computed timeout)
5	Out-direction classifier	(minimum expected delay, maximum expected delay)

Table 7.1: Sensor modes

prediction. They immediately turn on selected sensors and never turn them off until the end of the scenario. Mode 1 is to turn on all the sensors while Mode 2 and 3 are to select a subset of sensors based on the frequency-based algorithm and out-direction classifier, respectively.

Figure 7.3 shows sensor uptime for sensor mode 1, 2, and 3 as a function of C_s, confidence level in sensor prediction. Since mode 1 always selects all the sensors, C_s does not make any difference in mode 1. When selecting all the sensors, total uptime of sensors (i.e., the sum of uptime for all sensors) is about 139.7 minutes. Mode 2 and 3 greatly cut down sensor uptime compared to mode 1. Even with 100% C_s, mode 2 and 3 are only 50.9 minutes, which is less than a half the uptime of mode 1. With a decrease of C_s, sensor uptime further decreases. Mode 2 is 30 times smaller (4.6 minutes), and mode 3 is 35 time smaller (3.9 minutes) than mode 1 when C_s is 0.2 (i.e., when selecting the best sensor).

Mode 2 and 3 seem to perform similarly concerning sensor uptime, but they have a big difference in detection error rates. Figure 7.4 shows precision of mode 1, 2, and 3 with increasing C_s. Mode 1 has the lowest precision since it almost always turns on sensors no matter whether the target is in the sensor range or not. Mode 2 and 3 have higher precision than mode 1 in general, but the precision of both modes gets closer to mode 1 as C_s increases. Notably, mode 3 has higher precision than mode 2 especially when C_s is small. In general, all of the modes show low precision (less than 0.006) because the camera's active time is much longer than the target's stay in the system. This situation is similar to the security cameras in stores that see real burglars tiny fraction of the time. Precision becomes higher with precise delay prediction described in the next section.

Figure 7.5 shows recall of mode 1, 2, and 3 as a function of C_s. Mode 1 shows 100% recall, which means when the target is in the sensor range, the sensor is always active. The system never misses a target if mode 1 is selected. Mode 2 and 3, however, sometimes lose the target. Even when C_s is equal to 1.0, recall of both modes is 99%. The missing 1%

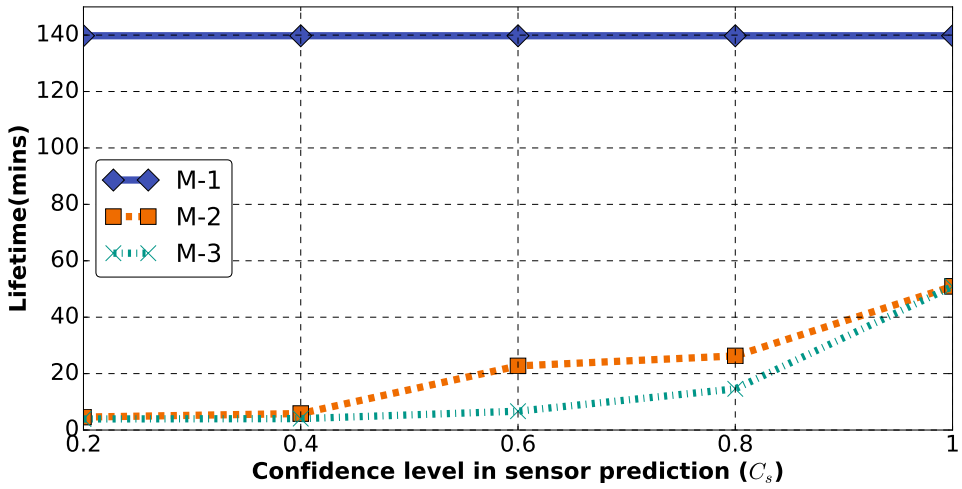


Figure 7.3: Uptime (minutes) of sensor mode 1, 2, and 3 as a function of confidence level in sensor prediction

occurs when the target movement is never learned before. In general, mode 3 shows better recall than mode 2, which confirms that the sensor prediction algorithm using the classifier performs better than the frequency-based as shown in earlier. Especially, when C_s is 0.2, recall of mode 2 is about 20% while that of mode 2 is around 70%.

So far, we have tested mode 1, 2, and 3 to show the impact of sensor prediction algorithms. Mode 1 is always able to detect the target while it dramatically increases resource consumption. Mode 2 selects frequently visited sensors first reducing resource consumption 30 times. Mode 3 offers the best savings but the worst recall.

The above results can be further optimized by accurately predicting delay. Mode 1, 2, and 3 immediately turn on the selected sensors. If the target appears, the sensor is turned off right after the target leaves the sensor. Otherwise, the sensor unnecessarily waits. Mode 4 immediately wakes up the next sensor but allows one to control the timeout period by adjusting confidence level in delay prediction, C_d. The larger C_d, the longer T_{out}. Mode 5 exploits learned delay in the given data set for better optimization. For this experiment, we set C_s to 0.8 for fair comparison among tested modes.

Figure 7.6 presents the uptime of sensor mode 3, 4, and 5 as a function of C_d. Mode 3 is independent of C_d, so its uptime is constant at 14.5 minutes. In the case of mode 4, uptime of sensors linearly increases with C_d because timeout period linearly increases. We set the proportionality factor such that the uptime is equal to mode 3 when C_d is 1.0. In mode 5, using delay percentiles, when C_d is 1.0, the uptime of sensors is only 4% compared to mode 3.

This result influences the precision of modes 3, 4, and 5 as shown in Figure 7.7. In general,

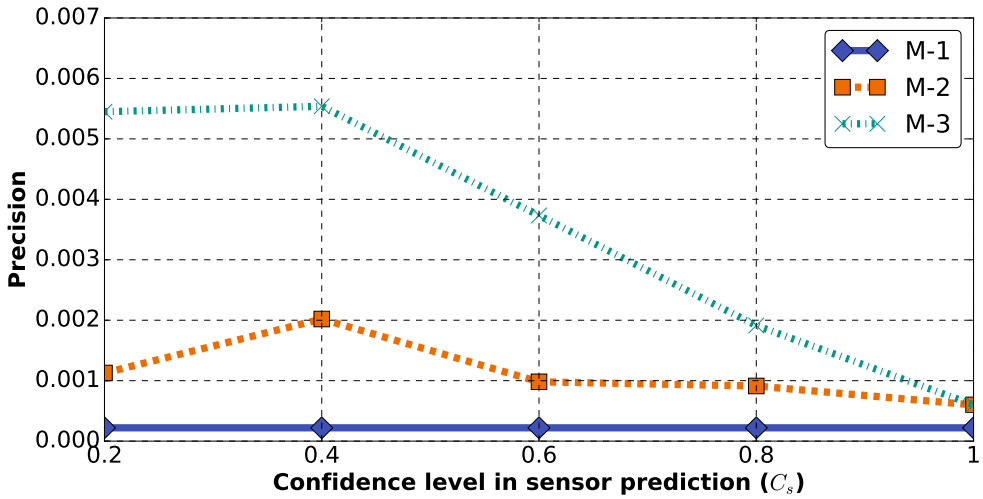


Figure 7.4: Precision of sensor mode 1, 2, and 3 as a function of confidence level in sensor prediction (C_s)

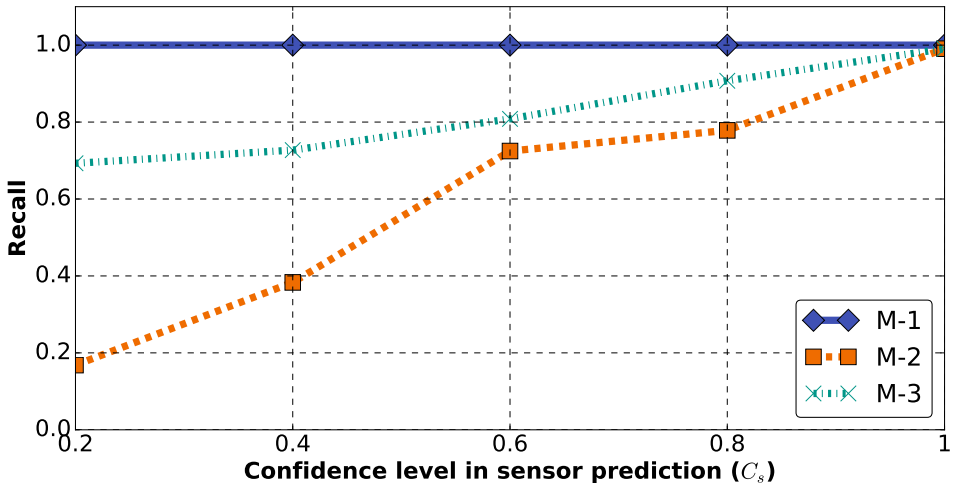


Figure 7.5: Recall of sensor mode 1, 2, and 3 as a function of confidence level in sensor prediction (C_s)

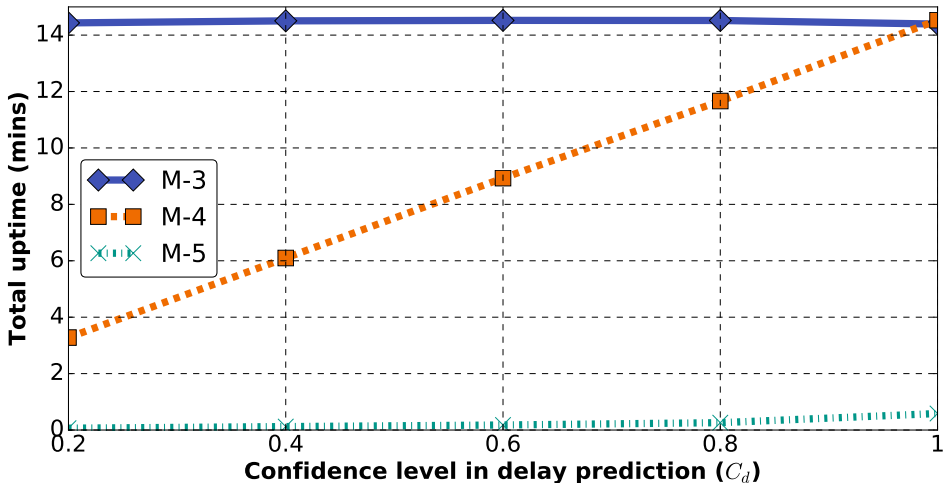


Figure 7.6: Uptime (minutes) of sensor mode 3, 4, and 5 as a function of confidence level in delay prediction (C_d)

mode 3 has the lowest precision, which is below 0.01. Mode 4 is slightly better than mode 3. The precision of mode 5 is at least an order of magnitude higher than that of the other modes under same C_d. Precision of mode 5 with 1.0 C_d is about 5 times higher than that of mode 4 with 0.2 C_d.

To lazily turn on sensors is not always the best policy, though. This strategy may cause sensors to miss targets because they are inactive while targets are detectable. Figure 7.8 shows recall of mode 3, 4, and 5 as a function of C_s. Recall of mode 3 and 4 is around 0.9, meaning that targets are detected with 90% chance if they are detectable. Although there is a slight drop in mode 4 if C_d is 0.2, its recall is still 0.89. Recall of mode 5 is highly affected by C_d. Its recall drops to 0.2 and goes up to 0.85 depending on C_d. This result means that sensors may be turned on too late or turned off too early depending on C_d.

Evaluations on uptime, precision, and recall for mode 3, 4, and 5 explain the impact of the delay prediction algorithm. Resources could be saved by choosing the right sensor and could be further optimized by selecting the proper delay. Sensors in mode 5 with 1.0 C_d, have an uptime that is about 24 times less than in mode 3, while the detection rate is only 5% lower. If we compare mode 5 with mode 1, the most vigilant mode, the difference is more dramatic. Sensor uptime can be saved by a factor of more than 238 times while targets remain detectable with an 85% chance.

The results provide a couple of lessons. First, our approach helps optimize resources by correctly predicting the right sensor and the proper delay. We have trained 284 targets and performed cross-validation. More features would help to improve prediction accuracy. When considering out-direction on top of mode 2, the sensor prediction accuracy dramatically

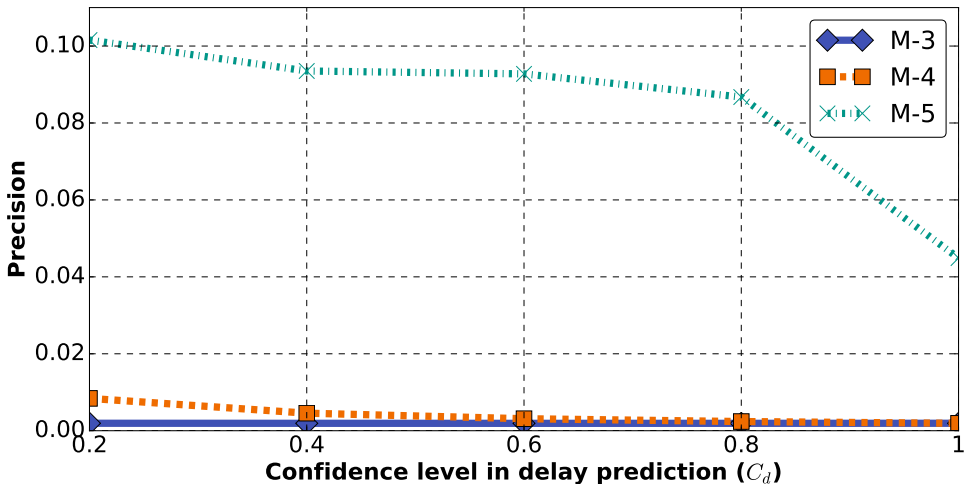


Figure 7.7: Precision of sensor mode 3, 4, and 5 as a function of confidence level in delay prediction (C_d)

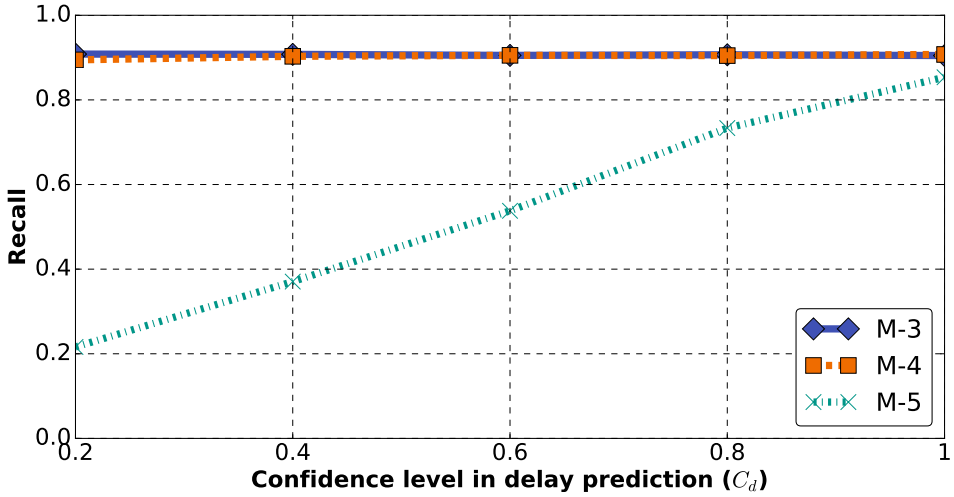


Figure 7.8: Recall of sensor mode 3, 4, and 5 as a function of confidence level in delay prediction (C_d)

increases. If we use more features to improve prediction, the system performance could be further optimized.

Second, there exists a trade-off between resource optimality and system reliability, and it is controllable. Each sensor mode has pros and cons. While mode 1 looks the worst regarding resource consumption, it is the only mode able to detect targets without misses. There is 1% error in recall for mode 2 and 3 even with a maximum C_s and C_d because some target mobility patterns were never learned before. Mode 1 can cover this unexpected case because it remains vigilant at all time.

On the other hand, Mode 5 shows a relatively low recall when C_d is 1.0, but it has the best precision and resource consumption. If 85% is a reasonable recall, then mode 5 is the best resource-efficient choice. This trade-off can be fine-tuned by adjusting control parameters. If the system resource budget is extremely limited, then mode 5 with low C_s and C_d could be the only viable solution. When the system budget is enough, then mode 5 with high C_s and C_d would be a cost-effective solution. The system can, therefore, be adaptable to the operating scenario. For example, if camera feeds need to be sent to a remote location over a contested network for visual inspection, the above controls allow appropriate selection of operating modes to the bandwidth available.

7.1.3 Related Work

Work described in this section was motivated by scenarios where local sparsely deployed sensors are used to track targets in a city. These sensors might belong to the infrastructure of a friendly government and themselves might not have the intelligence to identify targets. Hence, communication would typically be necessary between sensors and a remote location where a detection capability may exist. The goal becomes to minimize communication while still detecting most targets.

We chose target tracking because it is a well-studied problem [66]. Tree-based tracking [67, 68] statically organizes nodes in a hierarchical tree and lets border nodes be active all the time. Cluster-based protocols dynamically select a cluster head to prevent the battery of border nodes from getting quickly drained [69, 70, 71, 72]. Prediction-based tracking further optimizes above approaches by predicting the future location of the target [73, 74, 75]. While mobicast message-based monitoring is similar to the prediction-based methods, it tries to predict object moving direction to wake up appropriate nodes before the object arrives [76, 77]. Hybrid approaches were proposed that combine above protocols [78, 79].

In contrast to the much prior literature, we do not use dynamic models of targets to predict mobility patterns, but rather use machine learning. The reason is that the sparse

nature of deployed sensors assumed in this section makes it impossible to predict target locations from equations of motion. For evaluation, we extract ground-truth target location information from a real data set collected by Kestrel [64], the video analytic system. This approach is similar to prediction-based and mobicast message-based tracking in the sense that the system predicts the target movement. The work sheds light on the inherent degree of predictability of target mobility patterns in a real city. We conclude that most targets generally have highly predictable behaviors that allow for significant resource economies. Hybrid approaches are possible such as switching to an all-sensors-on mode when a target is "lost". If this is more of an exception (e.g., only 15% of the time), nearly an order of magnitude savings can still be attainable.

7.1.4 Conclusions

This work compares approaches to select the right sensor at the right time for target tracking. Data, extracted from Kestrel, is analyzed and predictors are trained first. Then, the trained data is fed to the system to make accurate predictions for sensor selection and wake-up delay. Evaluation results show that the approach can control resource consumption while introducing limited detection errors. The trade-off between resource efficiency and detection reliability can be adjusted by controlling the provided parameters. In future, we plan to integrate the algorithm into the real system to verify the the simulated results in the real system.

7.2 SITUATION AWARENESS

SNS applications can be a great source of social news because people often observe and post urban events just like sensors. This demo illustrates iApollo, a social news engine, that provides a highly summarized newsfeed service based on user-specified interests. To achieve its goal, the application first allows people to define the exact customized profile of the content feeds they want to receive. The profile is usually defined as a combination of search keywords and location tags. However, users cannot retrieve all the collected tweets because there are just too many. Therefore, it is necessary for iApollo to use a smart summarization or sampling technique. Samples could be either 1) more representative to provide a quick overview for the news topic or 2) more relevant to a particular sub-topic to provide information in depth. Let us call the two types of news summaries headline and complementary news, respectively.

To enable such summarization, the application server employs *Espresso*, the automatic naming agent, which constructs a hierarchical namespace for a given set of tweets based on their content similarity. It first generates a binary tree by iteratively clustering similar tweets according to a string metric until every cluster contains only one tweet. Thus, more relevant tweets are more likely to be placed under the same parent node. This favorable property translates the above sampling problem into a simple tree traversal. For example, the breadth-first-search-like tree traversal produces headline news, whereas the depth-first-search-like tree traversal produces complementary news. A detailed description about tree traversals and sampling orders can be found in the following subsection. Finally, these selected tweets are sent over a named data network (NDN), a new Internet architecture that requires every piece of transmitted data to be named. The main benefit of using an NDN is caching. Different than the end-to-end based IP network, intermediate NDN forwarders are capable of forwarding their cached data if the interest and data names are matched. This greatly reduces the retrieval delay of the client side and the network bottleneck of the server side. Moreover, the above NDN requirement is naturally resolved by *Espresso* because it assigns automated names to individual tweets.

7.2.1 System Design

This section provides a high-level description about the system design. It consists of the overall architecture, hierarchical namespace design, the two types of summarization, and iApollo's producer/consumer model.

The server first collects tweets from the Twitter through REST APIs based on user-specified interests; these are often a set of keywords with location tags. *Espresso* [80], the automatic naming agent, constructs a hierarchical namespace for the given set of tweets and decides its sampling order [43, 36]. There are two controllable knobs: a distance metric and a retrieval order. A detailed explanation about these knobs can be found in the following paragraph. Next, the sampled tweets are transmitted to news readers over NDN.

This subsection explains the automatic namespace design. As described above, *Espresso* constructs a hierarchical namespace for a given set of tweets. The overall process is shown in Figure 7.9. First, *Espresso* converts tweets into vector points after filtering noisy tweets and stripping useless keywords (e.g., hyperlink). After that, *Espresso* computes the distance among the tweets based on the Tanimoto distance, a vector extension of the Jaccard index. Then, a binary tree can be constructed by the recursive two-means clustering algorithm. The final process is to convert the generated tree into a namespace, which can be done by simply concatenating node names from root to leaf.

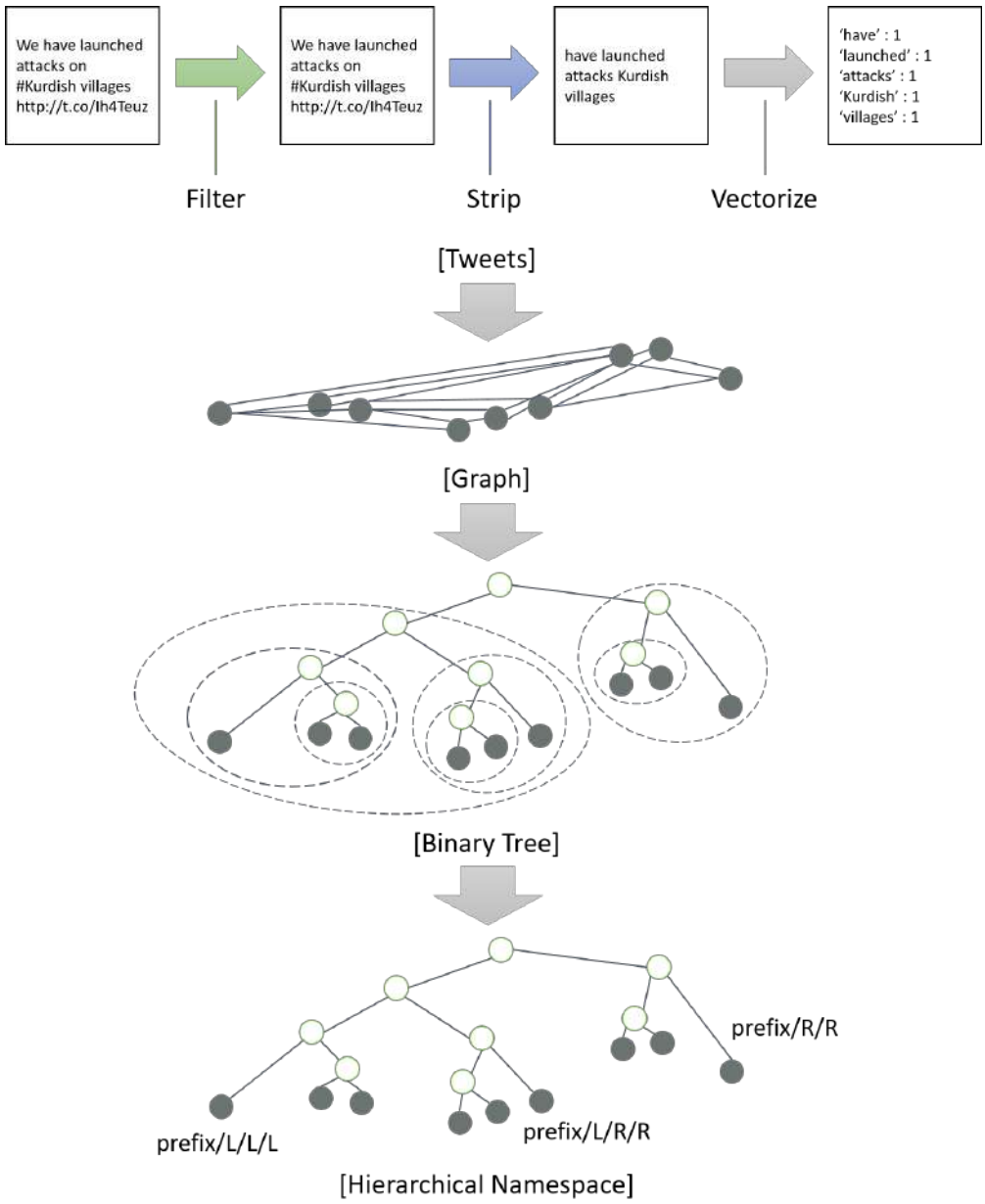


Figure 7.9: A process to construct a hierarchical namespace for tweets

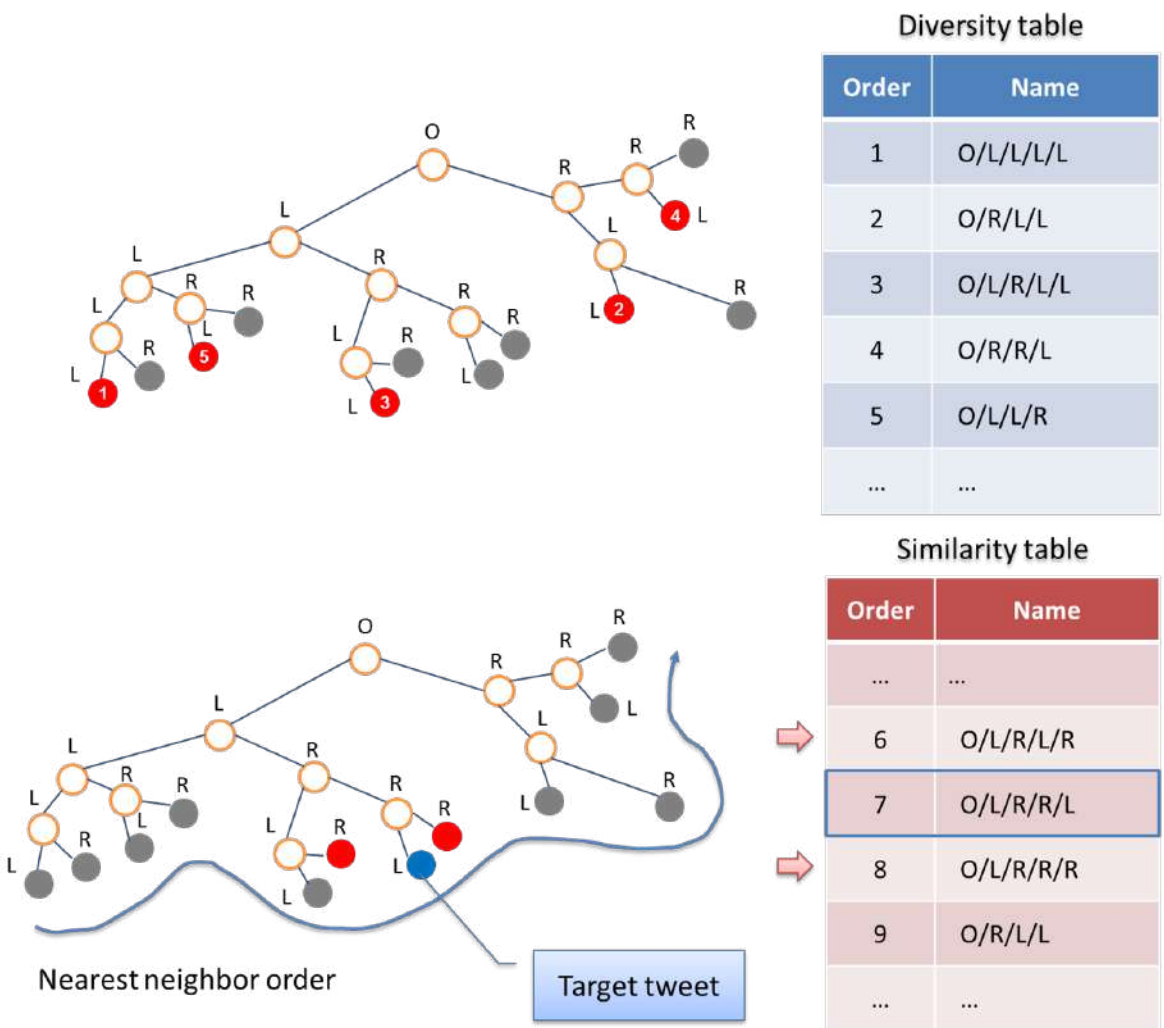


Figure 7.10: Two different types of retrieval orders in iApollo

iApollo provides two types of summarizations: headline and complementary news. When searching for a large data set, it is typical to combine macroscopic and microscopic views for better efficiency. Because the constructed tree (namespace) is based on content similarity, these sampling orders can be achieved through simple tree traversals, as shown in Figure 7.9. The upper figure shows how to select representative samples (headline news) from the given tree. The algorithm, similar to a BFS, traverses the tree from root to leaf by primarily selecting the least-visited branch. This results in the order shown in the diversity table, in which adjacent data are less likely to be redundant. On the other hand, the similarity table in the lower figure shows that adjacent data are more similar than distant data. The algorithm to achieve this is similar to a DFS except it does not select intermediate nodes because the NDN namespace assumes that the contents reside only in the leaf nodes.

The communication model of iApollo is as follows. The first interest is retrieving meta-

information from the user when launching the application. This includes the title, the top-five frequent keywords, and the latest version of each newsfeed. Because the server periodically crawls and organizes tweets, users need to know the latest version of the particular newsfeed so as not to retrieve stale news. This meta-information itself also needs to always be fresh. Therefore, the consumer marks the current UNIX timestamp on the interest packet to make it always reachable to the end producer, assuming that both the server and client are loosely synchronized through the NTP protocol.

The reader, upon selecting one of newsfeeds, triggers the network consumer for the next interest. To retrieve headline news for the chosen newsfeed, the consumer needs to know the data names of the headline news beforehand. Therefore, the second interest is requesting a name list (or catalogue) of the headline news. Upon receiving the name list, the consumer starts to request individual tweets (third interest). More details about this data retrieval protocol are described in [43]. If the user wants to have more headline news, then it simply repeats issuing the second and third interest packets. The fourth interest requests complementary news for the specific tweet. Similar to headline news, the data packet brings a name list, and the consumer can request actual tweets based on these names.

7.2.2 Application instruction

This subsection describes the system requirements to run iApollo. It also provides a few brief tutorials on how to use iApollo.

To run the iApollo, users need to have an iOS device.[1] iApollo is best supported on the iPad, though it supports other iOS family devices as well. Furthermore, iApollo users must be using iOS 10.2 or later. The source code of the iApollo server and client can be found in [81] and [82], respectively. We plan to publish the client application to Apple's App store in near future following several pilot tests.

When a user launches the application, they are prompted to input a user name and a password. Because the sign-up page is currently under construction, we allow users to log in through Facebook. If everything goes well, users can see meta-information, including the title, top-five frequent keywords, and the most recently updated timestamp of each newsfeed.

There are three views that can be selected by the bottom tab controller: public, private, and settings. The public view displays common newsfeeds created by the administrator. The public newfeeds are often the trending topics that may interest most readers. The private view, on the other hand, is the newsstand customized for individual users. Users can create their own news by clicking the upper-right "+" button, which opens the "create story" page.

[1] This means we only need some space to display two or three iPads for a demo.

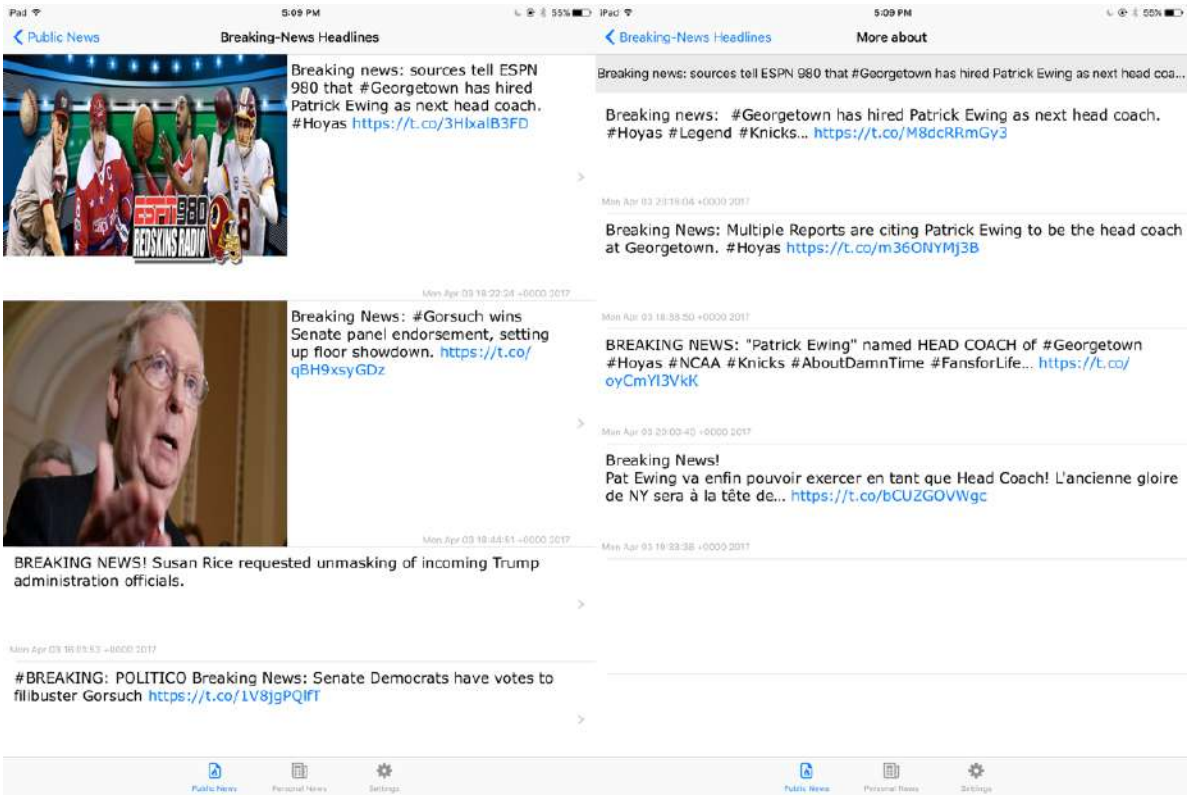


Figure 7.11: Headline and complementary views of iApollo on iPad

The new newsfeed will be available after about an hour from the request submission. This is necessary because we need to spend time to crawl and cluster tweets. Finally, the setting view allows users to configure various options.

Upon clicking one of newsfeeds, top-10 headline news would be displayed. If users scroll down the page, iApollo loads the next 10 headline news. Headline news with an indicator denotes that there is complementary news that is partially redundant. We cap the number of complementary news pieces at 10. The screenshots of headline and complementary views are shown in Figure 7.11.

7.2.3 Conclusions

The novelty of iApollo lies in the fact that this framework can be extended to support various summarization applications. *Espresso* is a great match with an NDN for this purpose. We expect to have more interesting summarization applications based on the combination of *Espresso* and an NDN in the future.

CHAPTER 8: RELATED WORK

There are four broad categories that are related to the current thesis. One of the crucial components in this work is the data retrieval scheduler; hence, we first review efficient data collection protocols. This work is broadly categorized into the decision-centric management framework; thus, the next section briefly discusses the literature on it. Redundancy minimization techniques would be next to consider because they provide similar functions to InfoMax, while most of the previous approaches are application specific. Finally, we briefly summarize literature work for target tracking, a classic military application. Situation awareness is another popular type of application, yet it is highly relevant to data collection protocols.

8.1 DATA COLLECTION PROTOCOLS

Efficient data collection protocols have been extensively studied in wireless sensor networks (WSNs) because the network environments in WSNs are often considered scarce or limited. Therefore, many researchers have proposed optimized data collection algorithms to save network bandwidth and prolong battery lifetime. The authors in [83] studied existing data collection mechanisms. According to this work, there are four categories: event driven, time driven, query driven, and hybrid.

Event driven is where sensors detecting significant or unusual events initiate the process, and the end user receives their reports through a sink node. The events are often defined as threshold values or probabilistic models configured at the sensor side. It is perhaps the most popular method because of its communication efficiency that results in low power consumption. Also, it is suitable for time-sensitive applications such as fire alarms. On the other hand, it is hard to capture continuous and complex events that may need multiple sensors. Another disadvantage is a high false positive error for event detection that is caused by inaccurate sensor measurements or sensor misconfigurations.

Earlier research proposed threshold-based event detection, where sensors report measurements if they are above the user-defined threshold value [84, 85]. A data fusion tool was introduced to decrease false positive errors for resilient systems in [86]. This work lets sensors send data to their cluster head instead of a sink so that the cluster head distinguishes between the outlier and event. Similarly, a double decision mechanism was proposed, allowing a sensor to request more data from nearby sensors to decide on the presence of an event [87]. More advanced methods are based on probabilistic models, as shown in [88, 89].

These works compressed the data volume by having sensors only report anomalous values that are not expected from the defined model. The authors of [90] exploited a decision tree to improve detection accuracy. Also, the classification that results from individual sensors are sent to a cluster head at which the values are aggregated. These aggregated values are shared among cluster heads for a vote to make a final decision.

Time driven is used for sensors to report their measurements to a sink periodically. One of the main advantages is enabling continuous monitoring of environmental changes such as habitat monitoring. Similar to event-driven approaches, the reporting period needs to be preconfigured depending on the application needs. This period is critical to determine network lifetime because more frequent reporting would more quickly drain battery power. Therefore, the literature has mainly searched for the best life cycle of sensors that extends the network lifetime without losing application correctness.

The authors of [88] constructed probabilistic models for sensors to determine the next reporting period. The sampling techniques were applied on top of the probabilistic models to decrease the number of transmitted messages in [91, 92]. In these work, sampler nodes are dynamically changed to balance the battery consumption of the sensors. Another approach is cascading data collection (CDC) presented in [93]. This work uses in-network processing that aggregates packets at intermediate nodes to compress transmitted messages. It is further studied in [94] to balance the overall energy consumption of sensor nodes by adapting its routing and MAC protocol behavior through shared local and neighborhood state information.

Different than the above approaches, where the initiator is a sensor, query driven is initiated by an end user. That is, query driven is request response. The end user issues a query in an appropriate language (*e.g.*, SQL), then, it is disseminated over the network. Upon receiving the query, sensors decide whether to route the data back to a requester based on the description of the query. The main benefit of the query driven is to provide end users with database-like interfaces by abstracting underlying complicated communication protocols. The downside of this approach is communication inefficiency. As mentioned, query driven is the request-response manner; thus, sensors become more reactive. Also, an application's need must be translated into queries because a query language is inappropriate to express higher-level behavior. Mistakes in formulating queries may result in retrieving a large amount of data or decreasing application correctness.

The first research introducing the concept of a distributed database to WSNs was COUGAR [95]. The authors of the work developed a query layer between the application and network layers so that the middleware would translate given queries to network protocol messages (*i.e.*, dissemination, and collection protocols). TinyDB is a similar approach, yet it was imple-

mented on the TinyOS operating system [96]. The framework is extensible, complete with declarative queries, and can be applied to in-network processing to reduce the transmitted data volume further. Because in-network processing incurs extra delay and energy consumption at intermediate nodes, pocket driven trajectories (PDT) was proposed to focus on the selectivity of aggregatable queries [97]. In [98], the authors considered the mobility of a mobile sink node that issues a query. The algorithm dynamically chooses cluster heads that perform data collection and aggregation so that they can send data back to a sink node. [99] allows users to embed optimization constraints in queries so that sensors can only send updates rather than their readings. Corona is energy-efficient multi query processing that is presented in [100]. The main features of the system are being user-friendly GUI and multi tasking capabilities by concurrently running multiple queries.

There is also a hybrid approach that combines the above methods. The work proposed in [101] adaptively switches the collection protocol between time driven and event driven. For instance, if sensors detect the defined event, then time driven is triggered to monitor the target event continuously. The authors of [102] combined event driven and query driven. Sensors send data only to the cluster heads based on predefined threshold values. The cluster heads then handle queries based on the collected data without forwarding them to individual sensors. Another hybrid-based protocol is [103]. A mobile node moving through the network region receives data from statically deployed sensors. The mobile node then periodically visits a sink node to drop off the collected data.

8.2 DECISION-CENTRIC MANAGEMENT

Athena is broadly related to the concepts of decision-driven resource management, decision-centric information monitoring, and decision support systems that have been studied extensively in prior literature in multiple domains. A significant amount work on decision-centric management revolves around managing software architecture decisions [104, 105, 106], product design [107], business processes [108], business intelligence [109], and physical organizations [110] (including human resources and financial assets). In contrast, we are the first to focus on decision-driven management of run-time communication and scheduling in networks that deliver sensory information. Below, we elaborate on this distinction, comparing our work more clearly to the general directions in current literature, including decision-centric management, decision-support tools, and decision-driven information monitoring tools.

Much of today's decision-centric management frameworks are studied in disciplines such as business management [108, 109] and organizational design [110]. They refer to human-centric processes that empower decision-makers, as opposed to run-time algorithms such

as communication protocols and scheduling policies in computing machines. A category that comes closer to computer science in this context is the category of managing software architecture design [104, 105, 106]. Architectural decisions affects software performance outcomes. Hence, much work is focused on understanding the dependencies between decisions and outcomes, offering tools to enable decisions that lead to improved outcomes, such as improved performance [106], improved security threat mitigation [111], and improved defense [112]. For example, decision-driven architecture reviews [106] offer opportunities to assess the suitability of architectural decisions to design purposes early on the development cycle. Tools for decision driven business performance management [113] document the relation between performance metrics on one hand and decisions/sub-decisions on another, allowing one to explore the decision space by modifying decisions and tracking their impact on performance metrics. These approaches are intended for use to inform design early on in the software development cycle (i.e., before deployment). In contrast, we are interested in communication protocols and scheduling policies that offer run-time support.

A somewhat closer tool category to our work is the category of decision support systems [114]. These systems use models of the world to allow decision-makers to play "what-if" scenarios and explore consequences of their decisions on the modeled domain of interest. Hence, for example, given an adequate model of the political, military, economic, social, and topological terrain features in some foreign state, a decision-maker might estimate potential consequences of a certain military operation. Various planning tools have also been proposed that allow decision-makers to compose detailed plans of action and contingencies based on domain models [115]. Work reported in this work does not constitute a decision-support tool in the above sense. The software described in this work does not have models of the world, and is unable to compose plans. Rather, it helps users to retrieve information objects in a manner that facilitates meeting decision needs.

The concept of decision-centric information monitoring comes closest to our work. A decision-centric information monitoring system [116] answers the following key question: which variables must one monitor in order to make an informed decision? The answer often comes from modeling the decision-making process [109, 117]. An interesting trade-off is involved. Monitoring too many variables may overwhelm the decision maker, whereas monitoring too few may impair quality of decisions. This work is synergistic with ours in that we assume that the problem of determining the relevant variables (or unknowns) has already been solved. Cast in the context of a cyber-physical systems in which unknowns that determine the viability of each potential course of action have been identified, our work addresses the complementary problem of contacting the sources (e.g., sensors) that would deliver information on these unknowns over a network while minimizing delivery cost.

This network communication and scheduling policy challenge is orthogonal to the manner in which relevance of specific unknowns was determined in the first place.

Recent work has made initial progress at solving the above cyber-physical network resource management problem [118, 119, 25]. The work bears resemblance to prior database research that considered explicit data access transactions and required a degree of data freshness [120, 121, 122, 123]. It is new in tying such retrieval policies explicitly to the decision needs, leading to new research challenges and solutions as described below.

8.3 REDUNDANCY MINIMIZATION

InfoMax falls in the category of saving resource by reducing data transmission volume. Much work was done in this area over the decades, from compressing data [124, 125, 126] to selectively sending a subset [127, 128]. Our research is closer to the latter category. It is unique in exploiting a generic scheme that sits in the communication layer and does not require application-specific knowledge. An advantage of this approach is that different applications can use it, instead of relying on their own application-specific solutions.

InfoMax extends a previous transport layer abstraction, called the *Information Funnel* [35], proposed earlier by a subset of the authors. While the latter was geared for data *collection* by a single consumer from multiple producers, InfoMax is geared for the complementary case of data *dissemination*, where multiple consumers pull data from the same producer, possibly at different levels of summarization. Together, the Information Funnel and the InfoMax dissemination protocol may constitute the two building blocks of many data-intensive services. The funnel will collect data into repositories from multiple sources, whereas InfoMax will serve it to multiple eventual consumers at different degrees of detail.

Our name space construction is inspired by SocialTrove [36]. This unique work provides a general summary model for any given data set, represented as a binary tree, if a correct distance metric is defined. While SocialTrove did not aim at data naming, it clustered data by recursively dividing objects into two disjoint sets using 2-means clustering algorithm [129]. We use the same approach for naming.

Espresso is also differentiated in that it flexibly balances importance of content versus diversity. While both predecessors only focus diversity, *Espresso* allows applications to control the trade between diversity and importance in a customizable manner.

8.4 TARGET TRACKING

Work described in this work was motivated by scenarios where local sparsely deployed sensors are used to track targets in a city. These sensors might belong to the infrastructure of a friendly government and themselves might not have the intelligence to identify targets. Hence, communication would typically be necessary between sensors and a remote location where a detection capability may exist. The goal becomes to minimize communication while still detecting most targets.

We chose target tracking because it is a well-studied problem [66]. Tree-based tracking [67, 68] statically organizes nodes in a hierarchical tree and lets border nodes be active all the time. Cluster-based protocols dynamically select a cluster head to prevent the battery of border nodes from getting quickly drained [69, 70, 71, 72]. Prediction-based tracking further optimizes above approaches by predicting the future location of the target [73, 74, 75]. While mobicast message-based monitoring is similar to the prediction-based methods, it tries to predict object moving direction to wake up appropriate nodes before the object arrives [76, 77]. Hybrid approaches were proposed that combine above protocols [78, 79].

In contrast to the much prior literature, we do not use dynamic models of targets to predict mobility patterns, but rather use machine learning. The reason is that the sparse nature of deployed sensors assumed in this work makes it impossible to predict target locations from equations of motion. For evaluation, we extract ground-truth target location information from a real data set collected by Kestrel [64], the video analytic system. This approach is similar to prediction-based and mobicast message-based tracking in the sense that the system predicts the target movement. The work sheds light on the inherent degree of predictability of target mobility patterns in a real city. We conclude that most targets generally have highly predictable behaviors that allow for significant resource economies. Hybrid approaches are possible such as switching to an all-sensors-on mode when a target is "lost". If this is more of an exception (e.g., only 15% of the time), nearly an order of magnitude savings can still be attainable.

CHAPTER 9: CONCLUSIONS

In the current thesis, we proposed a new mission-driven summarization layer as an adaptable solution for tactical communication in a contested battlefield environment. This layer optimizes network resources on behalf of applications to meet decision needs; thus, resource consumption is mission-driven. The idea is inspired by the command-by-intent paradigm that maximizes the autonomy of subordinate units by merely specifying the high-level purpose of the mission. Also, the novelty of the thesis lies in its generality that can implement various application contexts by simple changes on metadata, such as decision rules. Therefore, the system design tends to be simpler than application-specific solutions.

We built two applications to show the benefits of summarization services. The first application is target tracking, where the mission is to detect the given target by retrieving images from city cameras. The data retrieval scheduler exploits a logical short-circuiting evaluation to minimize the number of active cameras. Also, the application shows that the result can be further optimized with an accurate prediction algorithm. The second application addresses situation awareness: it simulates soldiers retrieving battlefield information to understand the current situation. To this end, we use the tweets as a dataset because these tweets have voluminous and noisy characteristics that are shared with war information. Redundancy-based summarization is helpful for this type of application because human soldiers may not be able to handle all the incoming reports. The summarization service would filter out redundancy and noisy data but highlight representative information first so that the users can understand the situation quickly with the minimal amount of data retrieval.

The primary purpose of the current thesis is to provide adaptable solutions to support tactical communication in a contested network environment. To show the effectiveness of the proposed idea, it requires building an actual system on which the real applications could work. This requires non-trivial effort to make the system work with the applications. The implementation is about 30K lines of the code in Python and pip compliant to simplify reuse [130]. One of the main contributions of this work is not only sharing the ideas for mission-driven summarization, but also providing a real implementation to the research community.

The components in the code have been independently developed for each different project, but they are merged into one package at the end. The InfoMax protocol was initially designed on the NDN protocols for the Future Internet Architecture project, while the Athena code was developed for the NS-CTA project. The two pieces are merged into one library package under the name Athena. The new code was tested on several different platforms. It

was initially tested with EMANE, the wireless ad-hoc emulator tools developed by the Navy Research Laboratory (NRL), on top of the DAVC framework, a virtual clustering system managed by ARL (Army Research Laboratory). Raytheon, one of the leading industrial corporations for military electronics, developed the disruption-tolerant routing protocol, called MUCAN (Mission, User, Content Adaptive Network). Athena could successfully run on MUCAN with minor modifications. Sandia National Laboratory (SNL) has a virtual machine management tool called Emulytics. It is known as fast, easy to deploy, and scalable compared with other virtual machine platforms. Athena could also run on top of Emulytics. These examples show that Athena is a light-weight and adaptable solution that can work with other protocols on top of various platforms.

We believe that application driven is the right way to advance technologies. There are many intelligent and creative theories in the world, but few of them contribute to the real world. This work wants to become a real and practical solution rather than just a theoretical model. Therefore, we wanted to show that the application could work on the summarization layer and could benefit from Athena. In this philosophy, one of the significant efforts made here was to implement the applications. Although the applications introduced in the current thesis may not be as mature as production software, they are sufficient to show the effectiveness of the summarization services. Developing more military applications on the real battlefield would be a great addition to this work. History has shown that killer applications drive technological advance as can be seen from web browsers and email.

One of the other ways to improve this work would be to study user satisfaction. The primary metric of the evaluations here is network resource usage such as bandwidth. However, the summarization services can benefit users. There are two types of users: programmers and end users. The programmers or application developers would be able to spend less effort to build the applications because the summarization functions would already be in use. Also, end users can benefit from Athena. They may avoid getting unnecessary or redundant data by utilizing the summarization services. For instance, users may get noisy data when collecting tweets if the summarization function is disabled. If the feature is enabled, high-level summary data would only be retrieved so that user readability is improved. There could be several ways to measure user satisfaction, and a case study could be one. The case study may need to be conducted for soldiers because our primary target applications are military.

There are several opportunities to optimize the proposed algorithms, and this could be one direction to extend this work. For example, in the data retrieval scheduler, we exploit a logical short-circuiting evaluation to retrieve only the necessary information. One way to optimize this would be to consider data caching. Cached objects have almost no cost to retrieve because they are stored locally or in the nearby nodes. Therefore, if cache

information is available, the algorithm can take this into account when deciding the retrieval order. The application can issue multiple decisions at the same time, and the summarization layer needs to schedule them to maximally resolve the given queries. The current system uses the simple FIFO (first-in-first-out) policy, but it would be interesting to study optimal scheduling policy to maximize schedulability. The problem becomes more complicated if there are timing constraints such as a decision deadline and data freshness periods.

REFERENCES

[1] M. Cox, "Army slashes old programs to fund new tech in 2020 budget request," 2019, Internet news. [Online]. Available: https://www.military.com/daily-news/2019/03/12/army-slashes-old-programs-fund-new-tech-2020-budget-request.html

[2] C. Barone, "Modernizing tactical waveforms for a more contested battlespace," https://modernbattlespace.com/2018/09/25/modernizing-tactical-waveforms-contested-battlespace.

[3] T. Meyer, L. Yamamoto, and C. Tschudin, "A self-healing multipath routing protocol," in *Proceedings of the 3rd International Conference on Bio-Inspired Models of Network, Information and Computing Sytems*, ser. BIONETICS '08. ICST, Brussels, Belgium, Belgium: ICST (Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering), 2008. [Online]. Available: http://dl.acm.org/citation.cfm?id=1512504.1512512 pp. 6:1–6:8.

[4] V. C. Giruka and M. Singhal, "A self-healing on-demand geographic path routing protocol for mobile ad-hoc networks," *Ad Hoc Networks*, vol. 5, no. 7, pp. 1113 – 1128, 2007. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1570870506000539

[5] C. Gui and P. Mohapatra, "Short: Self-healing and optimizing routing techniques for mobile ad hoc networks," in *Proceedings of the 4th ACM International Symposium on Mobile Ad Hoc Networking &Amp; Computing*, ser. MobiHoc '03. New York, NY, USA: ACM, 2003. [Online]. Available: http://doi.acm.org/10.1145/778415.778448 pp. 279–290.

[6] C. Douligeris and A. Mitrokotsa, "Ddos attacks and defense mechanisms: a classification," in *Proceedings of the 3rd IEEE International Symposium on Signal Processing and Information Technology (IEEE Cat. No.03EX795)*, Dec 2003, pp. 190–193.

[7] M. Aamir and M. A. Zaidi, "Ddos attack and defense: Review of some traditional and current techniques," *CoRR*, vol. abs/1401.6317, 2014, withdrawn. [Online]. Available: http://arxiv.org/abs/1401.6317

[8] J. Mirkovic and P. Reiher, "A taxonomy of ddos attack and ddos defense mechanisms," *SIGCOMM Comput. Commun. Rev.*, vol. 34, no. 2, pp. 39–53, Apr. 2004. [Online]. Available: http://doi.acm.org/10.1145/997150.997156

[9] G. Han, L. Xiao, and H. V. Poor, "Two-dimensional anti-jamming communication based on deep reinforcement learning," in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, March 2017, pp. 2087–2091.

[10] K. Grover, A. Lim, and Q. Yang, "Jamming and anti-jamming techniques in wireless networks: A survey," *Int. J. Ad Hoc Ubiquitous Comput.*, vol. 17, no. 4, pp. 197–215, Dec. 2014. [Online]. Available: http://dx.doi.org/10.1504/IJAHUC.2014.066419

[11] M. Pajic and R. Mangharam, "Anti-jamming for embedded wireless networks," in *2009 International Conference on Information Processing in Sensor Networks*, April 2009, pp. 301–312.

[12] M. Y. S. Uddin, H. Wang, F. Saremi, G.-J. Qi, T. Abdelzaher, and T. Huang, "Photonet: A similarity-aware picture delivery service for situation awareness," in *Proceedings of the 2011 IEEE 32Nd Real-Time Systems Symposium*, ser. RTSS '11, 2011, pp. 317–326.

[13] S. Wang, S. Hu, S. Li, H. Liu, M. Uddin, and T. Abdelzaher, "Minerva: Information-centric programming for social sensing," *Proceedings of the 22nd International Conference on Computer Communications and Networks*, pp. 1–9, July 2013.

[14] W. Dron, A. Leung, M. Uddin, S. Wang, T. Abdelzaher, R. Govindan, and J. Hancock, "Information-maximizing caching in ad hoc networks with named data networking," *Proceedings of Network Science Workshop*, pp. 90–93, 2013.

[15] S. Kumar, L. Shu, S. Gil, N. Ahmed, D. Katabi, and D. Rus, "Carspeak: A content-centric network for autonomous driving," *Proceedings of ACM SIGCOMM*, pp. 259–270, 2012.

[16] K. Marcus and J. Cannata, "Dynamically allocated virtual clustering management system," in *Proc.SPIE*, vol. 8742, 2013. [Online]. Available: http://dx.doi.org/10.1117/12.2018514 pp. 8742 – 8742 – 10.

[17] "Extendable mobile ad-hoc network emulator (emane)," 2019, Website. [Online]. Available: https://www.nrl.navy.mil/itd/ncs/products/emane

[18] "minimega: a distributed vm management tool," 2019, Website. [Online]. Available: http://minimega.org/

[19] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, "Internet of things for smart cities," *IEEE Internet of Things Journal*, vol. 1, no. 1, pp. 22–32, Feb 2014.

[20] R. Khatoun and S. Zeadally, "Smart cities: concepts, architectures, research opportunities," *Communications of the ACM*, vol. 59, no. 8, pp. 46–57, 2016.

[21] S. De, Y. Zhou, I. Larizgoitia Abad, and K. Moessner, "Cyber–physical–social frameworks for urban big data systems: A survey," *Applied Sciences*, vol. 7, no. 10, p. 1017, 2017.

[22] T. Abdelzaher, M. T. A. Amin, A. Bar-Noy, W. Dron, R. Govindan, R. Hobbs, S. Hu, J. E. Kim, J. Lee, K. Marcus, S. Yao, and Y. Zhao, "Decision-driven execution: A distributed resource management paradigm for the age of iot," in *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, June 2017, pp. 1825–1835.

[23] M. Srivastava, T. Abdelzaher, and B. Szymanski, "Human-centric sensing," *Phil. Trans. R. Soc. A*, vol. 370, no. 1958, pp. 176–197, 2012.

[24] J. A. Stankovic, "Research directions for the internet of things," *IEEE Internet of Things Journal*, vol. 1, no. 1, pp. 3–9, 2014.

[25] S. Hu, S. Yao, H. Jin, Y. Zhao, Y. Hu, X. Liu, N. Naghibolhosseini, S. Li, A. Kapoor, W. Dron, L. Su, A. Bar-Noy, P. Szekely, R. Govindan, R. Hobbs, and T. F. Abdelzaher, "Data acquisition for real-time decision-making under freshness constraints." in *RTSS*, 2015.

[26] S. Tufekci and W. A. Wallace, "The emerging area of emergency management and engineering," *IEEE Transactions on Engineering Management*, vol. 45, no. 2, pp. 103–105, May 1998.

[27] U. S. D. of the Army, *FM 100-5 Corprs Operations*. Department of the Army, 1993. [Online]. Available: https://books.google.com/books?id=6MaEAQAACAAJ

[28] T. Abdelzaher, M. T. A. Amin, A. Bar-Noy, W. Dron, R. Govindan, R. Hobbs, S. Hu, J. E. Kim, J. Lee, K. Marcus, S. Yao, and Y. Zhao, "Decision-driven execution: A distributed resource management paradigm for the age of iot," in *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, June 2017, pp. 1825–1835.

[29] J. Lee, K. Marcus, T. Abdelzaher, M. T. A. Amin, A. Bar-Noy, W. Dron, R. Govindan, R. Hobbs, S. Hu, J.-E. Kim, L. Sha, S. Yao, and Y. Zhao, "Athena: Towards decision-centric anticipatory sensor information delivery," *Journal of Sensor and Actuator Networks*, vol. 7, no. 1, 2018. [Online]. Available: http://www.mdpi.com/2224-2708/7/1/5

[30] V. Jacobson, D. K. Smetters, J. D. Thornton, M. F. Plass, N. H. Briggs, and R. L. Braynard, "Networking named content," in *Proceedings of the 5th international conference on Emerging networking experiments and technologies (CoNEXT'09)*. ACM, December 2009, pp. 1–12.

[31] L. Zhang, A. Afanasyev, J. Burke, V. Jacobson, kc claffy, P. Crowley, C. Papadopoulos, L. Wang, and B. Zhang, "Named data networking," in *SIGCOMM Computer Communication Review (CCR)*. ACM, July 2014.

[32] N. Suri, A. Hansson, J. Nilsson, P. Lubkowski, K. Marcus, M. Hauge, K. Lee, B. Buchin, L. Mısırhoğlu, and M. Peuhkuri, "A realistic military scenario and emulation environment for experimenting with tactical communications and heterogeneous networks," in *2016 International Conference on Military Communications and Information Systems (ICMCIS)*, May 2016, pp. 1–8.

[33] E. Elijah, "Information output is growing exponentially: 3 strategies for laboratory data management," May 2014, [Online; posted 22-May-2014].

[34] S. McCanne, V. Jacobson, and M. Vetterli, "Receiver-driven layered multicast," *SIGCOMM Comput. Commun. Rev.*, vol. 26, no. 4, pp. 117–130, Aug. 1996. [Online]. Available: http://doi.acm.org/10.1145/248157.248168

[35] S. Wang, T. Abdelzaher, S. Gajendran, A. Herga, S. Kulkarni, S. Li, H. Liu, C. Suresh, A. Sreenath, H. Wang, W. Dron, A. Leung, R. Govindan, and J. Hancock, "The information funnel: Exploiting named data for information-maximizing data collection," in *Proc. International Conference on Distributed Computing in Sensor Systems (DCOSS'14)*. IEEE, May 2014, pp. 92–100.

[36] M. T. A. Amin, S. Li, M. R. Rahman, P. T. Seetharamu, S. Wang, T. Abdelzaher, I. Gupta, M. Srivatsa, R. Ganti, R. Ahmed, and H. Le, "SocialTrove: A self-summarizing storage service for social sensing," in *International Conference on Autonomic Computing (ICAC'15)*. IEEE, July 2015, pp. 41–50.

[37] W. Shang, Q. Ding, A. Marianantoni, J. Burke, and L. Zhang, "Securing building management systems using named data networking," *Network, IEEE*, vol. 28, no. 3, pp. 50–56, May 2014.

[38] "C++ infomax," https://github.com/infomaxndn/InfoMax-CXX.

[39] "Infomax consumer for ios," https://github.com/infomaxndn/InfoMax-ConsumerSwift.

[40] Washington University in St. Louis, "NDN testbed," http://ndnmap.arl.wustl.edu.

[41] NFD developers, "NFD tutorial," http://named-data.net/doc/NFD/current/INSTALL.html.

[42] M. F. Bari, S. R. Chowdhury, R. Ahmed, R. Boutaba, and O. L. Bertrand Mathieu, "A survey of naming and routing in information-centric networks," *IEEE Comunications Magazine*, pp. 44–53, December 2012.

[43] J. Lee, A. Kapoor, M. T. A. Amin, Z. Wang, Z. Zhang, R. Goyal, and T. Abdelzaher, "InfoMax: An information maximizing transport layer protocol for named data networks," in *International Conference on Computer Communication and Networks (ICCCN'15)*. IEEE, August 2015, pp. 1–10.

[44] W. W. Cohen, P. Ravikumar, and S. E. Fienberg, "A comparison of string distance metrics for name-matching tasks," in *In Proceedings of IJCAI-03 Workshop on Information Integration*, August 2003, pp. 73–78.

[45] G. Hamerly and J. Drake, "Accelerating lloyd's algorithm for k-means clustering," in *Partitional Clustering Algorithms*, M. E. Celebi, Ed. Springer International Publishing, 2014, pp. 41–78.

[46] D. Arthur, B. Manthey, and H. Röglin, "Smoothed analysis of the k-means method," *J. ACM*, vol. 58, no. 5, pp. 19:1–19:31, Oct. 2011. [Online]. Available: http://doi.acm.org/10.1145/2027216.2027217

[47] S. K. Bhatia, "Adaptive k-means clustering." in *FLAIRS Conference*, 2004, pp. 695–699.

[48] Y. Yu, A. Afanasyev, Z. Zhu, and L. Zhang, "Ndn naming conventions," http://named-data.net/publications/techreports/ndn-tr-22-ndn-memo-naming-conventions/, UCLA, Tech. Rep., july 2014.

[49] I. Moiseenko, L. Wang, and L. Zhang, "Consumer / producer communication with application level framing in named data networking," in *Proc. 2nd International Conference on Information-Centric Networking (ACM ICN'15)*, September 2015.

[50] C. Elkan, "Using the triangle inequality to accelerate k-means," in *ICML*, vol. 3, 2003, pp. 147–153.

[51] S. Gupta and S. K. Gupta, "Abstractive summarization: An overview of the state of the art," *Expert Systems with Applications*, vol. 121, pp. 49 – 65, 2019. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0957417418307735

[52] N. Moratanch and S. Chitrakala, "A survey on extractive text summarization," in *2017 International Conference on Computer, Communication and Signal Processing (ICCCSP)*, Jan 2017, pp. 1–6.

[53] J. Lee, A. Kapoor, M. T. A. Amin, Z. Wang, Z. Zhang, R. Goyal, and T. Abdelzaher, "InfoMax: A transport layer paradigm for the age of data overload," in *Advances in Computer Communications and Networks - from green, mobile, pervasive networking to big data computing*, J. Fagerberg, D. C. Mowery, and R. R. Nelson, Eds. River Publisher, 2016.

[54] A. K. M. M. Hoque, S. O. Amin, A. Alyyan, B. Zhang, L. Zhang, and L. Wang, "Nlsr: Named-data link state routing protocol," in *Proceedings of the 3rd ACM SIGCOMM Workshop on Information-centric Networking*, ser. ICN '13. New York, NY, USA: ACM, 2013. [Online]. Available: http://doi.acm.org/10.1145/2491224.2491231 pp. 15–20.

[55] Y. Li, A. Afanasyev, J. Shi, H. Zhang, Z. Zhang, T. Li, E. Lu, B. Zhang, L. Wang, and L. Zhang, "Ndn automatic prefix propagation," Department of Computer Science, UCLA, Tech. Rep., March 2018. [Online]. Available: https://named-data.net/publications/techreports/ndn-0045-1-auto-prefix-propagation/

[56] Y. Ren, J. Li, S. Shi, L. Li, G. Wang, and B. Zhang, "Congestion control in named data networking – a survey," *Computer Communications*, vol. 86, pp. 1 – 11, 2016. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0140366416301566

[57] G. Carofiglio, M. Gallo, and L. Muscariello, "Icp: Design and evaluation of an interest control protocol for content-centric networking," in *2012 Proceedings IEEE INFOCOM Workshops*, March 2012, pp. 304–309.

[58] J. Lee, K. Marcus, T. Abdelzaher, M. T. A. Amin, A. Bar-Noy, W. Dron, R. Govindan, R. Hobbs, S. Hu, J.-E. Kim, L. Sha, S. Yao, and Y. Zhao, "Athena: Towards decision-centric anticipatory sensor information delivery," *Journal of Sensor and Actuator Networks*, vol. 7, no. 1, 2018. [Online]. Available: http://www.mdpi.com/2224-2708/7/1/5

[59] "NDN repositories," https://github.com/named-data.

[60] "Consumer-Producer-API," https://github.com/named-data/Consumer-Producer-API.

[61] K. Schneider, C. Yi, B. Zhang, and L. Zhang, "A practical congestion control scheme for named data networking," in *Proceedings of the 3rd ACM Conference on Information-Centric Networking*, ser. ACM-ICN '16. New York, NY, USA: ACM, 2016. [Online]. Available: http://doi.acm.org/10.1145/2984356.2984369 pp. 21–30.

[62] V. Paxson, M. Allman, J. Chu, and M. Sargent, "Computing tcp's retransmission timer," Internet Requests for Comments, RFC Editor, RFC 6298, June 2011, http://www.rfc-editor.org/rfc/rfc6298.txt. [Online]. Available: http://www.rfc-editor.org/rfc/rfc6298.txt

[63] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, ser. Hotnets-IX. New York, NY, USA: ACM, 2010. [Online]. Available: http://doi.acm.org/10.1145/1868447.1868466 pp. 19:1–19:6.

[64] H. Qiu, X. Liu, S. Rallapalli, A. J. Bency, R. Urgaonkar, B. Manjunath, R. Govindan, and K. Chan, "Kestrel: Video analytics for augmented multi-camera vehicle tracking," in *Proceedings of the 3rd ACM/IEEE International Conference on Internet-of-Things Design and Implementation(IoTDI'18)*, April 2018.

[65] J. Redmon, S. K. Divvala, R. B. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," *CoRR*, vol. abs/1506.02640, 2015. [Online]. Available: http://arxiv.org/abs/1506.02640

[66] S. Bhatti and J. Xu, "Survey of target tracking protocols using wireless sensor network," in *Proceedings of the 2009 Fifth International Conference on Wireless and Mobile Communications*, ser. ICWMC '09. Washington, DC, USA: IEEE Computer Society, 2009. [Online]. Available: https://doi.org/10.1109/ICWMC.2009.25 pp. 110–115.

[67] H.-W. Tsai, C.-P. Chu, and T.-S. Chen, "Mobile object tracking in wireless sensor networks," *Computer Communications*, vol. 30, no. 8, pp. 1811 – 1825, 2007. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S014036640700093X

[68] H. T. Kung and D. Vlah, "Efficient location tracking using sensor networks," in *2003 IEEE Wireless Communications and Networking, 2003. WCNC 2003.*, vol. 3, March 2003, pp. 1954–1961 vol.3.

[69] M. Wälchli, P. Skoczylas, M. Meer, and T. Braun, "Distributed event localization and tracking with wireless sensors," in *Wired/Wireless Internet Communications*, F. Boavida, E. Monteiro, S. Mascolo, and Y. Koucheryavy, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 247–258.

[70] H. Yang and B. Sikdar, "A protocol for tracking mobile targets using sensor networks," in *Proceedings of the First IEEE International Workshop on Sensor Network Protocols and Applications, 2003.*, May 2003, pp. 71–81.

[71] Y. Zou and K. Chakrabarty, "Sensor deployment and target localization in distributed sensor networks," *ACM Trans. Embed. Comput. Syst.*, vol. 3, no. 1, pp. 61–91, Feb. 2004. [Online]. Available: http://doi.acm.org/10.1145/972627.972631

[72] K. A. Darabkh, S. S. Ismail, M. Al-Shurman, I. F. Jafar, E. Alkhader, and M. F. Al-Mistarihi, "Performance evaluation of selective and adaptive heads clustering algorithms over wireless sensor networks," *Journal of Network and Computer Applications*, vol. 35, no. 6, pp. 2068 – 2080, 2012. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1084804512001841

[73] Y. Xu and W.-C. Lee, "On localized prediction for power efficient object tracking in sensor networks," in *23rd International Conference on Distributed Computing Systems Workshops, 2003. Proceedings.*, May 2003, pp. 434–439.

[74] Y. Xu, J. Winter, and W.-C. Lee, "Prediction-based strategies for energy saving in object tracking sensor networks," in *IEEE International Conference on Mobile Data Management, 2004. Proceedings. 2004*, 2004, pp. 346–357.

[75] M.-S. Huang and R. M. Narayanan, "Trilateration-based localization algorithm using the lemoine point formulation," *IETE Journal of Research*, vol. 60, no. 1, pp. 60–73, 2014. [Online]. Available: https://doi.org/10.1080/03772063.2014.890826

[76] Y.-S. Chen, S.-Y. Ann, and Y.-W. Lin, "Ve-mobicast: a variant-egg-based mobicast routing protocol for sensornets," *Wireless Networks*, vol. 14, no. 2, pp. 199–218, Apr 2008. [Online]. Available: https://doi.org/10.1007/s11276-006-9957-9

[77] Y.-S. Chen and Y.-J. Liao, "Hve-mobicast: a hierarchical-variant-egg-based mobicast routing protocol for wireless sensornets," in *IEEE Wireless Communications and Networking Conference, 2006. WCNC 2006.*, vol. 2, April 2006, pp. 697–702.

[78] M. Abdelkader, M. Hamdi, and N. Boudriga, "Voronoi-based sensor network engineering for target tracking using wireless sensor networks," in *2008 New Technologies, Mobility and Security*, Nov 2008, pp. 1–5.

[79] W.-P. Chen, J. C. Hou, and L. Sha, "Dynamic clustering for acoustic target tracking in wireless sensor networks," *IEEE Transactions on Mobile Computing*, vol. 3, no. 3, pp. 258–271, July 2004.

[80] J. Lee, M. T. A. Amin, and T. F. Abdelzaher, "Espresso: A data naming service for self-summarizing transport," in *14th Annual IEEE International Conference on Sensing, Communication, and Networking, SECON 2017, San Diego, CA, USA, June 12-14, 2017*. IEEE, 2017. [Online]. Available: https://doi.org/10.1109/SAHCN.2017.7964914 pp. 1–9.

[81] "iApollo server repository," https://bitbucket.org/jdlee6461/iapolloproducer.

[82] "iApollo client repository," https://bitbucket.org/jdlee6461/iapollo.

[83] T. Alsbou'i, M. Hammoudeh, Z. Bandar, and A. Nisbet, "An overview and classification of approaches to information extraction in wireless sensor networks," in *2011 Fifth International Conference on Sensor Technologies and Applications*, August 2011.

[84] M. L. Segal, F. P. Antonio, S. Elam, J. Erlenbach, and K. R. de Paolo, "Method and apparatus for automatic event detection in a wireless communication system," US Patent US6 124 810A, 09 26, 2000. [Online]. Available: https://patents.google.com/patent/US6124810A/en

[85] G. Werner-Allen, K. Lorincz, M. Ruiz, O. Marcillo, J. Johnson, J. Lees, and M. Welsh, "Deploying a wireless sensor network on an active volcano," *IEEE Internet Computing*, vol. 10, no. 2, pp. 18–25, March 2006.

[86] M. Bahrepour, N. Meratnia, and P. J. M. Havinga, "Sensor fusion-based event detection in wireless sensor networks," in *2009 6th Annual International Mobile and Ubiquitous Systems: Networking Services, MobiQuitous*, July 2009, pp. 1–8.

[87] S. Sardellitti, S. Barbarossa, and L. Pezzolo, "Distributed double threshold spatial detection algorithms in wireless sensor networks," in *2009 IEEE 10th Workshop on Signal Processing Advances in Wireless Communications*, June 2009, pp. 51–55.

[88] D. Chu, A. Deshpande, J. M. Hellerstein, and W. Hong, "Approximate data collection in sensor networks using probabilistic models," in *22nd International Conference on Data Engineering (ICDE'06)*, April 2006, pp. 48–48.

[89] D. Tulone and S. Madden, "An energy-efficient querying framework in sensor networks for detecting node similarities," in *Proceedings of the 9th ACM International Symposium on Modeling Analysis and Simulation of Wireless and Mobile Systems*, ser. MSWiM '06. New York, NY, USA: ACM, 2006. [Online]. Available: http://doi.acm.org/10.1145/1164717.1164768 pp. 191–300.

[90] M. Bahrepour, N. Meratnia, M. Poel, Z. Taghikhaki, and P. J. M. Havinga, "Distributed event detection in wireless sensor networks for disaster management," in *2010 International Conference on Intelligent Networking and Collaborative Systems*, Nov 2010, pp. 507–512.

[91] G.-S. Ahn, "Information-driven tracking and access control in wireless ad hoc and sensor networks," Ph.D. dissertation, Columbia University, 2009.

[92] B. Gedik, L. Liu, and P. S. Yu, "Asap: An adaptive sampling approach to data collection in sensor networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 18, no. 12, pp. 1766–1783, Dec 2007.

[93] H. Li, H. Yu, L. Li, and A. Liu, "On the cascading data collection mechanism in wireless sensor networks," in *2007 International Conference on Wireless Communications, Networking and Mobile Computing*, Sept 2007, pp. 2479–2482.

[94] R. Jurdak, P. Baldi, and C. V. Lopes, "Adaptive low power listening for wireless sensor networks," *IEEE Transactions on Mobile Computing*, vol. 6, no. 8, pp. 988–1004, Aug 2007.

[95] Y. Yao and J. Gehrke, "The cougar approach to in-network query processing in sensor networks," *SIGMOD Rec.*, vol. 31, no. 3, pp. 9–18, Sep. 2002. [Online]. Available: http://doi.acm.org/10.1145/601858.601861

[96] S. R. Madden, M. J. Franklin, J. M. Hellerstein, and W. Hong, "Tinydb: An acquisitional query processing system for sensor networks," *ACM Trans. Database Syst.*, vol. 30, no. 1, pp. 122–173, Mar. 2005. [Online]. Available: http://doi.acm.org/10.1145/1061318.1061322

[97] M. Umer, L. Kulik, and E. Tanin, "Optimizing query processing using selectivity-awareness in wireless sensor networks," *Computers, Environment and Urban Systems*, vol. 33, no. 2, pp. 79 – 89, 2009, distributed and mobile spatial computing. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0198971509000088

[98] L. Cheng, Y. Chen, C. Chen, and J. Ma, "Query-based data collection in wireless sensor networks with mobile sinks," in *Proceedings of the 2009 International Conference on Wireless Communications and Mobile Computing: Connecting the World Wirelessly*, ser. IWCMC '09. New York, NY, USA: ACM, 2009. [Online]. Available: http://doi.acm.org/10.1145/1582379.1582633 pp. 1157–1162.

[99] J.-Z. Sun, "An energy-efficient query processing algorithm for wireless sensor networks," in *Ubiquitous Intelligence and Computing*, F. E. Sandnes, Y. Zhang, C. Rong, L. T. Yang, and J. Ma, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 373–385.

[100] R. Khoury, T. Dawborn, B. Gafurov, G. Pink, E. Tse, Q. Tse, K. Almi'Ani, M. Gaber, U. Röhm, and B. Scholz, "Corona: Energy-efficient multi-query processing in wireless sensor networks," in *Database Systems for Advanced Applications*, H. Kitagawa, Y. Ishikawa, Q. Li, and C. Watanabe, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 416–419.

[101] B.-D. Lee, "Adaptive data dissemination protocol for wireless sensor networks," in *Security-Enriched Urban Computing and Smart Grid*, T.-h. Kim, A. Stoica, and R.-S. Chang, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 188–195.

[102] C.-H. Lee, C.-W. Chung, and S.-J. Chun, "Effective processing of continuous group-by aggregate queries in sensor networks," *J. Syst. Softw.*, vol. 83, no. 12, pp. 2627–2641, Dec. 2010. [Online]. Available: http://dx.doi.org/10.1016/j.jss.2010.08.049

[103] A. Rasheed and R. Mahapatra, "An energy-efficient hybrid data collection scheme in wireless sensor networks," in *2007 3rd International Conference on Intelligent Sensors, Sensor Networks and Information*, Dec 2007, pp. 703–708.

[104] W. Bu, A. Tang, and J. Han, "An analysis of decision-centric architectural design approaches," in *2009 ICSE Workshop on Sharing and Reusing Architectural Knowledge*, May 2009, pp. 33–40.

[105] X. Cui, Y. Sun, and H. Mei, "Towards automated solution synthesis and rationale capture in decision-centric architecture design," in *Seventh Working IEEE/IFIP Conference on Software Architecture (WICSA 2008)*, Feb 2008, pp. 221–230.

[106] U. van Heesch, V. P. Eloranta, P. Avgeriou, K. Koskimies, and N. Harrison, "Decision-centric architecture reviews," *IEEE Software*, vol. 31, no. 1, pp. 69–76, Jan 2014.

[107] J. H. Panchal, M. Gero Fernández, C. J. Paredis, J. K. Allen, and F. Mistree, "A modular decision-centric approach for reusable design processes," *Concurrent Engineering*, vol. 17, no. 1, pp. 5–19, 2009.

[108] A. Paschke, "A semantic rule and event driven approach for agile decision-centric business process management," *Towards a Service-Based Internet*, pp. 254–267, 2011.

[109] X. Feng, G. Richards, and B. Raheemi, "The road to decision-centric business intelligence," in *Business Intelligence and Financial Engineering, 2009. BIFE'09. International Conference on.* IEEE, 2009, pp. 514–518.

[110] M. W. Blenko, M. C. Mankins, and P. Rogers, "The decision-driven organization," *Harvard Business Review*, vol. 88, no. 6, pp. 54–62, 2010.

[111] J. Pecharich, S. Stathatos, B. Wright, A. Viswanathan, and K. Tan, "Mission-centric cyber security assessment of critical systems," in *AIAA SPACE 2016*, 2016, p. 5603.

[112] S. Jajodia, S. Noel, P. Kalapa, M. Albanese, and J. Williams, "Cauldron mission-centric cyber situational awareness with defense in depth," in *2011 - MILCOM 2011 Military Communications Conference*, Nov 2011, pp. 1339–1344.

[113] "Decision-driven resource managemen patent," http://www.google.com/patents/US20150142726.

[114] S. Liu and P. Zaraté, "Knowledge based decision support systems: A survey on technologies and application domains," in *Group Decision and Negotiation. A Process-Oriented View*, P. Zaraté, G. E. Kersten, and J. E. Hernández, Eds. Cham: Springer International Publishing, 2014, pp. 62–72.

[115] S. G. Vadlamudi, T. Chakraborti, Y. Zhang, and S. Kambhampati, "Proactive decision support using automated planning," *CoRR*, vol. abs/1606.07841, 2016. [Online]. Available: http://arxiv.org/abs/1606.07841

[116] L. Seligman, P. Lehner, K. Smith, C. Elsaesser, and D. Mattox, "Decision-centric information monitoring," *Journal of Intelligent Information Systems*, vol. 14, no. 1, pp. 29–50, 2000.

[117] A. Pourshahid, G. Richards, and D. Amyot, "Toward a goal-oriented, business intelligence decision-making framework," in *MCETECH*, 2011.

[118] J.-E. Kim, T. Abdelzaher, L. Sha, A. Bar-Noy, and R. Hobbs, "Sporadic Decision-centric Data Scheduling with Normally-off Sensors," in *Proc. of IEEE Int'l Real-Time Systems Symposium (RTSS)*, Dec. 2016.

[119] J.-E. Kim, T. Abdelzaher, L. Sha, A. Bar-Noy, R. Hobbs, and W. Dron, "On Maximizing Quality of Information for the Internet of Things: A Real-time Scheduling Perspective (Invited)," in *Proc. of IEEE Int'l Conference on Embedded and Real-Time Computing Systems and Applications*, Aug. 2016.

[120] B. Adelberg, H. Garcia-Molina, and B. Kao, "Applying update streams in a soft real-time database system," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD, 1995.

[121] K.-D. Kang, S. Son, J. Stankovic, and T. Abdelzaher, "A qos-sensitive approach for timeliness and freshness guarantees in real-time databases," in *Proceedings of the Euromicro Conference on Real-Time Systems*, ser. ECRTS, 2002.

[122] K.-D. Kang, S. H. Son, and J. A. Stankovic, "Managing deadline miss ratio and sensor data freshness in real-time databases," *IEEE Trans. on Knowl. and Data Eng.*, vol. 16, no. 10, pp. 1200–1216, Oct. 2004.

[123] M. Xiong, Q. Wang, and K. Ramamritham, "On earliest deadline first scheduling for temporal consistency maintenance," *Real-Time Systems*, vol. 40, no. 2, pp. 208–237, 2008.

[124] R. Cristescu, B. Beferull-Lozano, M. Vetterli, and R. Wattenhofer, "Network correlated data gathering with explicit communication: Np-completeness and algorithms," *IEEE/ACM Transactions on Networking*, pp. 41–54, 2006.

[125] S. Pattem, B. Krishnamachari, and R. Govindan, "The impact of spatial correlation on routing with compression in wireless sensor networks," *ACM Transactions on Sensor Networks (TOSN)*, vol. 4, no. 24, 2008.

[126] M. C. Vuran, Özgür B. Akan, and I. F. Akyildiz, "Spatio-temporal correlation: theory and applications for wireless sensor networks," *Computer Networks Journal (Elsevier)*, pp. 245–259, January 2004.

[127] H. Gupta, V. Navda, S. Das, and V.Chowdhary, "Efficient gathering of correlated data in sensor networks," *ACM Transactions on Sensor Networks (TOSN)*, vol. 4, no. 4, January 2008.

[128] C. Liu, K. Wu, and J. Pei, "An energy-efficient data collection framework for wireless sensor networks by exploiting spatiotemporal correlation," *IEEE Transaction on Parallel Distributed System*, pp. 1011–1023, 2007.

[129] C. C. Aggarwal and C. K. Reddy, Eds., *Data Clustering: Algorithms and Applications 1st ed.* Oxford: Chapman & Hall/CRC, 2013.

[130] "Athena Repository," https://github.com/jdlee6461/athena-framework.